

ĐẠI HỌC BÁCH KHOA HÀ NỘI



ĐỒ ÁN TỐT NGHIỆP

**Thiết kế hệ thống thử nghiệm (testbed) chuẩn IEEE
802.15.4 Ứng dụng trong đánh giá giao thức Zigbee**

Vũ Minh Đức

Duc.VM212776@sis.hust.edu.vn

Trần Văn Bắc

Bac.TV212700@sis.hust.edu.vn

Ngành Kỹ thuật Điều khiển và Tự động hóa

Giảng viên hướng dẫn: PGS.TS. Nguyễn Quốc Cường

Chữ ký của GVHD

Khoa: Tự động hóa

Trường: Điện - Điện tử

Hà Nội, 07/2025

NHIỆM VỤ
ĐỒ ÁN TỐT NGHIỆP

Họ và tên sinh viên: Vũ Minh Đức, Trần Văn Bắc

Khóa: K66

Trường: Điện – Điện tử

Ngành: KT ĐK & TĐH

1. Tên đề tài

Thiết kế hệ thống thử nghiệm (testbed) chuẩn IEEE 802.15.4 Ứng dụng trong đánh giá giao thức Zigbee

2. Nội dung đề tài

Thiết kế hệ thống thử nghiệm gồm cả phần cứng và phần mềm có chức năng theo dõi và phân tích các thông số lớp PHY và MAC chuẩn IEEE 802.15.4, triển khai hệ thống trên ứng dụng mạng Zigbee thực tế.

Vũ Minh Đức - Triển khai phần mềm máy tính trung tâm, các công việc bao gồm:

- Phân tích dữ liệu nhận được qua UART từ module log
- Xây dựng các API gửi lệnh điều khiển tới KIT và xử lý phản hồi
- Xây dựng server Flask và giao diện web
- Xây dựng database SQLite lưu trữ dữ liệu
- Tính toán PER và PLR từ các event trong database
- Công việc chung: Xây dựng kịch bản ứng dụng mạng Zigbee, viết hướng dẫn triển khai Testbed

Trần Văn Bắc - Triển khai phần mềm trên KIT, thiết kế phần cứng cho hệ thống, các công việc bao gồm:

- Xây dựng module log sử dụng RAIL
- Xây dựng module controller nhận và xử lý lệnh từ máy tính trung tâm
- Xây dựng kịch bản test BER để kiểm thử phần cứng
- Thiết kế mạch cứng cho testbed, tích hợp antenna lên mạch
- Công việc chung: Xây dựng kịch bản ứng dụng mạng Zigbee, viết hướng dẫn triển khai Testbed

3. Thời gian giao đề tài: 24/02/2025

4. Thời gian hoàn thành: 14/06/2025

Ngày 20 tháng 06 năm 2025

CÁN BỘ HƯỚNG DẪN

PGS.TS Nguyễn Quốc Cường

LỜI CẢM ƠN

Lời đầu tiên em xin chân thành cảm ơn Ban Giám hiệu Đại học Bách khoa Hà Nội, các thầy cô giáo thuộc chuyên ngành Kỹ thuật Điều khiển và Tự động hóa đã tạo điều kiện để em được trau dồi kiến thức, rèn luyện và phát triển trong suốt quá trình học tập tại đây. Em xin cảm ơn gia đình đã luôn động viên, giúp đỡ và luôn là nguồn động lực để em cố gắng học tập và phấn đấu.

Em xin bày tỏ lòng biết ơn sâu sắc đến thầy PGS.TS Nguyễn Quốc Cường - người đã giao đề tài và trực tiếp hướng dẫn, chỉ bảo, giúp đỡ em trong suốt quá trình làm đồ án tốt nghiệp. Trong thời gian này, thầy luôn dành thời gian trao đổi, đánh giá và đưa ra những góp ý quý báu giúp em có thể hoàn thành đồ án một cách tốt nhất. Được làm việc cùng thầy em không chỉ tiếp thu được rất nhiều kiến thức chuyên môn mà còn học hỏi được ở thầy phong cách làm việc kỉ luật, nghiêm túc.

Em cũng xin cảm ơn tất cả các anh chị, các bạn trong Smart Sensor Lab. Khoảng thời gian được làm việc tại đây đối với em thực sự tuyệt vời, mọi người không chỉ giúp đỡ, chỉ bảo em các kiến thức, kĩ năng mà còn cho em cảm giác đây như một gia đình thứ hai.

Trong quá trình thực hiện đồ án tốt nghiệp, không thể tránh khỏi những thiếu sót. Em rất mong nhận được những góp ý của các thầy cô để em có thể hoàn thiện tốt hơn trong tương lai.

Em xin chân thành cảm ơn !

Hà Nội, ngày 20 tháng 06 năm 2025

Sinh viên thực hiện

Vũ Minh Đức

TÓM TẮT ĐỒ ÁN

Đồ án tập trung giải quyết vấn đề xây dựng một hệ thống testbed hoàn chỉnh cho chuẩn mạng không dây IEEE 802.15.4, cho phép người dùng điều khiển, giám sát và phân tích các thông số trong mạng một cách trực quan.

Phương pháp thực hiện của đồ án là xây dựng một hệ thống theo kiến trúc client-server. Backend được phát triển bằng ngôn ngữ Python với framework Flask, đảm nhiệm việc xử lý logic, giao tiếp với các node phần cứng qua cổng serial và quản lý cơ sở dữ liệu SQLite. Frontend là một giao diện web tương tác với backend qua các API RESTful. Ngoài phần mềm, hệ thống cũng bao gồm cả phần cứng được thiết kế dựa theo SoC EFR32MG24 của Silicon Lab.

Kết quả đạt được là một hệ thống testbed hoạt động ổn định, đáp ứng đầy đủ các yêu cầu đặt ra. Giao diện web cho phép quản lý đồng thời nhiều node, thực hiện các thao tác mạng (tạo, gia nhập, rời mạng), điều khiển thiết bị và thực hiện các bài kiểm thử hiệu năng quan trọng như đánh giá Packet Error Rate cho từng thiết bị, đo thông số Bit Error Rate,... Tính năng phân tích sâu gói tin đã được hoàn thiện, cung cấp cái nhìn chi tiết về cấu trúc dữ liệu mạng.

Đồ án có tính thực tiễn cao, có thể ứng dụng trong giảng dạy, nghiên cứu và gỡ lỗi các hệ thống mạng Zigbee. Hướng phát triển trong tương lai bao gồm việc mở rộng hỗ trợ các giao thức khác (Thread, 6LoWPAN), tích hợp các công cụ trực quan hóa dữ liệu nâng cao, và tự động hóa các kịch bản kiểm thử.

MỤC LỤC

DANH MỤC KÝ HIỆU VÀ CHỮ VIẾT TẮT	iv
DANH MỤC HÌNH VẼ	v
DANH MỤC BẢNG BIỂU	vii
CHƯƠNG 1. GIỚI THIỆU CHUNG	1
1.1 Bối cảnh và lý do chọn đề tài	1
1.2 Mục tiêu	3
1.3 Phạm vi nghiên cứu	5
1.4 Cấu trúc báo cáo	5
CHƯƠNG 2. CƠ SỞ LÝ THUYẾT	7
2.1 Chuẩn truyền thông IEEE 802.15.4	7
2.1.1 Giới thiệu chung về IEEE 802.15.4	7
2.1.2 Mô hình giao thức và tầng được định nghĩa	8
2.1.3 Các kiến trúc mạng hỗ trợ	13
2.1.4 Các ứng dụng tiêu biểu của IEEE 802.15.4	15
2.2 Giao thức truyền thông không dây Zigbee	17
2.2.1 Tổng quan về Zigbee	17
2.2.2 Kiến trúc giao thức Zigbee	18
2.2.3 Định tuyến trong mạng Zigbee	23
2.3 RAIL(Radio Abstraction Interface Layer)	24
2.3.1 Tổng quan về RAIL	24
2.3.2 Cấu trúc của RAIL	25
2.3.3 Quy trình hoạt động của RAIL	26
2.4 SoC EFR32MG24 - Tham chiếu để thiết kế mạch cứng	28
2.4.1 Giới thiệu chung về EFR32MG24	28
2.4.2 Kiến trúc phần cứng EFR32MG24	28
2.5 Tổng quan về Inverted-F antenna và phối hợp trở kháng cho antenna	29
2.5.1 Inverted-F antenna	30
2.5.2 Phối hợp trở kháng cho antenna	30

CHƯƠNG 3. THIẾT KẾ HỆ THỐNG	32
3.1 Tổng quan hệ thống	32
3.1.1 Phần mềm nhúng trên kit	32
3.1.2 Phần mềm trên máy tính trung tâm	33
3.1.3 Giao diện và tương tác người dùng	34
3.1.4 Phần cứng cho hệ thống	34
3.2 Thiết kế phần mềm nhúng trên module EFR32MG24	35
3.2.1 Xây dựng khối tự động truyền các tham số sang máy tính trung tâm (Module Log)	35
3.2.2 Xây dựng khối nhận lệnh từ UART và thực thi lệnh	36
3.3 Xây dựng phần mềm trên máy tính trung tâm	37
3.3.1 Xây dựng khối nhận và xử lý dữ liệu qua UART	37
3.3.2 Xây dựng hệ thống cơ sở dữ liệu	39
3.3.3 Xây dựng server Flask	43
3.4 Xây dựng Engine giải mã các loại gói tin	45
3.5 Xây dựng giao diện hiển thị	57
3.5.1 Khái quát lại về luồng dữ liệu	57
3.5.2 Các chức năng của giao diện Web	58
3.6 Xây dựng phần cứng cho hệ thống Testbed	62
3.6.1 Vẽ sơ đồ nguyên lý và vẽ mạch in	62
3.6.2 Thiết kế antena cho mạch	63
3.6.3 Tính toán, mô phỏng giá trị các linh kiện cho mạch phối hợp trở kháng	65
 CHƯƠNG 4. KẾT QUẢ THỬ NGHIỆM	 66
4.1 Xây dựng kịch bản ứng dụng người dùng để kiểm thử hệ thống	66
4.1.1 Mô hình kịch bản ứng dụng	66
4.1.2 Xây dựng lưu đồ hoạt động cho các thành phần trong mạng	67
4.2 Tính toán PER và PLR từ các event trong database	70
4.2.1 PER	70
4.2.2 PLR	71
4.2.3 Đề xuất cách tính PER và PLR từ các sự kiện	71
4.3 Kết quả phần cứng thu được	73
4.3.1 Mạch in PCB	73

4.3.2	Kết quả thu được của Inverted-F antenna	74
4.3.3	Kết quả của mạch phối hợp trở kháng	77
4.4	Thử nghiệm hệ thống trong các điều kiện khác nhau	78
4.4.1	Môi trường thường không có vật cản	78
4.4.2	Môi trường có nhiều tín hiệu	79
4.5	Kết quả test Bit Error Rate(BER)	81
4.5.1	Bố trí hệ thống	81
4.5.2	Kết quả BER thu được	81
CHƯƠNG 4. KẾT LUẬN VÀ HƯỚNG PHÁT TRIỂN		83
5.1	Kết luận	83
5.2	Hướng phát triển	84
TÀI LIỆU THAM KHẢO		86
PHỤ LỤC		89
A	HƯỚNG DẪN NGƯỜI DÙNG SỬ DỤNG TESTBED	89
A.1	Hướng dẫn người dùng tích hợp module log và module controller vào firmware chính	89
A.1.1	Tích hợp module log vào firmware chính	89
A.1.2	Tích hợp module controller vào firmware chính	89
A.2	Hướng dẫn người dùng sử dụng giao diện web	90
A.2.1	Theo dõi thông tin log và gửi lệnh điều khiển	90
A.2.2	Sử dụng chức năng BER test	91

DANH MỤC KÝ HIỆU VÀ CHỮ VIẾT TẮT

IoT	Internet of Things
MAC	Medium Access Control
OSI	Open Systems Interconnection
LQI	Link Quality Indicator
PER	Packet Error Rate
BER	Bit Error Rate
PLR	Packet Loss Rate
RSSI	Received Signal Strength Indicator
UART	Universal Asynchronous Receiver-Transmitter
RAIL	Radio Abstraction Interface Layer
BPSK	Binary Phase Shift Keying
O-QPSK	Offset Quadrature Phase Shift Keying
DSSS	Direct Sequence Spread Spectrum
CSMA/CA	Carrier Sense Multiple Access with Collision Avoidance
PPDU	PHY Protocol Data Unit
SFD	Start of Frame Delimiter
PHR	PHY Header
PSDU	PHY Service Data Unit
MPDU	MAC Protocol Data Unit
MHR	MAC Header
FCS	Frame Check Sequence
AODV	Ad-hoc On-demand Distance Vector
ACK	Acknowledgment
MIC	Message Integrity Code

DANH MỤC HÌNH VẼ

Hình 1.1.	Kiến trúc tổng quát hệ thống IoT	1
Hình 2.1.	Cấu trúc phân lớp của IEEE 802.15.4 trong mô hình OSI	8
Hình 2.2.	Cấu trúc khung dữ liệu tầng PHY và MAC trong IEEE 802.15.4[1]	12
Hình 2.3.	Sơ đồ mạng hình cây phân cụm	15
Hình 2.4.	Mạng Zigbee mesh với Coordinator, Router và End Device	20
Hình 2.5.	Các loại kiến trúc mạng Zigbee	22
Hình 2.6.	Kiến trúc phần mềm RAIL với ngăn xếp tùy chỉnh	25
Hình 2.7.	Soc EFR32MG24	28
Hình 2.8.	Cấu trúc khối phần cứng module xGM240P sử dụng chip EFR32MG24	28
Hình 2.9.	Mô hình antenna và mạch phối hợp trở kháng	29
Hình 3.1.	Kiến trúc hệ thống	32
Hình 3.2.	Quy trình xử lý dữ liệu UART	39
Hình 3.3.	Kết quả giải mã tầng IEEE 802.15.4	50
Hình 3.4.	Kết quả giải mã tầng Zigbee Network	50
Hình 3.5.	Kết quả giải mã tầng Zigbee Security	51
Hình 3.6.	Kết quả giải mã Application Payload	51
Hình 3.7.	Kết quả giải mã gói ACK	52
Hình 3.8.	Kết quả giải mã gói tin Beacon Request	53
Hình 3.9.	Kết quả giải mã gói tin Beacon	54
Hình 3.10.	Kết quả giải mã gói tin Association Request	55
Hình 3.11.	Kết quả giải mã gói tin Association Response	56
Hình 3.12.	Kết quả giải mã gói tin Data Request	57
Hình 3.13.	Sơ đồ luồng xử lý ZigBee packet detail modal	60
Hình 3.14.	Sequence diagram mô tả quy trình thực hiện BER Test trong hệ thống	62
Hình 3.15.	Cấu trúc và kích thước antenna	64
Hình 3.16.	Sơ đồ nguyên lý mạch phối hợp trở kháng	65
Hình 4.1.	Sơ đồ mô hình kịch bản ứng dụng	66
Hình 4.2.	Lưu đồ hoạt động của Coordinator	67
Hình 4.3.	Lưu đồ hoạt động của End device	69
Hình 4.4.	Hình ảnh 3D của mạch in	73

Hình 4.5. Đồ thị S_{11} của antenna	74
Hình 4.6. Kết quả farfield từ mô phỏng	75
Hình 4.7. Kết quả đo được từ máy VNA	76
Hình 4.8. Kết quả đo được từ máy VNA (xanh) và từ mô phỏng trên CST (đỏ)	76
Hình 4.9. Kết quả mô phỏng mạch phối hợp trở kháng cho antenna	77
Hình 4.10. Thử nghiệm hệ thống trong môi trường thường không có vật cản	78
Hình 4.11. Kết quả đạt được trong môi trường ít nhiễu	78
Hình 4.12. Thử nghiệm hệ thống trong môi trường có nhiễu	79
Hình 4.13. Kết quả đạt được trong môi trường chủ động tạo nhiễu	80
Hình 4.14. Sơ đồ thiết lập để đánh giá BER	81
Hình 4.15. Test BER trên 2 kit EFR32MG24	81
Hình 4.16. Test BER trên 2 kit tự thiết kế	81
Hình 4.17. Test BER với kit gửi là kit tự thiết kế, kit nhận là EFR32MG24	82
Hình 4.18. Test BER với kit gửi là EFR32MG24, kit nhận là kit tự thiết kế	82
Hình A.1 Vị trí thêm các hàm module controller	90
Hình A.2 Giao diện chính hệ thống	90
Hình A.3 Giao diện hiển thị thông số gói tin	91
Hình A.4 Giao diện BER test	91

DANH MỤC BẢNG BIỂU

Bảng 2.1.	Các thông số kỹ thuật theo dải tần trong IEEE 802.15.4	8
Bảng 2.2.	So sánh các kiến trúc mạng trong IEEE 802.15.4	15
Bảng 2.3.	So sánh một số giao thức truyền thông không dây[2]	17
Bảng 2.4.	Các API thông dụng trong thư viện RAIL của Silicon Labs[3] .	27
Bảng 3.1.	Các sự kiện xảy ra trong chương trình	36
Bảng 3.2.	Cấu trúc bảng zigbee_packets	41
Bảng 3.3.	Cấu trúc bảng escan_data	42
Bảng 3.4.	Các loại Command Frame được hỗ trợ trong hệ thống	48
Bảng 3.5.	Các thông số về kích thước của antena	64

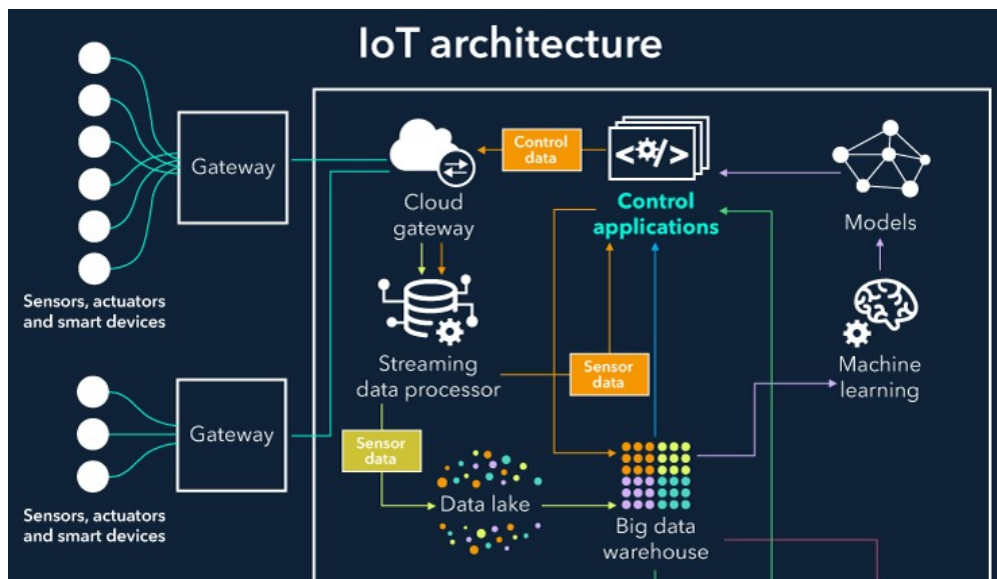
CHƯƠNG 1. GIỚI THIỆU CHUNG

Mở đầu chương

Chương 1 trình bày về bối cảnh, lý do chọn đề tài từ đó đưa ra mục tiêu để thiết kế hệ thống thử nghiệm.

1.1 Bối cảnh và lý do chọn đề tài

Trong thời đại công nghệ số và Cách mạng công nghiệp 4.0, công nghệ truyền thông không dây ngày càng đóng vai trò quan trọng trong việc xây dựng các hệ thống thông minh, tự động hóa và phân tán. Một trong những xu hướng nổi bật là sự phát triển mạnh mẽ của Internet of Things[4][5] – mạng lưới các thiết bị cảm biến, thu thập dữ liệu từ môi trường và truyền về các trung tâm xử lý để phân tích và đưa ra quyết định. Những ứng dụng IoT có thể được bắt gặp trong hầu hết các lĩnh vực, từ nông nghiệp, giám sát môi trường, nhà thông minh, y tế, quản lý năng lượng, cho đến các hệ thống tự động hóa công nghiệp. Trong đó, các mạng cảm biến không dây là một phần không thể thiếu, đóng vai trò như cánh tay nối dài giúp hệ thống IoT cảm nhận và phản hồi với môi trường.



Hình 1.1. Kiến trúc tổng quát hệ thống IoT

Các mạng cảm biến không dây bao gồm nhiều node cảm biến có khả năng tự tổ chức, hoạt động độc lập bằng pin, tương tác với nhau và truyền dữ liệu không dây đến node điều phối hoặc trung tâm xử lý. Để xây dựng một hệ thống như vậy đòi hỏi phải có những giao thức truyền thông vừa đáng tin cậy, tiêu thụ năng lượng thấp, lại vừa linh hoạt và mở rộng dễ dàng.

Trong số các tiêu chuẩn truyền thông không dây hiện nay, IEEE 802.15.4 đã được công nhận là một nền tảng quan trọng cho các hệ thống mạng cảm biến năng lượng

thấp. Đây là tiêu chuẩn được thiết kế riêng cho các hệ thống có yêu cầu cao về tiết kiệm năng lượng, độ trễ thấp, băng thông vừa phải và có khả năng hoạt động trong môi trường hạn chế tài nguyên. Tiêu chuẩn IEEE 802.15.4 quy định chi tiết cho tầng vật lý (PHY) và tầng điều khiển truy cập môi trường (MAC) – hai tầng cơ sở trong mô hình OSI, đóng vai trò kiểm soát truy cập kênh truyền và truyền dữ liệu hiệu quả. Ngoài ra, chuẩn này còn hỗ trợ các chế độ tiết kiệm năng lượng, cho phép thiết bị ngủ sâu trong thời gian dài mà vẫn duy trì khả năng kết nối. Với đặc điểm như vậy, IEEE 802.15.4 không chỉ được sử dụng như một giao thức độc lập mà còn là nền tảng cho nhiều giao thức lớp cao như Zigbee, WirelessHART, 6LoWPAN, Thread và nhiều hệ thống mạng khác. Nhờ đó, các thiết bị dựa trên chuẩn IEEE 802.15.4 đã và đang được ứng dụng rộng rãi trong thực tế, góp phần tạo nên các mạng cảm biến linh hoạt, hoạt động ổn định trong thời gian dài, đặc biệt ở những nơi không thể cung cấp nguồn điện liên tục.

Tuy nhiên, khi bước vào giai đoạn triển khai thực tế, rất nhiều yếu tố ảnh hưởng đến hiệu năng của các hệ thống truyền thông không dây mà không thể được mô phỏng đầy đủ bằng phần mềm giả lập. Những yếu tố như nhiễu nền, suy hao đường truyền, xung đột kênh, đa đường hay thậm chí các sai số phần cứng đều có thể ảnh hưởng đến các chỉ số như tỷ lệ lỗi bit (BER), tỷ lệ mất gói (PER), thời gian truyền trễ, hoặc chất lượng liên kết (LQI). Chính vì vậy, để nghiên cứu một cách chính xác và đánh giá hiệu năng thực sự của mạng cảm biến không dây, đặc biệt ở tầng MAC và PHY theo chuẩn IEEE 802.15.4, việc xây dựng một hệ thống testbed thực nghiệm là vô cùng cần thiết.

Testbed là một hệ thống kiểm thử thực nghiệm được thiết kế với mục đích cung cấp một môi trường thử nghiệm có kiểm soát, cho phép đánh giá, phân tích và xác thực hoạt động của các hệ thống phần cứng, phần mềm, giao thức truyền thông hoặc các giải pháp công nghệ mới trong điều kiện gần giống thực tế. Không giống như các phương pháp mô phỏng chỉ tái hiện hành vi hệ thống dựa trên các mô hình toán học và giả định lý tưởng, testbed vận hành trên nền tảng các thiết bị vật lý thật và dữ liệu thật, từ đó mang lại kết quả kiểm thử có độ tin cậy và tính thực tiễn cao hơn. Mục tiêu chính của testbed là cho phép người nghiên cứu và phát triển hệ thống quan sát hành vi của hệ thống trong môi trường thực, kiểm tra hiệu suất vận hành, phát hiện các điểm yếu hoặc lỗi phát sinh trong quá trình triển khai, và đồng thời tạo điều kiện để điều chỉnh, tối ưu hóa thiết kế một cách linh hoạt.

Ngoài ra, testbed còn đóng vai trò như một bước trung gian quan trọng trong quá trình chuyển giao công nghệ, từ giai đoạn nghiên cứu lý thuyết sang giai đoạn triển khai thực tế, giúp giảm thiểu rủi ro kỹ thuật và chi phí thử nghiệm quy mô lớn. Với khả năng tái cấu hình linh hoạt, testbed có thể hỗ trợ kiểm thử nhiều kịch bản khác nhau, từ những tình huống đơn giản đến những môi trường phức tạp như mạng không dây đa điểm, hệ thống nhúng thời gian thực, mạng cảm biến quy mô lớn hoặc môi trường công nghiệp có nhiễu cao. Trong bối cảnh phát triển nhanh chóng của các công nghệ như IoT, 5G, trí tuệ nhân tạo và mạng truyền thông thế hệ mới, testbed ngày càng trở

thành công cụ không thể thiếu trong các hoạt động nghiên cứu, thử nghiệm và triển khai ứng dụng thực tiễn, đồng thời là nền tảng vững chắc để kiểm chứng các ý tưởng sáng tạo và định hướng cải tiến hệ thống trong tương lai.

Một testbed hoàn chỉnh và hoạt động hiệu quả cần có sự phối hợp của nhiều thành phần phần cứng và phần mềm. Việc thiết kế testbed đòi hỏi sự hiểu biết sâu sắc về kiến trúc hệ thống, mục tiêu kiểm thử cũng như các chỉ tiêu đánh giá. Dưới đây là các thành phần cơ bản và quan trọng của một testbed tiêu chuẩn:

- **Thiết bị đầu cuối:** Đây là những thiết bị được đưa vào kiểm thử trong testbed, có thể là các cảm biến, thiết bị truyền tín hiệu, node Zigbee, thiết bị IoT, v.v. Mỗi thiết bị thường được lập trình để thực hiện một tác vụ cụ thể như gửi hoặc nhận dữ liệu, đo lường tín hiệu, phản hồi điều khiển hoặc thực thi các thuật toán giao thức. Việc kiểm thử sẽ đánh giá khả năng hoạt động của thiết bị dưới các điều kiện và cấu hình khác nhau.
- **Máy tính trung tâm:** Máy tính trung tâm đóng vai trò điều phối toàn bộ quá trình kiểm thử trong testbed. Nó chịu trách nhiệm lập trình cho các thiết bị đầu cuối, khởi tạo các kịch bản kiểm thử, theo dõi trạng thái hoạt động và thu thập dữ liệu từ các thiết bị. Ngoài ra, máy tính trung tâm cũng có thể xử lý dữ liệu kiểm thử và hiển thị kết quả cho người dùng theo thời gian thực.
- **Phần mềm điều khiển và giám sát:** Đây là một phần mềm chạy trên máy tính trung tâm nhằm cung cấp giao diện người dùng để dễ dàng quản lý, cấu hình và điều khiển các phiên kiểm thử. Phần mềm này thường bao gồm các chức năng như tải chương trình cho thiết bị, chọn kịch bản kiểm thử, thu thập dữ liệu đo lường, xuất báo cáo kết quả, và có thể hỗ trợ lập lịch kiểm thử tự động.
- **Cơ sở dữ liệu hoặc bộ lưu trữ kết quả:** Dữ liệu đo lường và kết quả kiểm thử sẽ được ghi lại và lưu trữ để phân tích sau. Hệ thống lưu trữ này có thể là file log, cơ sở dữ liệu cục bộ hoặc hệ thống lưu trữ từ xa. Các thông số được lưu bao gồm tỷ lệ lỗi, độ trễ truyền, cường độ tín hiệu, công suất tiêu thụ, v.v., giúp đánh giá hiệu suất thiết bị một cách toàn diện.

Những lý do trên đã đặt ra nhu cầu cần xây dựng một nền tảng testbed phục vụ cho việc nghiên cứu và thử nghiệm các hệ thống mạng cảm biến không dây theo chuẩn IEEE 802.15.4.

1.2 Mục tiêu

Mục tiêu chính của đề án là xây dựng một hệ thống thử nghiệm (testbed) mạng cảm biến không dây sử dụng chuẩn IEEE 802.15.4. Cụ thể sẽ có các kịch bản test hiệu suất truyền nhận của phần cứng và các kịch bản test phần mềm ứng dụng của người dùng. Với việc triển khai testbed, đề án không chỉ nhằm kiểm thử tính năng truyền thông của

mạng mà còn hỗ trợ phân tích chi tiết các thông số ở lớp MAC và PHY – hai lớp then chốt trong mô hình truyền thông OSI.

Về mặt phần cứng, hệ thống testbed sẽ được xây dựng gồm các nút cảm biến tự thiết kế lại dựa trên cấu trúc vi mạch EFR32MG24 của hãng Silicon Labs – một trong những dòng SoC nổi bật với khả năng tích hợp cao, tiết kiệm năng lượng và hỗ trợ truyền thông không dây chuẩn IEEE 802.15.4. Đây là kit phát triển chuyên dụng dành cho các ứng dụng Zigbee, Thread, Bluetooth Low Energy, và các giao thức mạng không dây công suất thấp khác. EFR32MG24 được tích hợp bộ thu phát RF công suất thấp nhưng có hiệu năng cao, hỗ trợ đầy đủ các chức năng truyền và nhận dữ liệu theo chuẩn IEEE 802.15.4.

Mỗi node trong hệ thống sẽ được cấp nguồn độc lập và giao tiếp với máy tính trung tâm qua UART, cho phép truyền nhận dữ liệu điều khiển cũng như trích xuất các thông tin nội bộ. Các node sẽ được bố trí ở những vị trí cố định trong phòng thí nghiệm, mô phỏng các điều kiện truyền thông thực tế trong môi trường trong nhà, bao gồm cả các yếu tố gây nhiễu như vật cản, khoảng cách truyền và ảnh hưởng của phản xạ tín hiệu. Ngoài ra, trong tương lai có thể mở rộng thêm các cảm biến môi trường (nhiệt độ, độ ẩm, ánh sáng,...) vào các node để mô phỏng đầy đủ hơn một mạng cảm biến thực thụ.

Về mặt phần mềm, máy tính trung tâm đóng vai trò là bộ điều khiển chính và giao diện tương tác với toàn bộ hệ thống testbed. Trên máy tính này, một phần mềm giám sát chuyên dụng sẽ được phát triển nhằm cung cấp đầy đủ các tính năng cần thiết để vận hành và quan sát hoạt động của mạng Zigbee.

Trước hết, phần mềm sẽ hiển thị các kịch bản để người dùng có thể sử dụng để test hệ thống. Cụ thể, sẽ có 2 kịch bản chính: kịch bản kiểm thử phần cứng và kịch bản mô tả ứng dụng người dùng. Các kịch bản này được nạp xuống các kit tương ứng thông qua công cụ Simplicity Studio 5 của Silicon Lab.

Bên cạnh đó, phần mềm cũng cho phép người dùng hiệu chỉnh các thông số truyền thông một cách linh hoạt như: công suất phát (TX Power), khoảng thời gian giữa các gói tin, cơ chế truyền (đơn tuyến, quảng bá), hoặc thậm chí cả các tham số của tầng MAC như thời gian backoff, số lần truyền lại tối đa.

Một trong những điểm mạnh của hệ thống phần mềm là khả năng theo dõi và ghi nhận các thông số hoạt động ở lớp MAC và PHY trong thời gian thực. Phần mềm sẽ liên tục thu thập dữ liệu từ các node bao gồm: số lượng gói tin đã truyền và nhận, tỷ lệ lỗi bit (BER), tỷ lệ lỗi gói (PER), cường độ tín hiệu nhận được (RSSI), số lần truyền lại,... Các thông tin này sẽ được hiển thị dưới dạng biểu đồ, bảng thống kê và nhật ký hoạt động, giúp người dùng dễ dàng đánh giá hiệu năng mạng trong từng kịch bản thử nghiệm cụ thể.

1.3 Phạm vi nghiên cứu

Đề tài tập trung vào việc xây dựng và triển khai một hệ thống testbed phục vụ mục đích kiểm thử và đánh giá hiệu năng chuẩn IEEE 802.15.4, ứng dụng cho giao thức Zigbee. Phạm vi nghiên cứu chủ yếu xoay quanh việc thiết kế, lập trình và đo lường các thông số truyền thông ở cấp độ thấp, nhằm cung cấp cái nhìn thực nghiệm về hoạt động của hệ thống mạng cảm biến.

Trong khuôn khổ này, nghiên cứu chỉ sử dụng chuẩn truyền thông IEEE 802.15.4 kết hợp với giao thức Zigbee, phù hợp với các ứng dụng đòi hỏi tiêu thụ năng lượng thấp, phạm vi truyền ngắn và yêu cầu độ tin cậy cao. Các lớp mạng cao hơn như lớp ứng dụng hoặc bảo mật Zigbee nâng cao sẽ không được đề cập chi tiết trong phạm vi đề tài.

Phần mềm và phần cứng trong hệ thống testbed được thiết kế để thu thập và theo dõi các chỉ số kỹ thuật ở tầng vật lý và tầng liên kết dữ liệu, bao gồm: RSSI (cường độ tín hiệu nhận được), LQI (chỉ số chất lượng liên kết) BER (tỉ lệ lỗi bit), PER (tỉ lệ lỗi gói tin), độ trễ truyền và mức tiêu thụ năng lượng. Những thông số này sẽ được ghi nhận và hiển thị thông qua một phần mềm điều khiển trung tâm chạy trên máy tính.

Về phần cứng, chúng được thiết kế dựa trên SoC EFR32MG24 của Silicon Labs làm node cảm biến. Mỗi node được kết nối trực tiếp với máy tính trung tâm qua giao tiếp UART, cho phép truyền nhận dữ liệu và cập nhật firmware kiểm thử.

1.4 Cấu trúc báo cáo

Báo cáo được chia thành 4 chương chính, trình bày theo trình tự từ lý do chọn đề tài đến kết quả thực nghiệm, cụ thể như sau:

- Chương 1 – Bối cảnh và lý do chọn đề tài: Trình bày tổng quan về nhu cầu kiểm thử mạng trong thực tế, lý do lựa chọn đề tài, mục tiêu và phạm vi nghiên cứu của đề án.
- Chương 2 – Cơ sở lý thuyết: Giới thiệu tổng quan về chuẩn IEEE 802.15.4, giao thức mạng Zigbee, kiến trúc của RAIL và cơ sở để thiết kế phần cứng dựa trên EFR32MG24.
- Chương 3 – Phương pháp luận: Mô tả cách thiết kế các phần của hệ thống: phần mềm trên kit, phần mềm trên máy tính trung tâm, giao diện hiển thị và thiết kế phần cứng.
- Chương 4 – Kết quả thực nghiệm: Trình bày về kịch bản ứng dụng người dùng để thử nghiệm hệ thống, kịch bản kiểm thử phần cứng, các thông số liên quan tính toán được.

Kết luận chương

Chương 1 đã trình bày bối cảnh nghiên cứu, nhu cầu kiểm thử hiệu suất mạng không dây trong các ứng dụng thực tế, cũng như lý do chọn đề tài. Qua đó, đề án xác định rõ mục tiêu xây dựng một hệ thống thử nghiệm chuẩn IEEE 802.15.4 có khả năng đo lường các chỉ số ở lớp vật lý và MAC. Nội dung chương cũng giới hạn phạm vi nghiên cứu phù hợp với điều kiện phần cứng và thời gian thực hiện, làm cơ sở cho các chương tiếp theo về lý thuyết, phương pháp và thực nghiệm.

CHƯƠNG 2. CƠ SỞ LÝ THUYẾT

Mở đầu chương

Chương 2 trình bày các kiến thức và lý thuyết nền tảng về chuẩn truyền thông IEEE 802.15.4, giao thức Zigbee, kiến trúc và quy trình hoạt động của RAIL, lý thuyết về antena trong thiết kế phần cứng. Những lý thuyết này là cơ sở cho những thiết kế sau.

2.1 Chuẩn truyền thông IEEE 802.15.4

2.1.1 Giới thiệu chung về IEEE 802.15.4

Chuẩn IEEE 802.15.4[6] là một tiêu chuẩn kỹ thuật được phát triển bởi IEEE (Institute of Electrical and Electronics Engineers) nhằm mục tiêu cung cấp một lớp vật lý (PHY) và lớp điều khiển truy cập môi trường (MAC) tối ưu cho các ứng dụng truyền thông không dây tầm ngắn, tốc độ thấp và tiêu thụ năng lượng cực thấp. Đây là một trong những tiêu chuẩn nền tảng quan trọng cho các giao thức mạng tầng trên như Zigbee, 6LoWPAN, WirelessHART, Thread, MiWi và các giao thức truyền thông không dây khác được ứng dụng rộng rãi trong lĩnh vực IoT, nhà thông minh, công nghiệp thông minh, nông nghiệp chính xác, và các hệ thống mạng cảm biến không dây.

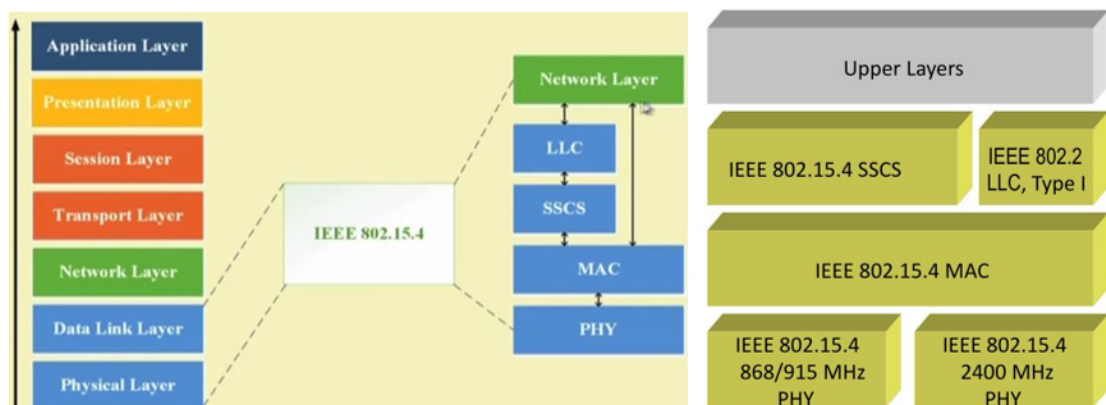
IEEE 802.15.4 được công bố lần đầu tiên vào năm 2003 và đã trải qua nhiều lần cập nhật, hoàn thiện để thích ứng với những thay đổi nhanh chóng trong công nghệ không dây. Mỗi phiên bản cập nhật đều bổ sung các tính năng mới nhằm cải thiện hiệu suất truyền thông, mở rộng dải tần hoạt động, tăng độ tin cậy, mở rộng phạm vi ứng dụng và tăng cường khả năng bảo mật. Tiêu chuẩn này tập trung chủ yếu vào hai lớp trong mô hình OSI: lớp vật lý và lớp liên kết dữ liệu (Data Link Layer – cụ thể là MAC sublayer), nhằm đơn giản hóa thiết kế phần cứng và phần mềm của các thiết bị đầu cuối.

Một trong những điểm mạnh nổi bật của IEEE 802.15.4 là khả năng hỗ trợ tiêu thụ năng lượng thấp. Thiết kế này đặc biệt phù hợp với các thiết bị sử dụng nguồn năng lượng hạn chế như pin hoặc năng lượng thu được từ môi trường. Các cơ chế như chế độ ngủ sâu, truyền dữ liệu theo khung thời gian, và cơ chế điều phối mạng cho phép giảm thiểu đáng kể năng lượng tiêu thụ mà vẫn duy trì độ tin cậy và khả năng truyền nhận dữ liệu ổn định trong môi trường không dây phức tạp.

Về mặt kỹ thuật, IEEE 802.15.4 hỗ trợ nhiều dải tần số khác nhau trên toàn cầu bao gồm 868 MHz (châu Âu), 915 MHz (Bắc Mỹ), và đặc biệt là băng tần ISM 2.4 GHz được sử dụng rộng rãi trên toàn thế giới. Tùy theo dải tần, tốc độ truyền dữ liệu có thể dao động từ 20 kbps (868 MHz) đến 250 kbps (2.4 GHz). Dù tốc độ thấp hơn nhiều so với Wi-Fi hay Bluetooth, nhưng chuẩn này vẫn rất phù hợp cho các ứng dụng truyền dữ liệu không yêu cầu băng thông lớn mà yêu cầu độ tin cậy và ổn định cao như giám sát môi trường, thu thập dữ liệu cảm biến, truyền trạng thái thiết bị,...

2.1.2 Mô hình giao thức và tầng được định nghĩa

Chuẩn IEEE 802.15.4 được thiết kế theo mô hình phân tầng, tuân theo nguyên lý của mô hình OSI, tuy nhiên chỉ tập trung định nghĩa và tiêu chuẩn hóa hai tầng dưới cùng là tầng vật lý và tầng điều khiển truy cập môi trường. Đây là hai tầng then chốt chịu trách nhiệm về truyền dẫn dữ liệu không dây giữa các thiết bị trong mạng cảm biến và mạng IoT công suất thấp. Các tầng cao hơn như mạng (Network Layer), vận chuyển (Transport Layer) hay ứng dụng (Application Layer) sẽ do các giao thức tầng trên như Zigbee, Thread hoặc 6LoWPAN định nghĩa và đảm nhận. Cấu trúc phân tầng rõ ràng này giúp cho IEEE 802.15.4 trở thành một nền tảng linh hoạt để phát triển các giao thức truyền thông không dây theo yêu cầu riêng biệt của từng ứng dụng.



Hình 2.1. Cấu trúc phân lớp của IEEE 802.15.4 trong mô hình OSI

2.1.2.1 Tầng vật lý

Tầng vật lý trong IEEE 802.15.4 có nhiệm vụ đảm bảo việc truyền nhận bit dữ liệu thô qua môi trường truyền không dây[7]. Nó chịu trách nhiệm mã hóa, giải mã tín hiệu, điều chế, phát và thu sóng vô tuyến, xác định tốc độ dữ liệu, dải tần hoạt động, độ rộng băng thông và công suất phát. Tầng này hỗ trợ ba dải tần chính: 868 MHz, 915MHz và 2.4 GHz.

Bảng 2.1. Các thông số kỹ thuật theo dải tần trong IEEE 802.15.4

Băng tần	Khu vực sử dụng	Kênh	Tốc độ dữ liệu	Điều chế
868 MHz	Châu Âu	1	20 kbps	BPSK
915 MHz	Bắc Mỹ	10	40 kbps	BPSK
2.4 GHz	Toàn cầu	16	250 kbps	O-QPSK + DSSS

Tầng vật lý trong chuẩn IEEE 802.15.4 không chỉ đơn thuần đảm nhận nhiệm vụ truyền và nhận bit dữ liệu qua môi trường không dây mà còn cung cấp một loạt các chức năng và dịch vụ hỗ trợ thiết yếu nhằm đảm bảo tính tin cậy, hiệu quả và khả năng thích ứng linh hoạt của hệ thống mạng không dây. Những dịch vụ này đặc biệt quan

trọng trong bối cảnh các hệ thống mạng cảm biến không dây hoặc mạng IoT hiện đại yêu cầu tính năng tự tổ chức, tự thích nghi và tiêu thụ năng lượng thấp.

Một trong những dịch vụ quan trọng đầu tiên do tầng PHY cung cấp là quét kênh. Trong môi trường mạng không dây, có nhiều kênh truyền khác nhau tương ứng với các tần số riêng biệt trong dải tần cho phép. Thiết bị cần chọn một kênh phù hợp để truyền thông tin – sao cho giảm thiểu xung đột với các thiết bị khác và tránh nhiễu. Quá trình quét kênh cho phép thiết bị rà soát các kênh khả dụng để xác định kênh nào đang rảnh, kênh nào bị chiếm dụng hoặc nhiễu nhiều. Điều này hỗ trợ cho việc thiết lập mạng ban đầu, tái định tuyến khi có thay đổi trong môi trường mạng, hoặc trong các trường hợp mạng tự tổ chức và mở rộng. Quá trình này đặc biệt hữu ích trong các môi trường có mật độ thiết bị cao như thành phố thông minh, nơi nhiều thiết bị cùng hoạt động trong dải tần 2.4 GHz.

Tiếp theo, phát hiện năng lượng tín hiệu là một dịch vụ được tích hợp trong PHY nhằm đánh giá mức năng lượng hiện tại tại một kênh nhất định. Điều này không chỉ hữu ích trong quá trình quét kênh mà còn trong các thuật toán tránh va chạm như CSMA/CA (Carrier Sense Multiple Access with Collision Avoidance). Khi thiết bị cần truyền dữ liệu, nó có thể thực hiện phát hiện năng lượng để đánh giá xem kênh có đang được sử dụng hay không. Nếu mức năng lượng cao, có thể kết luận rằng kênh đang bận hoặc có nhiễu, thiết bị sẽ trì hoãn việc truyền để tránh va chạm. Tính năng này góp phần quan trọng trong việc tiết kiệm năng lượng và duy trì hiệu quả truyền dẫn trong mạng.

LQI (Link Quality Indicator) – chỉ số chất lượng liên kết – là một chức năng nổi bật khác của tầng PHY. LQI là một giá trị định lượng, thường được biểu diễn dưới dạng số nguyên trong một dải nhất định (ví dụ 0–255), phản ánh chất lượng tín hiệu của gói tin được nhận. Nó có thể được tính toán dựa trên nhiều yếu tố như tỷ lệ lỗi bit (BER), cường độ tín hiệu nhận (RSSI), hoặc mức độ nhiễu và biến dạng. LQI đặc biệt hữu ích trong các mạng cảm biến dạng lưới, nơi thông tin thường được truyền qua nhiều thiết bị trung gian. Với LQI, giao thức tầng trên (như tầng mạng hoặc tầng MAC) có thể lựa chọn tuyến truyền tốt nhất – tức là tuyến có thiết bị trung gian với chất lượng liên kết cao nhất, từ đó tối ưu độ tin cậy và giảm số lần truyền lại do lỗi. Ngoài ra, LQI còn hỗ trợ việc xác định các thiết bị gần kề đáng tin cậy, góp phần vào việc thiết lập bảng định tuyến và phát hiện các nút không ổn định trong mạng.

Bên cạnh các chức năng trên, tầng PHY của IEEE 802.15.4 còn được thiết kế để hoạt động ổn định trong các môi trường có nhiễu và can thiệp vô tuyến nhờ vào việc áp dụng kỹ thuật điều chế và trải phổ tiên tiến. Cụ thể, đối với băng tần 2.4 GHz – là băng tần phổ biến và được sử dụng trên phạm vi toàn cầu – tầng PHY sử dụng kỹ thuật trải phổ trực tiếp DSSS (Direct Sequence Spread Spectrum) kết hợp với O-QPSK (Offset Quadrature Phase Shift Keying).

DSSS[8] là kỹ thuật mã hóa tín hiệu trước khi truyền bằng cách nhân tín hiệu dữ liệu với một chuỗi bit giả ngẫu nhiên có tốc độ cao gọi là “chip”. Việc này làm trải phổ tín hiệu gốc ra một dải tần rộng hơn, qua đó giúp chống lại nhiễu hẹp băng và tăng khả năng phát hiện tín hiệu trong môi trường nhiễu nhiều. DSSS còn hỗ trợ việc giảm xác suất mất gói do xung đột kênh, một vấn đề phổ biến trong môi trường đông thiết bị không dây như trong các ứng dụng nhà thông minh, nông nghiệp thông minh, hay hệ thống giám sát công nghiệp.

O-QPSK[9] là một dạng điều chế pha, trong đó các tín hiệu I và Q (In-phase và Quadrature-phase) được truyền lệch nhau một nửa chu kỳ chip, từ đó làm giảm biến động biên độ tín hiệu và giảm khả năng gây ra nhiễu phổ rộng. Khi kết hợp với DSSS, O-QPSK giúp tín hiệu truyền ổn định, duy trì độ chính xác và khả năng giải điều chế cao ngay cả khi cường độ tín hiệu yếu hoặc trong môi trường có nhiễu phản xạ. Nhờ đó, việc sử dụng O-QPSK + DSSS cho tầng PHY trong băng tần 2.4 GHz mang lại hiệu năng vượt trội, đặc biệt phù hợp với các ứng dụng yêu cầu độ tin cậy cao, tốc độ truyền vừa phải (250 kbps), và khả năng hoạt động ổn định trong môi trường phức tạp.

2.1.2.2 Tầng MAC

Tầng MAC trong giao thức IEEE 802.15.4[10] chịu trách nhiệm kiểm soát quyền truy cập vào môi trường truyền thông không dây, là cầu nối giữa tầng vật lý và tầng mạng. Tầng này giữ vai trò then chốt trong việc đảm bảo truyền dữ liệu hiệu quả và tiết kiệm năng lượng, điều đặc biệt quan trọng đối với các ứng dụng mạng cảm biến không dây và IoT, nơi thiết bị thường hoạt động bằng pin trong thời gian dài.

Một trong những chức năng quan trọng của tầng MAC là kiểm soát truy cập kênh truyền thông. Nó sử dụng kỹ thuật đa truy cập phát hiện sóng mang tránh va chạm (CSMA/CA – Carrier Sense Multiple Access with Collision Avoidance). Trước khi gửi dữ liệu, thiết bị sẽ nghe kênh để xác định xem có thiết bị khác đang truyền hay không. Nếu kênh đang rảnh, nó sẽ truyền; nếu kênh đang bận, thiết bị sẽ chờ một khoảng thời gian ngẫu nhiên (backoff) rồi thử lại. Điều này giúp giảm nguy cơ va chạm giữa các thiết bị truyền đồng thời trên cùng một kênh.

Tầng MAC xử lý ba loại frame chính: Data Frame để truyền dữ liệu, Acknowledgment Frame để xác nhận dữ liệu đã nhận, và MAC Command Frame để điều khiển và quản lý hoạt động mạng. Ngoài ra, nó cũng có khả năng gửi hoặc không gửi khung xác nhận tùy theo cài đặt, giúp cân bằng giữa độ tin cậy và tiêu thụ năng lượng.

Trong chế độ hoạt động có beacon (Beacon-enabled mode), tầng MAC triển khai một cấu trúc gọi là superframe. Một siêu khung bắt đầu bằng một beacon được gửi từ Coordinator nhằm đồng bộ thời gian cho toàn mạng. Siêu khung chia thời gian truyền thành các phần: Beacon, Contention Access Period (CAP), Contention-Free Period (CFP), và Inactive Period. Trong CAP, các thiết bị sử dụng CSMA/CA để truy cập kênh; trong CFP, các thiết bị có thể được cấp thời gian truy cập riêng (GTS –

Guaranteed Time Slot); còn Inactive Period là khoảng thời gian các thiết bị có thể tắt radio để tiết kiệm năng lượng.

Tầng MAC cũng đảm nhiệm việc quản lý kết nối của thiết bị với mạng thông qua quá trình Association và Disassociation. Khi một thiết bị muốn gia nhập mạng, nó gửi một yêu cầu Association tới Coordinator. Nếu được chấp nhận, Coordinator sẽ gửi phản hồi cho phép thiết bị trở thành một phần của mạng. Khi thiết bị muốn rời khỏi mạng, nó gửi thông báo Disassociation để giải phóng tài nguyên và duy trì tính nhất quán trong bảng thành viên của mạng.

Bảo mật là một phần tích hợp trong tầng MAC của IEEE 802.15.4. Giao thức hỗ trợ các cơ chế mã hóa dữ liệu bằng thuật toán AES-128, kiểm tra toàn vẹn dữ liệu bằng mã MIC (Message Integrity Code)[11], và xác thực nguồn gửi để chống giả mạo. Tùy theo cấu hình, các khung MAC có thể bao gồm các trường bảo mật giúp bảo vệ dữ liệu khỏi bị nghe lén, thay đổi hoặc tái sử dụng bất hợp pháp.

Một đặc điểm quan trọng của tầng MAC trong IEEE 802.15.4 là khả năng hỗ trợ hoạt động năng lượng thấp. Bằng cách kết hợp cấu trúc siêu khung, khả năng tắt radio trong thời gian không hoạt động, và quản lý truy cập kênh một cách hiệu quả, tầng MAC giúp các thiết bị hoạt động tiết kiệm năng lượng nhưng vẫn duy trì kết nối mạng ổn định. Điều này làm cho IEEE 802.15.4 trở thành một chuẩn truyền thông lý tưởng cho các thiết bị IoT công suất thấp, như cảm biến môi trường, thiết bị đeo, và mạng nhà thông minh.

2.1.2.3 Cấu trúc khung dữ liệu

Tầng vật lý (PHY) của IEEE 802.15.4 chịu trách nhiệm xử lý tín hiệu ở mức phần cứng, bao gồm điều chế, truyền và nhận dữ liệu qua kênh vô tuyến. Gói dữ liệu do tầng PHY xử lý được gọi là PPDU (PHY Protocol Data Unit). Cấu trúc của PPDU bao gồm bốn thành phần chính: Preamble, Start of Frame Delimiter (SFD), PHY Header (PHR) và PHY Service Data Unit (PSDU), trong đó PSDU chính là khung MAC do tầng trên gửi xuống.

Thành phần đầu tiên là Preamble, có độ dài 32 bit, được sử dụng để đồng bộ tần số và thời gian giữa máy phát và máy thu trước khi dữ liệu thực sự được truyền đi. Preamble thường là một chuỗi các bit '0', cho phép máy thu nhận biết tín hiệu đến và đồng bộ tốc độ bit.

Tiếp theo là SFD (Start of Frame Delimiter), dài 8 bit. Đây là một chuỗi bit xác định điểm bắt đầu chính xác của khung dữ liệu, giúp máy thu phân biệt rõ ràng đâu là phần header và payload.

Sau đó là PHY Header (PHR), dài 8 bit, chứa thông tin điều khiển liên quan đến khung dữ liệu, chẳng hạn như độ dài của phần dữ liệu (PSDU), cho phép tầng PHY biết được số byte cần tiếp nhận.

Cuối cùng là PSDU (PHY Service Data Unit), là phần dữ liệu thực tế được truyền

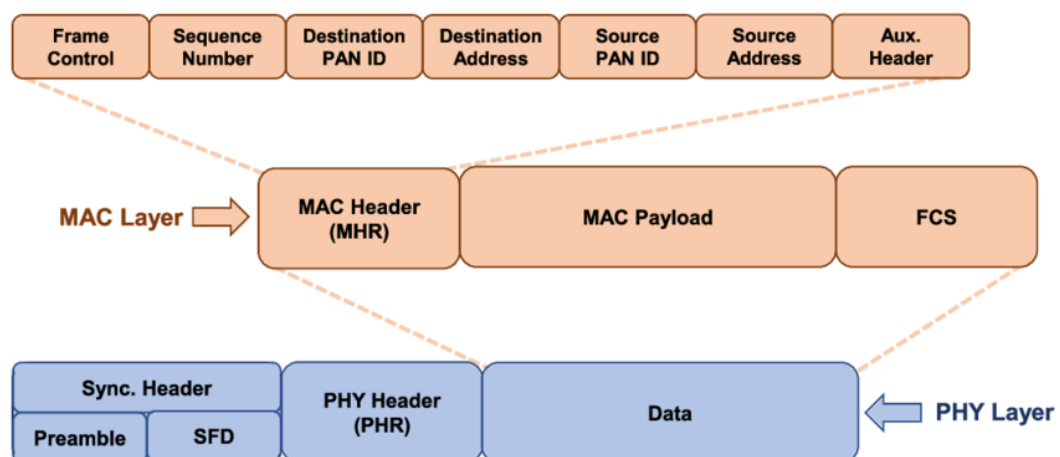
đi. Nó bao gồm toàn bộ khung MAC (MAC Header + MAC Payload + FCS) được tạo ra từ tầng MAC. Tầng PHY không xử lý nội dung của PSDU mà chỉ truyền và nhận nó một cách trung lập. Tầng MAC trong IEEE 802.15.4 quản lý việc truy cập kênh truyền, đảm bảo các thiết bị không truyền chồng lấn lên nhau, đồng thời kiểm soát quá trình truyền/nhận gói tin giữa các node trong mạng. Dữ liệu do tầng MAC tạo ra được gọi là MAC Protocol Data Unit (MPDU) và bao gồm ba phần chính: MAC Header (MHR), MAC Payload và Frame Check Sequence (FCS).

MAC Header (MHR) là phần đầu tiên của khung MAC, chứa các trường điều khiển như sau:

- Frame Control (16 bit): xác định loại khung (Beacon, Data, ACK, Command), định dạng địa chỉ (16-bit/64-bit), và các cờ báo hiệu khác như bảo mật, ACK yêu cầu,...
- Sequence Number (8 bit): dùng để đánh số thứ tự khung, giúp nhận dạng và theo dõi các gói dữ liệu (có thể dùng để tránh lặp gói).
- Addressing Fields: bao gồm Destination PAN ID, Destination Address, Source PAN ID và Source Address. Các trường này giúp định tuyến và xác định nguồn/gửi đích của khung. Độ dài các trường này phụ thuộc vào chế độ địa chỉ (short 16-bit hoặc extended 64-bit).
- Auxiliary Security Header (nếu có): chứa các thông tin liên quan đến cơ chế bảo mật như mức độ bảo vệ, chỉ số frame counter,...

Tiếp theo là MAC Payload, chứa dữ liệu thực tế mà ứng dụng hoặc tầng trên gửi xuống. Nội dung của payload thay đổi tùy theo loại khung (Data, Command,...).

Cuối cùng là FCS, dài 16 bit, là trường kiểm tra lỗi được tính bằng phương pháp CRC-16. Trường này giúp tầng MAC kiểm tra tính toàn vẹn của khung dữ liệu sau khi nhận.



Hình 2.2. Cấu trúc khung dữ liệu tầng PHY và MAC trong IEEE 802.15.4[1]

2.1.3 Các kiến trúc mạng hỗ trợ

Một trong những điểm mạnh quan trọng của chuẩn này là khả năng hỗ trợ nhiều loại kiến trúc mạng khác nhau, giúp đáp ứng các yêu cầu đa dạng của các ứng dụng thực tế. Cụ thể, IEEE 802.15.4 hỗ trợ ba mô hình kiến trúc mạng chính là: mạng sao, mạng Mesh và mạng cây phân cụm.

2.1.3.1 Mạng sao

- Mạng sao là mô hình đơn giản nhất trong IEEE 802.15.4, nơi toàn bộ mạng được điều phối bởi một thiết bị trung tâm gọi là PAN Coordinator. Các thiết bị còn lại là thiết bị đầu cuối (End Devices), chúng chỉ có thể gửi và nhận dữ liệu với PAN Coordinator mà không giao tiếp trực tiếp với nhau.
- PAN Coordinator là thiết bị duy nhất có vai trò thiết lập và duy trì mạng, phân phối địa chỉ cho các thiết bị đầu cuối và điều phối truy cập kênh truyền thông trong mạng. Điều này giúp giảm độ phức tạp trong thiết bị đầu cuối, phù hợp với các thiết bị có tài nguyên phần cứng hạn chế và yêu cầu tiêu thụ năng lượng thấp.
- Mạng sao thường sử dụng chế độ Beacon-enabled, trong đó PAN Coordinator phát các beacon định kỳ để đồng bộ thời gian và thông báo trạng thái mạng cho các thiết bị đầu cuối. Điều này rất hiệu quả trong các hệ thống yêu cầu đồng bộ như hệ thống báo động, cảm biến thời gian thực hoặc điều khiển từ xa.
- Mô hình này có ưu điểm là dễ cài đặt, dễ triển khai, chi phí thấp, và tiêu tốn ít năng lượng. Nó phù hợp cho các ứng dụng nhỏ gọn như điều khiển thiết bị gia dụng, mạng cảm biến trong phòng, hệ thống thu thập dữ liệu tập trung ở quy mô hạn chế. Tuy nhiên, nhược điểm của mạng sao là khả năng mở rộng kém và độ tin cậy thấp trong môi trường có nhiều hoặc khoảng cách truyền xa. Nếu PAN Coordinator bị lỗi hoặc mất kết nối, toàn bộ mạng sẽ ngừng hoạt động. Ngoài ra, vì các thiết bị đầu cuối không giao tiếp trực tiếp với nhau, nên không thể thực hiện định tuyến hay chia sẻ dữ liệu giữa các nút.

2.1.3.2 Mạng mesh

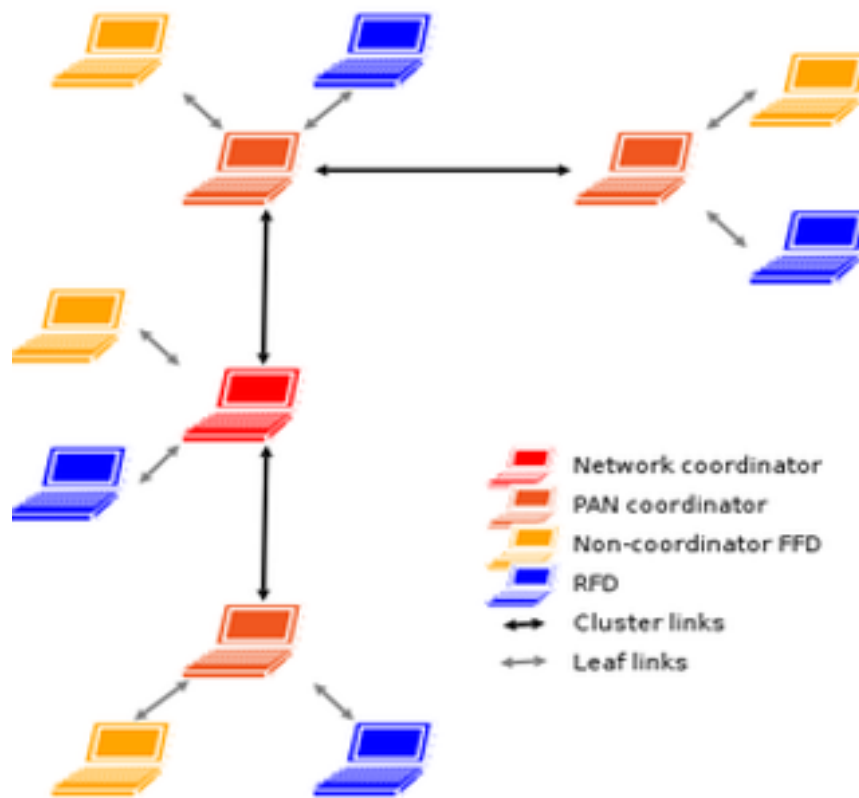
- Mạng mesh trong IEEE 802.15.4 là mô hình tiên tiến hơn, cho phép các thiết bị trong mạng giao tiếp ngang hàng với nhau và hỗ trợ cơ chế định tuyến nhiều bước (multi-hop routing). Điều này giúp dữ liệu được truyền từ thiết bị này đến thiết bị khác qua các tuyến đường khác nhau để đến đích cuối cùng.
- Mạng bao gồm các thiết bị loại FFD (Full-Function Device) và RFD (Reduced-Function Device). FFD có khả năng tham gia vào quá trình định tuyến và hoạt động như các Router hoặc Coordinator phụ, còn RFD chỉ có thể giao tiếp với một

FFD duy nhất và thường đóng vai trò cảm biến hoặc thiết bị đầu cuối với năng lượng thấp.

- Ưu điểm lớn nhất của mạng lưới là khả năng mở rộng cao và độ tin cậy vượt trội. Nếu một đường truyền bị ngắt, mạng có thể tự động tìm tuyến đường thay thế. Điều này đặc biệt quan trọng trong các ứng dụng yêu cầu độ bền mạng cao như giám sát công nghiệp, mạng cảm biến trong nông nghiệp, thành phố thông minh hoặc hạ tầng giao thông.
- Mạng lưới hoạt động hiệu quả trong môi trường động hoặc không ổn định, nơi có thể có nhiều vật cản hoặc nhiễu vô tuyến. Các giao thức cấp cao như Zigbee hoặc Thread thường xây dựng trên kiến trúc này để cung cấp thêm các tính năng định tuyến và bảo mật.
- Nhược điểm của mạng lưới là phức tạp trong thiết kế và triển khai, yêu cầu thiết bị mạnh hơn (FFD), bộ nhớ lớn hơn và phải cài đặt các thuật toán định tuyến như AODV hoặc RPL. Ngoài ra, quản lý đồng bộ và tiết kiệm năng lượng trong mạng lưới là một thách thức lớn do các thiết bị phải duy trì liên lạc liên tục.

2.1.3.3 *Mạng cây phân cụm*

- Mạng cây phân cụm là một kiến trúc lai giữa mạng sao và mạng lưới, trong đó một PAN Coordinator đóng vai trò gốc của cây, và các Coordinator phụ (cũng là các FFD) mở rộng mạng bằng cách hình thành các cụm (cluster) mới. Mỗi Coordinator phụ có thể điều phối một số thiết bị đầu cuối hoặc các Coordinator phụ khác, tạo thành một cấu trúc cây phân tầng.
- Dữ liệu được truyền trong mạng theo đường dẫn duy nhất từ nút con đến nút cha, và cuối cùng đến PAN Coordinator. Điều này giúp mạng dễ quản lý và có cấu trúc rõ ràng, phù hợp với các ứng dụng phân tầng như giám sát trong tòa nhà, nơi mỗi tầng hoặc khu vực là một cụm riêng biệt.
- Mô hình này hỗ trợ khả năng mở rộng hợp lý bằng cách thêm các cụm con mà không làm gián đoạn hoạt động của mạng tổng thể. Tuy không linh hoạt như mạng lưới, nhưng nó đơn giản hơn trong việc định tuyến và kiểm soát lưu lượng.
- Một ưu điểm nữa là tính ổn định và định hướng rõ ràng trong truyền thông, đặc biệt khi mạng hoạt động trong môi trường ít thay đổi. Việc triển khai theo cụm cũng giúp dễ xác định vị trí lỗi và tối ưu việc phân bổ tài nguyên. Nhược điểm là tính linh hoạt kém: nếu một Coordinator trung gian bị lỗi, tất cả các nút con phía dưới sẽ bị mất kết nối. Ngoài ra, không có khả năng định tuyến lại nên mạng không tự phục hồi tốt như mạng lưới. Do đó, mạng cây phù hợp hơn với các hệ thống cố định và ít thay đổi.



Hình 2.3. Sơ đồ mạng hình cây phân cụm

Bảng 2.2. So sánh các kiến trúc mạng trong IEEE 802.15.4

Tiêu chí	Star	Mesh	Cluster Tree
Số lượng Coordinator	1	Nhiều	1 chính, nhiều phụ
Giao tiếp ngang hàng	Không	Có	Có giới hạn
Định tuyến (Routing)	Không	Có	Có giới hạn
Độ mở rộng	Thấp	Cao	Trung bình
Tính tin cậy	Thấp (1 điểm lỗi)	Cao	Trung bình
Mức độ phức tạp	Thấp	Cao	Trung bình
Ứng dụng tiêu biểu	Nhà thông minh nhỏ	Cảm biến nông nghiệp	Tòa nhà, phân tầng

2.1.4 Các ứng dụng tiêu biểu của IEEE 802.15.4

2.1.4.1 Mạng cảm biến không dây

IEEE 802.15.4 được ứng dụng rộng rãi trong các mạng cảm biến không dây – nơi mà hàng loạt thiết bị cảm biến nhỏ gọn, hoạt động bằng pin được triển khai để thu thập dữ liệu từ môi trường như nhiệt độ, độ ẩm, ánh sáng, áp suất hoặc chuyển động.

Với đặc tính tiêu thụ năng lượng thấp và tốc độ truyền dữ liệu vừa đủ, tiêu chuẩn này đặc biệt phù hợp cho các mạng cảm biến cần hoạt động ổn định trong thời gian dài mà không cần thay pin thường xuyên. Mạng WSN sử dụng 802.15.4 thường được triển khai trong nông nghiệp thông minh, giám sát môi trường rừng, đê điều, và theo dõi sức khỏe công trình.

2.1.4.2 Zigbee trong tự động hóa nhà và tòa nhà

Một trong những giao thức nổi bật xây dựng trên nền IEEE 802.15.4 là Zigbee, được sử dụng rộng rãi trong các ứng dụng nhà thông minh và tự động hóa tòa nhà. Với khả năng hình thành mạng mesh, các thiết bị Zigbee như cảm biến chuyển động, đèn, ổ cắm, công tắc,... có thể tự động kết nối và chuyển tiếp dữ liệu lẫn nhau. Điều này giúp người dùng dễ dàng điều khiển thiết bị qua điện thoại hoặc hẹn giờ tự động mà không cần đi dây. Trong các tòa nhà thông minh, Zigbee giúp tiết kiệm năng lượng thông qua điều khiển ánh sáng, điều hòa và giám sát an ninh một cách linh hoạt và hiệu quả.

2.1.4.3 Hệ thống y tế và theo dõi sức khỏe

IEEE 802.15.4 đóng vai trò quan trọng trong các hệ thống theo dõi bệnh nhân không dây, đặc biệt là các mạng Body Area Network (BAN). Các cảm biến đeo trên người bệnh nhân có thể liên tục ghi nhận các thông số y tế như nhịp tim, huyết áp, điện tâm đồ,... và gửi về trung tâm điều khiển để theo dõi từ xa. Với khả năng tiêu thụ năng lượng cực thấp và hỗ trợ chế độ ngủ sâu, các thiết bị sử dụng 802.15.4 có thể hoạt động lâu dài, giảm số lần sạc pin, từ đó tăng sự tiện lợi và an toàn cho bệnh nhân – đặc biệt là người cao tuổi hoặc bệnh nhân cần giám sát liên tục tại nhà.

2.1.4.4 Hệ thống đo lường thông minh

IEEE 802.15.4 cũng được sử dụng trong các hệ thống đo lường tự động, giúp thu thập dữ liệu từ các đồng hồ điện, nước và gas mà không cần nhân viên đến từng nhà ghi chỉ số. Giao thức Zigbee Smart Energy Profile là một ứng dụng điển hình, cho phép thiết bị đo thông minh giao tiếp hai chiều với trung tâm điều khiển hoặc hệ thống quản lý năng lượng. Từ đó, người dùng có thể nhận được cảnh báo về mức tiêu thụ điện vượt ngưỡng, điều chỉnh hành vi sử dụng năng lượng và thậm chí phối hợp với lưới điện thông minh để tối ưu hóa hiệu suất vận hành.

2.1.4.5 Giao thông thông minh và hạ tầng đô thị số

IEEE 802.15.4 cũng góp phần quan trọng trong các hệ thống giao thông thông minh. Các cảm biến đặt dọc tuyến đường có thể thu thập dữ liệu về lưu lượng xe, tình trạng giao thông, ô nhiễm không khí hoặc hỗ trợ hệ thống đèn tín hiệu thích ứng. Trong các bãi đỗ xe thông minh, cảm biến dựa trên 802.15.4 có thể phát hiện xe vào/ra và báo chỗ trống về trung tâm điều khiển hoặc ứng dụng điện thoại người dùng. Nhờ tiêu chuẩn hỗ trợ mạng mesh và độ tin cậy cao, các hệ thống này hoạt động ổn định trong môi trường đô thị nhiều nhiễu và khoảng cách truyền xa.

2.2 Giao thức truyền thông không dây Zigbee

2.2.1 Tổng quan về Zigbee

Zigbee[12] là một chuẩn truyền thông không dây được thiết kế chủ yếu cho các ứng dụng yêu cầu tiêu thụ năng lượng thấp, tốc độ dữ liệu vừa phải và khả năng kết nối nhiều thiết bị trong một mạng. Nhờ đặc điểm này, Zigbee đặc biệt phù hợp cho các ứng dụng như nhà thông minh, hệ thống tự động hóa công nghiệp, giám sát môi trường và các mạng cảm biến không dây.

Một trong những ưu điểm nổi bật của Zigbee là khả năng hỗ trợ mạng mesh. Với mô hình mạng mesh, các thiết bị trong mạng có thể giao tiếp gián tiếp với nhau thông qua các nút trung gian, giúp mở rộng phạm vi truyền và tăng tính tin cậy. Mỗi thiết bị Zigbee có thể đóng vai trò như một thiết bị điều phối (coordinator), thiết bị router hoặc thiết bị đầu cuối (end device). Kiến trúc này cho phép hàng trăm đến hàng ngàn thiết bị cùng hoạt động trong một mạng Zigbee ổn định, đồng thời tăng khả năng chống mất kết nối khi một thiết bị bị lỗi hoặc hết pin.

Zigbee cung cấp tốc độ truyền dữ liệu khoảng 250 kbps – tuy không cao so với các công nghệ như Wi-Fi hay Bluetooth, nhưng lại là mức lý tưởng cho các ứng dụng cần truyền dữ liệu định kỳ, nhỏ gọn và tiêu thụ điện năng thấp. Một ví dụ điển hình là các cảm biến đo nhiệt độ, độ ẩm hoặc trạng thái cửa trong nhà thông minh – những thiết bị này thường chỉ cần truyền dữ liệu vài byte mỗi phút hoặc mỗi giờ.

Ngoài ra, Zigbee còn được biết đến với khả năng tiêu thụ năng lượng rất thấp. Các thiết bị đầu cuối có thể được lập trình để "ngủ" phần lớn thời gian và chỉ "thức dậy" khi cần truyền hoặc nhận dữ liệu, từ đó kéo dài thời gian hoạt động lên đến vài năm chỉ với một viên pin nhỏ. Chính nhờ ưu điểm này mà Zigbee trở thành lựa chọn lý tưởng cho các ứng dụng IoT không cần cấp nguồn liên tục. Tính bảo mật trong Zigbee cũng được chú trọng. Chuẩn này sử dụng mã hóa AES 128-bit để bảo vệ dữ liệu truyền tải giữa các thiết bị, đồng thời hỗ trợ các cơ chế xác thực và quản lý khóa nhằm đảm bảo an toàn cho toàn bộ hệ thống.

Bảng 2.3. So sánh một số giao thức truyền thông không dây[2]

Giao thức	Tốc độ	Phạm vi	Năng lượng	Bảo mật	Mở rộng
Zigbee	250 kbps	10–100 m	Rất thấp	AES-128	Rất cao
Wi-Fi	11–1000 Mbps	50–100 m	Cao	WPA2	Trung bình
Bluetooth	1–3 Mbps	10 m	Trung bình	AES-128	Thấp
LoRa	<50 kbps	2–15 km	Rất thấp	AES-128	Cao

Bảng so sánh cho thấy mỗi giao thức không dây có những ưu điểm riêng phù hợp với từng ứng dụng. Zigbee có tốc độ thấp (250 kbps) nhưng tiêu thụ năng lượng rất thấp và khả năng mở rộng mạng cao, thích hợp cho các mạng cảm biến và điều khiển tự động trong nhà. Wi-Fi tuy có tốc độ rất cao (11–1000 Mbps) nhưng tiêu tốn nhiều

năng lượng, phù hợp cho các ứng dụng cần băng thông lớn như truyền video.

Bluetooth có tốc độ trung bình (1–3 Mbps), phạm vi ngắn (10 m) và năng lượng tiêu thụ ở mức trung bình, phù hợp với các thiết bị cá nhân như tai nghe hoặc cảm biến đeo. Trong khi đó, LoRa nổi bật với phạm vi truyền xa (2–15 km) và tiêu thụ năng lượng rất thấp, dù tốc độ truyền rất thấp (<50 kbps), nên rất thích hợp cho các ứng dụng giám sát từ xa như nông nghiệp thông minh hoặc thành phố thông minh.

Về bảo mật, cả Zigbee, Bluetooth và LoRa đều sử dụng mã hóa AES-128, đảm bảo an toàn ở mức cao. Wi-Fi dùng chuẩn WPA2, cũng đáp ứng yêu cầu bảo mật tốt trong mạng gia đình và công nghiệp. Tổng thể, Zigbee là lựa chọn cân bằng giữa năng lượng, bảo mật và khả năng mở rộng cho các ứng dụng mạng cảm biến tầm ngắn.

2.2.2 Kiến trúc giao thức Zigbee

Giao thức Zigbee được xây dựng dựa trên mô hình phân lớp tương tự như mô hình OSI, trong đó mỗi lớp đảm nhiệm một chức năng riêng biệt, từ truyền tín hiệu vật lý đến xử lý dữ liệu ứng dụng. Mô hình này bao gồm năm lớp chính[11]: lớp vật lý (PHY), lớp điều khiển truy cập môi trường (MAC), lớp mạng (NWK), lớp hỗ trợ ứng dụng (APS) và lớp ứng dụng (APL).

2.2.2.1 Lớp vật lý

Lớp vật lý là tầng thấp nhất trong mô hình Zigbee, đảm nhiệm việc truyền và nhận dữ liệu thô qua môi trường truyền dẫn không dây. Cụ thể, lớp này định nghĩa các đặc tả về tần số hoạt động, sơ đồ điều chế, tốc độ truyền dữ liệu, mức công suất phát, và các đặc tính vật lý khác của tín hiệu. Zigbee sử dụng sơ đồ điều chế O-QPSK (Offset Quadrature Phase-Shift Keying) trong băng tần 2.4 GHz, cho phép truyền dữ liệu ổn định với mức tiêu thụ năng lượng thấp. Ngoài ra, nó còn sử dụng mã hóa dữ liệu và kỹ thuật kiểm soát công suất để tối ưu hóa hiệu năng truyền dẫn.

Zigbee hỗ trợ ba dải tần chính: băng tần 2.4 GHz (toàn cầu) với 16 kênh (Kênh 11 đến Kênh 26) cho tốc độ truyền lên đến 250 kbps; băng tần 915 MHz (chủ yếu tại Bắc Mỹ) với 10 kênh, tốc độ 40 kbps; và băng tần 868 MHz (tại châu Âu) với 1 kênh duy nhất, tốc độ 20 kbps. Khoảng cách truyền dữ liệu trong mạng Zigbee dao động từ vài mét đến hàng trăm mét, tùy thuộc vào công suất phát, môi trường xung quanh (vật cản, nhiễu), và vị trí lắp đặt thiết bị.

2.2.2.2 Lớp MAC

Lớp MAC chịu trách nhiệm điều phối quyền truy cập của các thiết bị vào môi trường truyền dẫn chung, đảm bảo rằng các gói tin không bị va chạm khi nhiều thiết bị cùng gửi dữ liệu. Nó thực hiện việc đóng gói dữ liệu thành các khung (frame), cấp phát địa chỉ MAC, xác thực đơn giản giữa các thiết bị, và hỗ trợ cơ chế tái truyền gói tin khi có lỗi xảy ra trong quá trình truyền.

Lớp MAC trong Zigbee hỗ trợ hai chế độ hoạt động chính: beacon-enabled mode

và non-beacon mode. Ở chế độ beacon-enabled, thiết bị điều phối (coordinator) sẽ phát các beacon định kỳ để đồng bộ hóa các thiết bị con, rất phù hợp cho các mạng có cấu trúc phân cấp, tiết kiệm năng lượng. Trong khi đó, non-beacon mode hoạt động dựa trên cơ chế CSMA/CA (Carrier Sense Multiple Access with Collision Avoidance), cho phép các thiết bị truyền dữ liệu khi phát hiện kênh truyền đang rảnh, tránh va chạm. Ngoài ra, lớp MAC còn hỗ trợ chế độ ngủ (sleep mode), cho phép các thiết bị giảm tiêu thụ năng lượng trong những khoảng thời gian không hoạt động.

2.2.2.3 *Lớp mạng*

Lớp mạng trong Zigbee giữ vai trò quan trọng trong việc hình thành, cấu hình, quản lý và duy trì mạng lưới các thiết bị không dây. Nó phụ trách định tuyến gói tin giữa các thiết bị, cấp phát địa chỉ mạng (network address), và đảm bảo tính ổn định của toàn mạng. Zigbee hỗ trợ các cấu trúc mạng như star (hình sao), tree (cây) và đặc biệt là mesh (mắt lưới), trong đó các thiết bị có thể chuyển tiếp dữ liệu qua nhau để mở rộng phạm vi mạng.

Về định tuyến, lớp mạng hỗ trợ cả hai phương pháp: định tuyến dựa trên bảng (table-based routing), nơi mỗi thiết bị duy trì bảng định tuyến riêng, và định tuyến theo yêu cầu (on-demand routing), nơi đường đi được thiết lập chỉ khi cần thiết. Nhờ khả năng tự cấu hình và tự phục hồi, Zigbee có thể duy trì hoạt động ổn định ngay cả khi một số thiết bị trong mạng bị lỗi hoặc thay đổi vị trí.

2.2.2.4 *Lớp hỗ trợ ứng dụng*

Lớp APS đóng vai trò trung gian giữa lớp mạng và lớp ứng dụng, chịu trách nhiệm phân phối dữ liệu giữa các thiết bị ứng dụng và quản lý thông tin liên kết. Nó duy trì bảng liên kết (binding table), trong đó ánh xạ các endpoint (điểm cuối của thiết bị ứng dụng) với các chức năng cụ thể. Nhờ đó, lớp APS có thể điều phối việc truyền dữ liệu một cách đáng tin cậy giữa các thiết bị với nhau.

Lớp này cũng cung cấp các dịch vụ như phân mảnh gói tin nếu dữ liệu vượt quá kích thước tối đa cho phép, hoặc tái kết hợp gói tin khi nhận về. Ngoài ra, lớp APS còn xử lý các yêu cầu dịch vụ từ ứng dụng, hỗ trợ bảo mật dữ liệu trong quá trình truyền và tạo nền tảng để các ứng dụng người dùng hoạt động hiệu quả trên mạng Zigbee.

2.2.2.5 *Lớp ứng dụng*

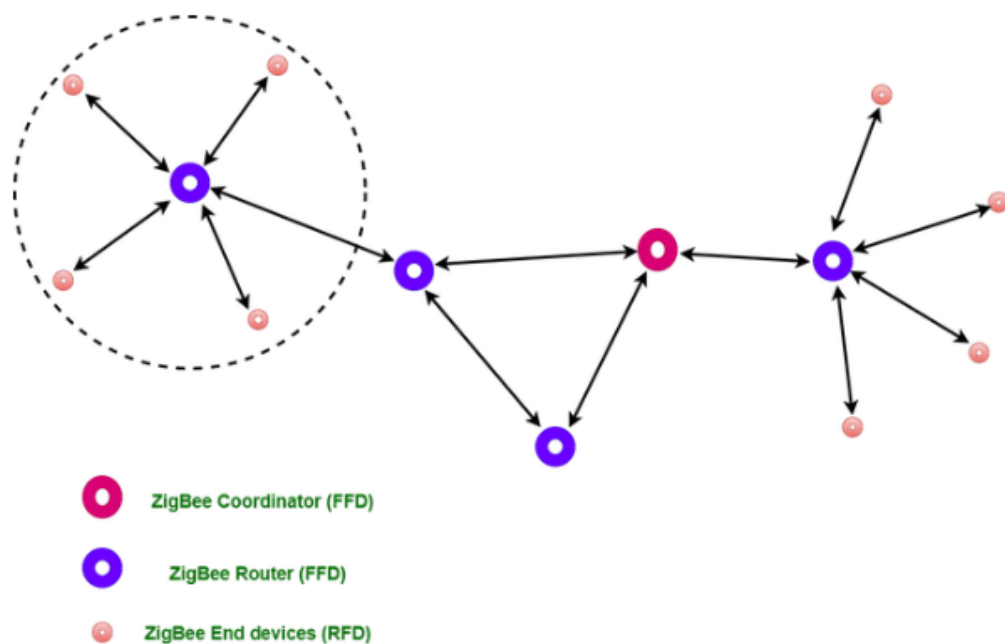
Lớp ứng dụng là tầng cao nhất trong mô hình Zigbee, nơi triển khai các chức năng cụ thể mà người dùng cuối tương tác. Nó bao gồm hai thành phần chính: Zigbee Device Object (ZDO) và các Application Object (đối tượng ứng dụng). ZDO quản lý các nhiệm vụ thiết yếu như khởi tạo thiết bị, khám phá và kết nối dịch vụ, xác định vai trò của thiết bị (coordinator, router, end device) trong mạng Zigbee.

Mỗi thiết bị Zigbee có thể chứa nhiều Application Object, tương ứng với các chức năng cụ thể như cảm biến nhiệt độ, công tắc, bộ điều khiển đèn, v.v. Các chức năng này được mô tả thông qua các Cluster ID – tập hợp các lệnh và thuộc tính dùng

chung cho một loại dịch vụ nhất định. Lớp ứng dụng cho phép thiết bị giao tiếp theo chuẩn Zigbee Application Framework, đảm bảo tính tương thích và khả năng mở rộng của hệ thống.

2.2.2.6 Các loại thiết bị trong mạng Zigbee

Trong mạng Zigbee, các thiết bị được phân loại dựa trên vai trò và chức năng của chúng trong mạng. Có ba loại thiết bị chính trong một mạng Zigbee: Thiết bị điều phối (Zigbee Coordinator - ZC), Thiết bị định tuyến (Zigbee Router - ZR) và Thiết bị đầu cuối (Zigbee End Device - ZED). Mỗi loại thiết bị đảm nhận một vai trò khác nhau trong việc hình thành, duy trì và vận hành mạng. Việc phân chia này cho phép tối ưu hóa tiêu thụ năng lượng, hiệu suất truyền thông và khả năng mở rộng mạng.



Hình 2.4. Mạng Zigbee mesh với Coordinator, Router và End Device

- Zigbee Coordinator (ZC) – Thiết bị điều phối:

Zigbee Coordinator là thiết bị quan trọng nhất trong mạng Zigbee. Mỗi mạng chỉ có duy nhất một thiết bị điều phối, chịu trách nhiệm khởi tạo mạng, lựa chọn kênh truyền thông, gán địa chỉ mạng, và quản lý thông tin an ninh như khóa mã hóa mạng. ZC cũng có khả năng lưu trữ bảng định tuyến và thông tin thiết bị trong toàn mạng.

Ngoài chức năng điều phối, ZC có thể thực hiện nhiệm vụ truyền nhận dữ liệu giống như các thiết bị khác. Trong một số ứng dụng như nhà thông minh, ZC thường được kết nối với một máy chủ trung tâm hoặc gateway để thu thập và điều khiển toàn bộ hệ thống. Vì đảm nhiệm vai trò trung tâm và cần hoạt động liên tục, ZC thường được cấp nguồn không giới hạn (kết nối điện lưới thay vì pin).

- Zigbee Router (ZR) – Thiết bị định tuyến:

Zigbee Router là những thiết bị trung gian có khả năng truyền tiếp gói tin, mở rộng phạm vi mạng và định tuyến dữ liệu giữa các nút khác nhau. Không giống như thiết bị đầu cuối, ZR có khả năng lưu trữ bảng định tuyến và xử lý các yêu cầu định tuyến động. Các router đóng vai trò duy trì kết nối giữa các phần khác nhau trong mạng, đặc biệt quan trọng trong mạng có cấu trúc mesh hoặc tree, nơi gói tin cần truyền qua nhiều nút trung gian để đến đích.

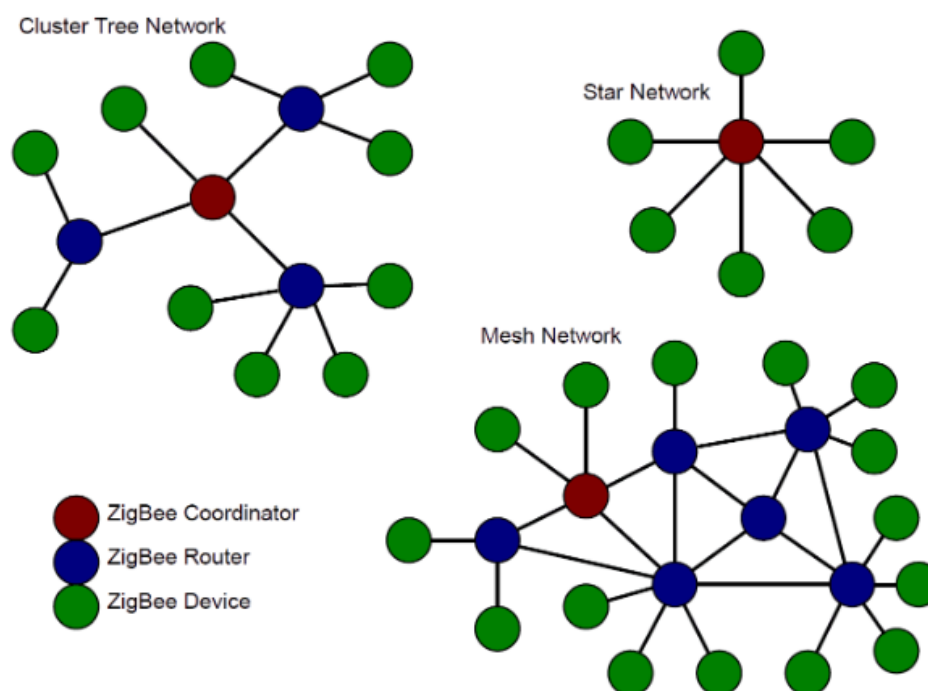
ZR cũng có thể đóng vai trò như một thiết bị đầu cuối để thu thập hoặc điều khiển dữ liệu. Trong thực tế, các thiết bị như công tắc, bộ điều khiển trung gian hoặc các điểm nút cảm biến cố định có thể hoạt động như một ZR. Vì có trách nhiệm định tuyến, các router cần hoạt động liên tục và cũng thường được cấp nguồn ổn định.

- Zigbee End Device (ZED) – Thiết bị đầu cuối:

Zigbee End Device là thiết bị đơn giản và tiết kiệm năng lượng nhất trong mạng Zigbee. ZED không thực hiện chức năng định tuyến và chỉ giao tiếp với một thiết bị cha (thường là ZR hoặc ZC). Điều này giúp giảm yêu cầu về bộ nhớ và xử lý, đồng thời tiết kiệm pin cho ZED – một yếu tố rất quan trọng trong các ứng dụng như cảm biến nhiệt độ, cảm biến chuyển động, hoặc công tắc pin.

ZED có thể được cấu hình để hoạt động ở chế độ ngủ (sleep mode) phần lớn thời gian và chỉ thức dậy định kỳ để gửi dữ liệu hoặc nhận lệnh. Do đó, thời gian sử dụng pin của ZED có thể kéo dài từ vài tháng đến nhiều năm, tùy vào tần suất hoạt động và loại cảm biến.

2.2.2.7 Các cấu trúc mạng Zigbee



Hình 2.5. Các loại kiến trúc mạng Zigbee

- Mạng hình sao

Trong kiến trúc hình sao, tất cả các thiết bị đầu cuối (Zigbee End Devices - ZED) đều kết nối trực tiếp với một thiết bị điều phối trung tâm gọi là Zigbee Coordinator (ZC). Trong mô hình này, ZC đóng vai trò như một trung tâm điều khiển, chịu trách nhiệm giao tiếp và quản lý toàn bộ các thiết bị con. Do thiết kế đơn giản, mô hình này dễ triển khai và cấu hình, đồng thời tiêu tốn ít tài nguyên định tuyến. Tuy nhiên, điểm yếu lớn nhất của kiến trúc hình sao là sự phụ thuộc hoàn toàn vào ZC: nếu thiết bị trung tâm bị lỗi, toàn bộ mạng sẽ ngừng hoạt động. Kiến trúc này phù hợp cho các ứng dụng quy mô nhỏ như mạng cảm biến trong nhà, điều khiển đèn chiếu sáng hay các thiết bị gia dụng thông minh.

- Mạng hình cây

Kiến trúc cây lại sử dụng một cấu trúc phân cấp, trong đó ZC là nút gốc, các Zigbee Router (ZR) hoạt động như các nút trung gian, và các ZED nằm ở lá cây. Mỗi router có thể quản lý một số lượng nhất định các nút con, giúp mạng dễ dàng được mở rộng theo nhánh. Dữ liệu sẽ truyền theo hướng đi rõ ràng dọc theo các nhánh của cây. Ưu điểm của mô hình này là dễ quản lý, cho phép mở rộng phạm vi mạng mà không cần tăng công suất truyền. Tuy nhiên, kiến trúc cây dễ bị gián đoạn nếu một router trung gian gặp lỗi vì các thiết bị con sẽ mất kết nối. Đây là mô hình phù hợp cho các ứng dụng giám sát môi trường, chiếu sáng ngoài trời hoặc mạng cảm biến trong các khu công nghiệp có cấu trúc định sẵn.

- Mạng mesh

Kiến trúc mesh là mô hình linh hoạt và mạnh mẽ nhất trong Zigbee. Trong mạng mesh, các Zigbee Router có thể giao tiếp trực tiếp với nhiều router khác, tạo thành một mạng lưới liên kết phức tạp. Dữ liệu có thể truyền qua nhiều đường đi khác nhau giữa các nút, giúp tăng tính chịu lỗi và khả năng tự phục hồi của mạng. Nhờ sử dụng các thuật toán định tuyến động như AODV, mạng mesh có thể tự động tìm đường đi tối ưu khi có nút gặp sự cố hoặc khi mạng thay đổi cấu trúc. Mặc dù phức tạp hơn và tiêu tốn năng lượng nhiều hơn so với các kiến trúc khác, mesh là lựa chọn tối ưu cho các ứng dụng quy mô lớn, yêu cầu độ tin cậy cao như nhà thông minh toàn diện, hệ thống an ninh hoặc mạng cảm biến trong công nghiệp.

2.2.3 Định tuyến trong mạng Zigbee

Định tuyến là một thành phần quan trọng trong mạng Zigbee, đặc biệt trong các kiến trúc tree và mesh nơi mà các gói dữ liệu cần phải đi qua nhiều thiết bị trung gian để đến được đích. Zigbee sử dụng các cơ chế định tuyến được tiêu chuẩn hóa trong lớp mạng, cho phép thiết bị tự động tìm đường đi tối ưu và duy trì kết nối ổn định trong môi trường mạng không dây, nơi các nút có thể di chuyển, rơi rụng hoặc thay đổi trạng thái.

2.2.3.1 Bảng định tuyến và thông tin mạng

Mỗi thiết bị Zigbee có khả năng định tuyến (tức Zigbee Coordinator hoặc Zigbee Router) đều duy trì một bảng định tuyến và các thông tin cấu trúc mạng như địa chỉ cha, con, bảng láng giềng, bảng chuyển tiếp,... Những bảng này được sử dụng để xác định đường đi tốt nhất của gói tin đến đích. Việc cập nhật bảng định tuyến diễn ra tự động thông qua các quá trình phát hiện nút lân cận và trao đổi thông tin mạng định kỳ.

2.2.3.2 Các thuật toán định tuyến phổ biến

- Định tuyến AODV (Ad-hoc On-demand Distance Vector)[13]:

AODV là thuật toán định tuyến theo yêu cầu, nghĩa là tuyến đường chỉ được thiết lập khi có nhu cầu truyền dữ liệu. Khi một thiết bị cần gửi gói tin đến một thiết bị khác mà không có tuyến sẵn, nó sẽ phát ra một Route Request (RREQ). Các thiết bị trung gian sẽ chuyển tiếp gói này và nếu đích nhận được, sẽ trả lời bằng Route Reply (RREP). Sau đó, tuyến đường được thiết lập và lưu vào bảng định tuyến.

Ưu điểm của AODV là tiết kiệm tài nguyên vì không cần duy trì liên tục bảng định tuyến đầy đủ cho toàn mạng. Nhược điểm là có độ trễ khi lần đầu thiết lập đường truyền. AODV thường được sử dụng trong mạng mesh Zigbee nhờ khả năng tự phục hồi khi thay đổi cấu trúc mạng.

- Định tuyến theo cây (Tree Routing)[14]:

Mô hình định tuyến cây sử dụng cấu trúc phân cấp, trong đó mỗi nút biết được nút cha của mình và các nút con. Dữ liệu được truyền theo nhánh từ nút con đến nút cha cho đến khi đến được đích hoặc đạt đến một nút có thể định tuyến đến đích.

Ưu điểm của tree routing là đơn giản, không yêu cầu lưu trữ bảng định tuyến phức tạp. Tuy nhiên, nhược điểm là không linh hoạt: nếu một nhánh của cây bị hỏng, các thiết bị phụ thuộc vào nhánh đó có thể mất kết nối. Tree routing phù hợp cho các ứng dụng có cấu trúc mạng ổn định, ít thay đổi.

- Định tuyến mesh (Mesh Routing)[15]:

Mesh routing là mô hình định tuyến động và linh hoạt nhất. Mỗi thiết bị router có thể kết nối với nhiều thiết bị lân cận và chuyển tiếp gói tin theo các đường đi khác nhau. Kết hợp với AODV, mạng có thể tự động cập nhật tuyến đường khi có sự thay đổi trong cấu trúc mạng hoặc khi có nút gặp lỗi.

Ưu điểm chính của mesh routing là độ tin cậy cao, khả năng phục hồi mạnh và dễ mở rộng. Dù tiêu tốn tài nguyên hơn, đây là lựa chọn lý tưởng cho các ứng dụng công nghiệp hoặc nhà thông minh với nhiều thiết bị và yêu cầu tính liên tục cao.

2.2.3.3 Cơ chế xử lý lỗi và phục hồi

Khi một nút trong tuyến đường bị mất hoặc không phản hồi, lớp mạng Zigbee sẽ khởi tạo lại quá trình khám phá đường đi mới bằng AODV hoặc gửi thông báo lỗi để các nút liên quan cập nhật lại bảng định tuyến. Nhờ vậy, mạng có khả năng tự phục hồi, đảm bảo tính ổn định và độ tin cậy trong truyền thông.

2.3 RAIL(Radio Abstraction Interface Layer)

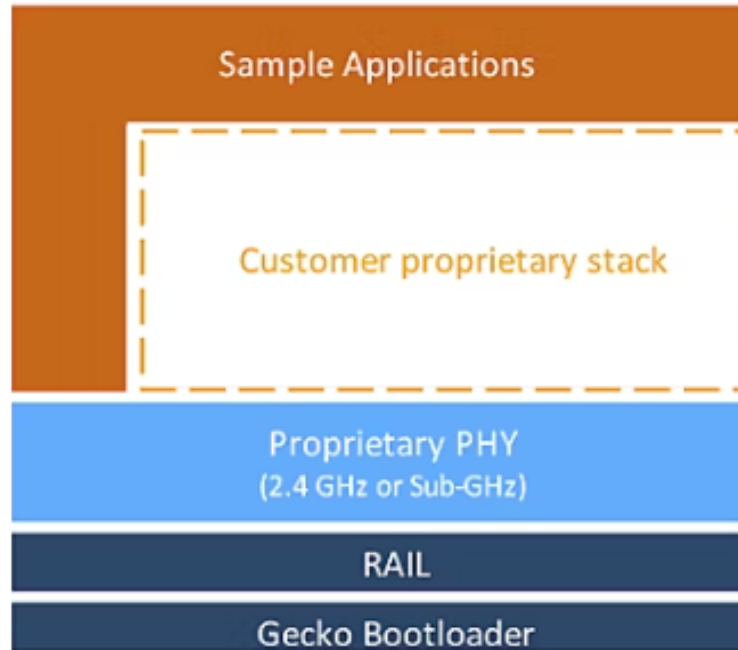
2.3.1 Tổng quan về RAIL

RAIL (Radio Abstraction Interface Layer)[16] là một lớp trừu tượng phần mềm được phát triển bởi Silicon Labs nhằm đơn giản hóa việc điều khiển radio trong các hệ thống nhúng sử dụng dòng SoC EFR32. Thay vì phải thao tác trực tiếp với các thanh ghi phần cứng phức tạp, RAIL cung cấp một giao diện lập trình ứng dụng (API) thống nhất, rõ ràng và dễ sử dụng. Nhờ đó, các nhà phát triển có thể điều khiển toàn bộ hoạt động của radio như truyền, nhận, cấu hình kênh, công suất phát, xử lý sự kiện... chỉ thông qua các lệnh API cấp cao. Những lệnh này sẽ được RAIL chuyển đổi nội bộ thành các thao tác cấp thanh ghi tương ứng để điều khiển phần cứng radio một cách chính xác và hiệu quả.

Một trong những ưu điểm nổi bật của RAIL là khả năng hỗ trợ đa giao thức truyền thông. Nó có thể hoạt động linh hoạt với các giao thức chuẩn như BLE (Bluetooth Low Energy), Zigbee, Thread, Wi-SUN cũng như các giao thức độc quyền (proprietary). Điều này giúp RAIL trở thành nền tảng trung gian lý tưởng để phát triển các ứng dụng không dây tùy chỉnh hoặc dựa trên các chuẩn công nghiệp mà không yêu cầu người lập trình phải hiểu sâu về chi tiết phần cứng cụ thể của từng dòng SoC.

2.3.2 Cấu trúc của RAIL

RAIL được thiết kế với kiến trúc phân tầng, chia thành nhiều lớp riêng biệt nhằm tối ưu hóa khả năng mở rộng, tái sử dụng và dễ bảo trì. Mỗi lớp đảm nhận một chức năng cụ thể trong hệ thống điều khiển radio:



Hình 2.6. Kiến trúc phần mềm RAIL với ngăn xếp tùy chỉnh

- Lớp API (Application Programming Interface):

Đây là lớp giao diện chính mà ứng dụng người dùng tương tác trực tiếp. Nó cung cấp hàng loạt hàm để điều khiển các chức năng quan trọng của radio như thiết lập kênh truyền, bắt đầu truyền hoặc nhận dữ liệu, xử lý lỗi, cấu hình công suất phát và các tham số vật lý. Tất cả được đóng gói dưới dạng các hàm lập trình rõ ràng và dễ sử dụng.

- Lớp Scheduler (Bộ lập lịch):

Scheduler trong RAIL đảm nhận vai trò điều phối việc sử dụng tài nguyên radio trong môi trường có nhiều giao thức hoặc nhiều tiến trình truyền thông chạy song song. Nó có khả năng ưu tiên, xếp lịch và chuyển đổi kênh phù hợp để đảm bảo các giao thức không bị xung đột trong quá trình hoạt động, đặc biệt quan trọng trong các ứng dụng đa giao thức.

- Lớp HAL (Hardware Abstraction Layer):

HAL là lớp trung gian giữa phần mềm và phần cứng. Nó cung cấp các hàm trừu tượng hóa việc truy xuất phần cứng radio, như điều khiển bộ thu phát RF, các bộ định thời, hoặc cấu hình GPIO liên quan đến truyền thông. Nhờ lớp HAL,

ứng dụng RAIL có thể chạy trên nhiều dòng SoC khác nhau của Silicon Labs mà không cần thay đổi logic ứng dụng.

- Lớp cấu hình PHY (PHY Configuration):

Lớp này cho phép người phát triển tùy chỉnh các tham số vật lý của radio như tốc độ truyền dữ liệu (data rate), độ rộng băng thông kênh, modulation scheme (ASK, FSK, OQPSK...), công suất phát, độ nhạy thu,... Để hỗ trợ việc cấu hình này, Silicon Labs cung cấp công cụ trực quan gọi là Radio Configurator tích hợp trong Simplicity Studio. Công cụ này cho phép người dùng chọn cấu hình mong muốn mà không cần thao tác tay trên mã nguồn.

2.3.3 Quy trình hoạt động của RAIL

Quy trình hoạt động của RAIL thường bắt đầu bằng bước khởi tạo hệ thống radio, thông qua hàm *RAIL_Init()*. Hàm này sẽ khởi tạo thư viện RAIL, cấp phát các tài nguyên phần cứng cần thiết và đưa hệ thống radio vào trạng thái sẵn sàng. Sau khi khởi tạo, ứng dụng sẽ tiến hành cấu hình radio dựa trên các thông số về PHY như tần số hoạt động, tốc độ truyền, công suất phát và độ rộng băng thông kênh. Việc cấu hình này có thể được thực hiện thủ công thông qua API hoặc thông qua công cụ trực quan Radio Configurator, từ đó sinh ra các tệp cấu hình .c/.h dùng cho chương trình.

Sau khi hoàn tất cấu hình, radio sẽ được đưa vào chế độ nghe (RX) hoặc truyền (TX). Trong chế độ nhận, hàm *RAIL_StartRx()* sẽ được gọi để bắt đầu lắng nghe dữ liệu trên kênh đã chọn. Khi có gói tin đến, RAIL sẽ kiểm tra tính hợp lệ (CRC) và đưa gói tin vào hàng đợi, chờ xử lý. Ngược lại, trong chế độ truyền, ứng dụng sử dụng *RAIL_StartTx()* để phát một gói tin, với dữ liệu được cung cấp từ bộ đệm. Cả hai hoạt động này đều được kiểm soát chặt chẽ bởi RAIL nhằm đảm bảo không có xung đột trong quá trình truyền nhận.

Một thành phần quan trọng trong hoạt động của RAIL là cơ chế xử lý sự kiện (event handling). Khi một sự kiện liên quan đến radio xảy ra, chẳng hạn như gói tin nhận thành công, truyền hoàn tất, kênh bận hoặc lỗi CRC, RAIL sẽ kích hoạt các callback do người dùng đăng ký như *RAILCb_Generic()* để thông báo và xử lý. Điều này giúp ứng dụng chủ động theo dõi và phản ứng với các tình huống trong quá trình truyền thông một cách hiệu quả.

Bên cạnh việc điều khiển truyền nhận, RAIL cũng hỗ trợ quản lý năng lượng tối ưu. Nó cung cấp các API để đưa hệ thống radio về các chế độ tiết kiệm năng lượng như sleep hoặc idle khi không hoạt động, từ đó giúp kéo dài thời lượng pin trong các ứng dụng IoT. Khi cần hoạt động trở lại, RAIL sẽ tự động khôi phục trạng thái radio và tiếp tục truyền nhận như bình thường.

Tổng thể, RAIL cung cấp một quy trình hoạt động chặt chẽ và hiệu quả, bao gồm: khởi tạo → cấu hình radio → truyền/nhận dữ liệu → xử lý sự kiện → quản lý

năng lượng. Nhờ tính trừu tượng cao, khả năng cấu hình linh hoạt và hỗ trợ đa giao thức, RAIL là một nền tảng lý tưởng cho việc phát triển các ứng dụng không dây trên vi điều khiển của Silicon Labs, đặc biệt trong các hệ thống nhúng cần tối ưu hóa về công suất và hiệu suất truyền thông.

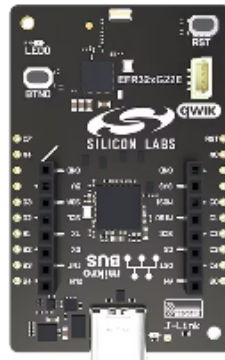
Bảng 2.4. Các API thông dụng trong thư viện RAIL của Silicon Labs[3]

API	Chức năng
RAIL_Init()	Khởi tạo thư viện RAIL và cấp phát tài nguyên phần cứng. Trả về một RAIL_Handle_t dùng cho các API khác.
RAIL_ConfigChannels()	Cấu hình các kênh radio dựa trên cấu hình từ Radio Configurator.
RAIL_ConfigRxOptions()	Thiết lập các tùy chọn cho chế độ nhận như lọc địa chỉ, tự động xác nhận.
RAIL_StartRx()	Bắt đầu chế độ nhận dữ liệu trên kênh chỉ định.
RAIL_StartTx()	Bắt đầu truyền dữ liệu với nội dung từ bộ đệm FIFO.
RAIL_Idle()	Đưa radio về trạng thái nhàn rỗi (idle).
RAIL_ConfigEvents()	Đăng ký các sự kiện radio để nhận qua callback.
RAIL_ConfigSleep()	Cấu hình chế độ tiết kiệm năng lượng cho radio.
RAIL_Sleep()	Đưa radio vào chế độ sleep.
RAIL_Wake()	Đánh thức radio từ chế độ sleep.
RAIL_GetTxFifoSpaceAvailable()	Trả về dung lượng trống trong FIFO truyền.
RAIL_WriteTxFifo()	Ghi dữ liệu vào FIFO truyền.
RAIL_ReadRxFifo()	Đọc dữ liệu từ FIFO nhận.
RAIL_HoldRxPacket()	Giữ gói tin đã nhận để xử lý sau.
RAIL_ReleaseRxPacket()	Giải phóng gói tin đã giữ sau khi xử lý.
RAIL_GetRxPacketInfo()	Lấy thông tin chi tiết về gói tin nhận.
RAIL_ConfigAutoAck()	Cấu hình tính năng tự động xác nhận (Auto-ACK).
RAIL_SetTxPower()	Thiết lập công suất phát của radio.
RAIL_GetRssi()	Đo cường độ tín hiệu nhận được (RSSI).
RAIL_ConfigData()	Cấu hình cách quản lý dữ liệu: chế độ PACKET hoặc FIFO.

2.4 SoC EFR32MG24 - Tham chiếu để thiết kế mạch cứng

2.4.1 Giới thiệu chung về EFR32MG24

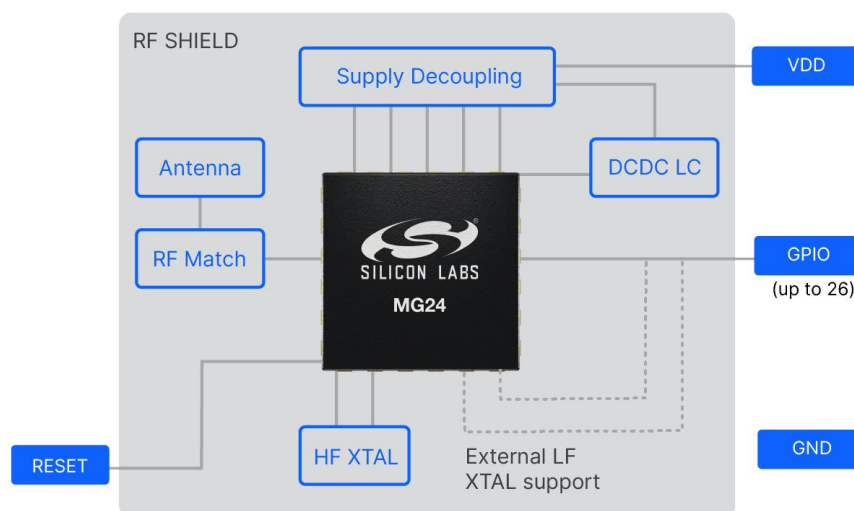
EFR32 là dòng vi điều khiển tích hợp RF (SoC – System on Chip) do hãng Silicon Labs phát triển, được thiết kế chuyên biệt cho các ứng dụng kết nối không dây công suất thấp như Zigbee, Thread, Bluetooth Low Energy, và chuẩn IEEE 802.15.4. Dòng EFR32 nổi bật nhờ khả năng tích hợp cao, tiêu thụ năng lượng thấp, hiệu suất truyền nhận mạnh mẽ, và bộ công cụ phần mềm mạnh mẽ đi kèm như Simplicity Studio, RAIL, Gecko SDK.



Hình 2.7. Soc EFR32MG24

EFR32MG là dòng chip hỗ trợ đa giao thức không dây, trong đó "MG" viết tắt của "Multiprotocol, General purpose", cho phép truyền thông qua nhiều chuẩn khác nhau, đặc biệt là Zigbee và Thread. Trong số đó, EFR32MG24 là một trong những dòng mới nhất, được tối ưu hóa cho mạng IoT công suất thấp và bảo mật cao.

2.4.2 Kiến trúc phần cứng EFR32MG24



Hình 2.8. Cấu trúc khối phần cứng module xGM240P sử dụng chip EFR32MG24

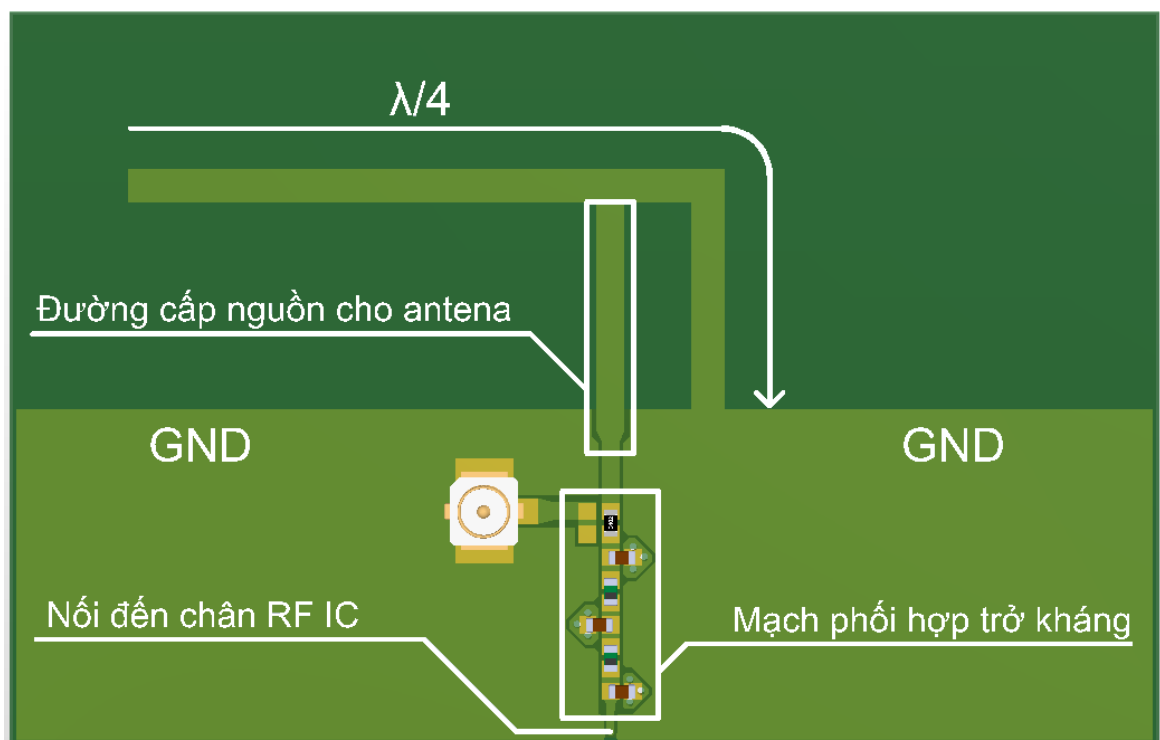
EFR32MG24[17] là một vi mạch SoC do Silicon Labs phát triển, tích hợp khả

năng xử lý mạnh mẽ, thu phát sóng không dây và nhiều ngoại vi hỗ trợ trong một chip duy nhất. Trái tim của EFR32MG24 là vi xử lý ARM Cortex-M33, hoạt động ở tần số lên tới 78 MHz. Kiến trúc Cortex-M33 không chỉ cung cấp hiệu năng cao mà còn hỗ trợ các tính năng bảo mật phần cứng như TrustZone và đơn vị xử lý dấu chấm động (FPU), cho phép xử lý tín hiệu nhanh và an toàn trong các hệ thống nhúng hiện đại.

Về bộ nhớ, EFR32MG24 được trang bị lên đến 1024 KB bộ nhớ Flash và 128 KB RAM, đảm bảo không gian đủ lớn cho các ứng dụng IoT phức tạp và khả năng cập nhật chương trình qua mạng (OTA). Bên cạnh đó, chip còn tích hợp đầy đủ các khối ngoại vi như UART, I2C, SPI, ADC, DAC, bộ định thời và GPIO có thể cấu hình linh hoạt, đáp ứng đa dạng nhu cầu kết nối với cảm biến và thiết bị ngoại vi.

Điểm nổi bật nhất của EFR32MG24 là khối radio tích hợp, hỗ trợ truyền thông không dây trong băng tần 2.4 GHz theo nhiều chuẩn như Zigbee, Thread, Bluetooth Low Energy và đặc biệt là IEEE 802.15.4. Bộ thu phát này cho phép điều chỉnh công suất phát tối đa đến +20 dBm và đạt độ nhạy thu lên đến -104 dBm, giúp đảm bảo kết nối ổn định và tiết kiệm năng lượng. Ngoài ra, chip cũng hỗ trợ nhiều chế độ tiết kiệm điện năng, từ chế độ hoạt động đầy đủ (EM0) đến chế độ ngủ sâu (EM4), phù hợp với các ứng dụng cảm biến yêu cầu hoạt động lâu dài bằng pin.

2.5 Tổng quan về Inverted-F antenna và phối hợp trở kháng cho antenna



Hình 2.9. Mô hình antenna và mạch phối hợp trở kháng

Hình sau minh họa cấu trúc thiết kế của một anten Planar Inverted-F (PIFA)[18] được tích hợp trên bảng mạch in (PCB) cùng với mạch phối hợp trở kháng cho antenna

này. Mạch phối hợp trở kháng trên sử dụng 5 phần tử ghép thành mạng chữ 'M' bao gồm các phần tử thụ động: tụ điện, cuộn cảm.

2.5.1 *Inverted-F antenna*

Anten Inverted-F (IFA) là một loại anten được sử dụng rộng rãi trong thông tin vô tuyến, đặc biệt ở tần số UHF và vi sóng. Anten này bao gồm một anten monopole chạy song song với mặt phẳng đất và được nối đất ở một đầu, với điểm cấp nguồn được đặt ở vị trí trung gian nằm giữa. Cấu trúc cơ bản của anten Inverted-F bao gồm ba phần chính: phần tử bức xạ là cánh tay ngang thường dài một phần tư bước sóng tại tần số hoạt động, chân ngắn mạch là phần tử thẳng đứng kết nối một đầu của phần tử bức xạ với mặt phẳng đất, và điểm cấp nguồn nằm giữa chân ngắn mạch và đầu hở của phần tử bức xạ. (Được mô tả ở hình 2.9)

2.5.2 *Phối hợp trở kháng cho antenna*

Phối hợp trở kháng[19] là một yêu cầu thiết yếu trong thiết kế mạch điện tử, đặc biệt quan trọng ở tần số cao và các hệ thống RF. Nguyên nhân cốt lõi xuất phát từ việc đảm bảo truyền công suất hiệu quả, để công suất phát từ IC truyền tối đa đến antenna và bức xạ ra môi trường được nhiều nhất. Trong đó, công suất truyền đến tải tính được bằng công thức:

$$P_t = \frac{E^2 \times R_t}{(R_n + R_t)^2} \quad (2.1)$$

Trong đó:

- * P_t : Công suất trên tải
- * E : Điện áp của nguồn
- * R_t : Trở kháng của tải
- * R_n : Trở kháng của nguồn

Công suất này đạt giá trị lớn nhất khi $R_n = R_t$. Điều này có nghĩa là công suất tải chỉ có thể đạt cực đại khi trở kháng tải bằng với phức liên hợp trở kháng nguồn. Ngoài ra, nếu không được phối hợp trở kháng thì tín hiệu sẽ bị phản xạ thay vì được truyền hoàn toàn đến tải. Điều này gây ra nhiều vấn đề nghiêm trọng như hình thành sóng dừng trên đường truyền do tín hiệu phản xạ tương tác với tín hiệu thuận, thứ hai là biến dạng tín hiệu thể hiện qua độ trễ dữ liệu và biến dạng pha làm giảm tỷ lệ tín hiệu trên nhiễu, và cuối cùng là tổn hao năng lượng do một phần công suất bị phản xạ về nguồn thay vì được truyền đến tải.

Kết luận chương

Chương 2 đã trình bày chi tiết về chuẩn IEEE 802.15.4, cách nó định nghĩa các tầng PHY và MAC, kiến trúc giao thức và định tuyến trong mạng Zigbee, nêu tổng quan về

RAIL, thứ giúp phần mềm trên kit log, quản lý các thông số quan trọng ở lớp PHY và MAC. Ngoài ra chương này cũng đã đề cập đến lý thuyết về antena. Tiếp theo, Chương 3 sẽ mô tả cách triển khai các thành phần của hệ thống dựa trên các lý thuyết này.

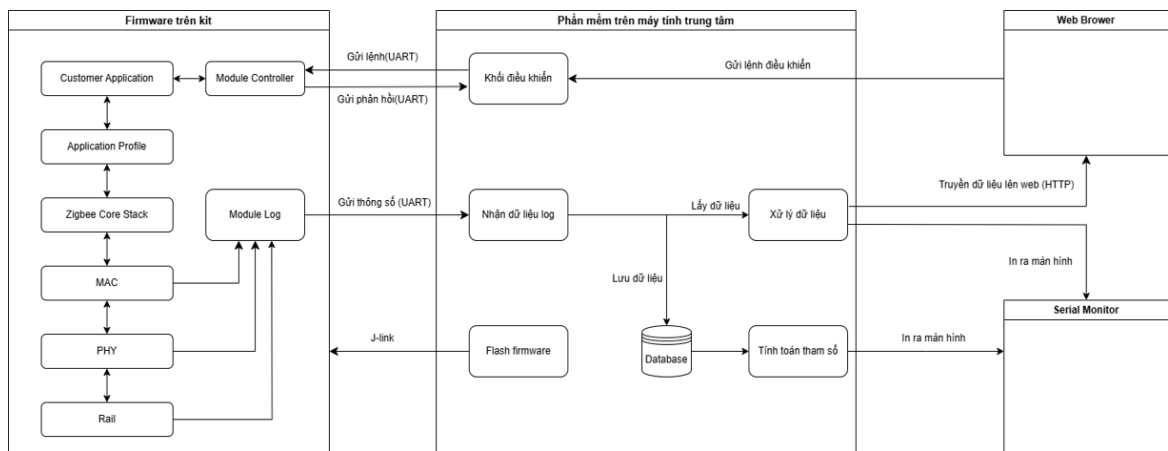
CHƯƠNG 3. THIẾT KẾ HỆ THỐNG

Mở đầu chương

Chương 3 trình bày chi tiết về cách xây dựng các thành phần của hệ thống: phần mềm nhúng trên KIT, phần mềm trên máy tính trung tâm, xây dựng giao diện hiển thị, xây dựng phần cứng có tích hợp antenna.

3.1 Tổng quan hệ thống

Để xây dựng một môi trường thử nghiệm (Testbed)[20][21] toàn diện nhằm đánh giá hiệu năng của một hệ thống truyền thông không dây, cần phải thiết kế một kiến trúc bao gồm các khối chức năng cốt lõi, hoạt động hài hòa với nhau.



Hình 3.1. Kiến trúc hệ thống

Hình 3.2 mô tả kiến trúc hệ thống Testbed, được thiết kế theo mô hình Client-Server. Hệ thống được phân chia thành ba khối chức năng chính: Firmware trên kit (đóng vai trò Client, là các thiết bị trong mạng được kiểm thử), Phần mềm trên máy tính trung tâm (đóng vai trò Server, là trạm điều khiển và phân tích) và giao diện người dùng.

3.1.1 Phần mềm nhúng trên kit

Đây là phần mềm nhúng chạy trực tiếp trên các thiết bị phần cứng Zigbee. Trong kiến trúc testbed, khối firmware này trực tiếp chạy các kịch bản kiểm thử trong môi trường vật lý thực tế. Quan trọng hơn, nó là nguồn cung cấp duy nhất những dữ liệu thô và có độ chính xác cao về hoạt động của mạng.

Zigbee Protocol Stack:

- Rail (Radio Abstraction Interface Layer): Đây là lớp trừu tượng hóa phần cứng vô tuyến, nằm ở tầng thấp nhất. Vai trò của Rail là cung cấp một bộ API đồng nhất, cho phép các lớp phần mềm phía trên có thể điều khiển chip vô tuyến mà không cần phải biết về các chi tiết phần cứng phức tạp và đặc thù của nhà sản xuất.

- **Lớp PHY và MAC:** Lớp PHY chịu trách nhiệm các tác vụ ở mức tín hiệu, như điều chế, giải điều chế và chuyển đổi bit thành sóng vô tuyến để truyền đi. Lớp MAC quản lý việc truy cập vào kênh truyền để tránh xung đột (sử dụng cơ chế CSMA/CA) và xử lý các gói tin xác nhận (Ack) ở mức liên kết dữ liệu. Các gói tin xác nhận này là cơ sở quan trọng để phát hiện một gói tin có được truyền thành công hay không.
- **Zigbee Core Stack:** Bao gồm các lớp mạng và hỗ trợ ứng dụng (APS). Đây là nơi xử lý các logic phức tạp của mạng Zigbee như hình thành mạng (PAN), định tuyến các gói tin trong mạng mesh, quản lý địa chỉ mạng và các cơ chế bảo mật.
- **Application Profile Customer Application:** Lớp Application Profile định nghĩa các quy tắc chung cho một loại ứng dụng (ví dụ: chiếu sáng thông minh), còn Customer Application là nơi logic chính của kịch bản thử nghiệm được lập trình viên triển khai. Chính tại đây, các lệnh từ máy tính trung tâm được thực thi, ví dụ như tạo và gửi một số lượng lớn các gói tin dữ liệu theo yêu cầu để kiểm tra hiệu năng mạng.

Các Module chức năng tùy chỉnh:

- **Module Controller:** Module này liên tục lắng nghe trên cổng giao tiếp nối tiếp UART để nhận các lệnh điều khiển được gửi từ máy tính trung tâm. Sau khi ra lệnh cho Customer Application thực thi, nó sẽ gửi một gói tin phản hồi về máy tính để xác nhận rằng lệnh đã được nhận và xử lý. Cơ chế này tạo ra một vòng lặp điều khiển khép kín, đảm bảo sự đồng bộ và tin cậy giữa giữa các phần trong hệ thống
- **Module Log:** Đây có thể coi là module quan trọng nhất của toàn bộ hệ thống thu thập dữ liệu. Mỗi khi có một gói tin được truyền đi, nhận về, hoặc bị lỗi, module này sẽ ghi nhận lại các thông số liên quan (ví dụ: trạng thái thành công/thất bại, cường độ tín hiệu nhận được - RSSI, số lần truyền lại). Các thông số này sau đó được đóng gói thành một định dạng nhất quán và gửi liên tục về máy tính trung tâm qua UART, làm đầu vào cho quá trình phân tích hiệu năng.

3.1.2 Phần mềm trên máy tính trung tâm

Đây là một ứng dụng phần mềm có nhiệm vụ điều phối toàn bộ quá trình thử nghiệm, từ việc ra lệnh, thu thập, lưu trữ, xử lý, phân tích cho đến trực quan hóa kết quả cuối cùng.

Khởi điều khiển: Là cầu nối nhận lệnh từ người dùng. Khi người dùng thực hiện một thao tác trên giao diện web (ví dụ: nhấn nút "Bắt đầu"), khối này sẽ dịch yêu cầu đó thành một lệnh máy tính cụ thể theo giao thức đã định sẵn và gửi đến Module Controller trên kit qua giao tiếp UART.

Luồng xử lý dữ liệu:

- Nhận dữ liệu log: Một tiến trình chạy nền liên tục lắng nghe trên cổng UART để nhận dòng dữ liệu thông số do Module Log trên kit gửi về. Việc này đảm bảo không thông tin nào từ thiết bị bị bỏ lỡ.
- Lưu dữ liệu vào Database: Dữ liệu từ module log sẽ được lưu đầy đủ trong database kèm thời gian tương ứng. Database này lưu giữ tất cả dữ liệu từ lần chạy đầu tiên, người dùng có thể theo dõi, đánh giá và thực hiện tính toán.
- Xử lý dữ liệu: Dữ liệu nhận được qua UART từ module log sẽ được giải mã và lấy ra thông tin từ đó sẽ truyền và hiển thị lên web browser.
- Tính toán tham số: Từ dữ liệu đã được xử lý, khối này áp dụng các công thức và thuật toán thống kê để tính ra các chỉ số hiệu năng quan trọng như PER (Packet Error Rate - Tỷ lệ lỗi gói), PLR (Packet Loss Rate - Tỷ lệ mất gói)

3.1.3 Giao diện và tương tác người dùng

Đây là lớp trên cùng của kiến trúc, là nơi người vận hành và nhà phát triển tương tác với hệ thống, qua 2 cách: giao diện web và serial monitor Web browser:

- Đây là giao diện đồ họa chính, được thiết kế để thân thiện với người dùng.
- Thông qua giao diện này, người dùng có thể dễ dàng cấu hình các tham số của kịch bản, ra lệnh điều khiển (bắt đầu, dừng, tạm dừng), và quan trọng nhất là xem kết quả được trực quan hóa dưới dạng các bảng động theo thời gian thực.
- Dữ liệu sau khi được xử lý sẽ được truyền từ máy tính trung tâm lên giao diện web thông qua giao thức HTTP.

Serial monitor:

- Đây là một công cụ gỡ lỗi và giám sát cấp thấp, chủ yếu dành cho các lập trình viên và kỹ sư phát triển hệ thống.
- Nó hiển thị trực tiếp dữ liệu thô hoặc các thông báo debug được in ra màn hình từ các khối xử lý. Công cụ này hữu ích để kiểm tra xem dữ liệu có đang được truyền/nhận và xử lý chính xác hay không một cách tức thời, giúp phát hiện và khắc phục sự cố một cách hiệu quả mà không cần đợi dữ liệu hiển thị hoàn chỉnh trên giao diện web.

3.1.4 Phần cứng cho hệ thống

Khi xây dựng Testbed cho mạng Zigbee, việc tự thiết kế phần cứng mang lại nhiều lợi ích quan trọng so với việc sử dụng các module thương mại có sẵn.

Đầu tiên, việc tự thiết kế cho phép tùy chỉnh phần cứng phù hợp với các yêu cầu đặc thù của Testbed. Thay vì bị giới hạn bởi các module thương mại, bạn có thể tối ưu hóa thiết kế cho từng loại thiết bị cụ thể như Coordinator, Router, hoặc End Device với các đặc tính riêng biệt về tiêu thụ năng lượng và chức năng.

Bên cạnh đó, tự thiết kế phần cứng cho phép kiểm soát đầy đủ các tính năng cần thiết cho việc thử nghiệm và đánh giá mạng Zigbee. Có thể tích hợp các cảm biến, actuator, và giao diện cần thiết mà không phụ thuộc vào những gì có sẵn trên thị trường.

Mặc dù đầu tư ban đầu có thể cao hơn, việc tự thiết kế phần cứng thường mang lại chi phí thấp hơn trong dài hạn, đặc biệt khi cần sản xuất số lượng lớn các node cho Testbed. Điều này cũng giúp tránh phụ thuộc vào giá cả và tính sẵn có của các module thương mại.

Chính vì vậy, việc tự thiết kế phần cứng cho testbed không chỉ mang lại sự linh hoạt và kiểm soát tối đa mà còn tạo điều kiện thuận lợi cho việc nghiên cứu và phát triển các giải pháp tối ưu cho mạng Zigbee trong tương lai.

3.2 Thiết kế phần mềm nhúng trên module EFR32MG24

3.2.1 Xây dựng khối tự động truyền các tham số sang máy tính trung tâm (Module Log)

Module log là thành phần quan trọng, hoạt động song song với các chức năng truyền nhận dữ liệu của hệ thống. Module này nhận thông tin về từng gói tin truyền (TX) hoặc nhận (RX) từ thư viện RAIL và Zigbee stack, thu thập các khung dữ liệu thô theo chuẩn IEEE 802.15.4 ở tầng MAC/PHY, sau đó đóng gói dữ liệu này theo định dạng nhị phân tối ưu, kèm theo các metadata quan trọng như RSSI, LQI, timestamp, trạng thái lỗi, v.v. Cuối cùng, module truyền các bản ghi này qua UART về máy tính trung tâm để phục vụ cho việc phân tích, đánh giá hiệu suất và phát hiện lỗi mạng. Khối này thực hiện log thông tin hoàn toàn độc lập với các stack Zigbee ở trên, không ảnh hưởng đến phần mềm ứng dụng của người dùng.

3.2.1.1 Mục tiêu

- Xác định các sự kiện hiện tại của module.
- Phát hiện các bản tin truyền nhận của module.
- Lấy các tham số quan trọng của bản tin.
- Đóng gói các tham số cần truyền thành 1 chuỗi Json, sau đó truyền qua máy tính trung tâm bằng UART

3.2.1.2 Cách triển khai

Khối tự động truyền thông tin này được tích hợp vào hàm xử lý sự kiện của module, trích xuất các thông tin hữu ích tùy vào từng sự kiện xảy ra. Các thông tin được truyền qua máy tính trung tâm bao gồm:

- Tên sự kiện.
- Thời gian hệ thống khi xảy ra sự kiện.
- ID của module (Là chuỗi HEX dài 16 ký tự).
- Đối với bản tin RX, còn có các tham số sau: RSSI, LQI, IsACK, SyncWord, CRCPass, SubPhyID, AntenaID, Channel, Payload.
- Đối với bản tin TX, có các tham số sau: Payload

Bảng 3.1. Các sự kiện xảy ra trong chương trình

Tên sự kiện	Mô tả sự kiện
RX FILTERED	Frame bị lọc bỏ (không phù hợp với bộ lọc)
RX CORRUPTED	Frame bị lỗi/hỏng trong quá trình nhận
RX ENDED	Kết thúc quá trình nhận frame
RX IDLED	Radio đã chuyển về trạng thái nghỉ sau khi nhận
RX ACKING	Đang chuẩn bị gửi ACK cho frame vừa nhận
RX ACK BLOCKED	Việc gửi ACK bị chặn
RX ACK SENT	ACK đã được gửi thành công
TX PENDED MAC	Gói tin đang chờ ở tầng MAC
TX PENDED PHY	Gói tin đang chờ ở tầng PHY
TX CCA SOON	Sắp thực hiện CCA
TX CCA BUSY	Kênh bận, CCA thất bại
TX BLOCKED	Việc truyền bị chặn
TX ENDED	Kết thúc quá trình truyền
TX IDLED	Radio chuyển về trạng thái idle sau khi truyền
TX ACK WAITING	Đang chờ nhận ACK
TX ACK RECEIVED	Đã nhận được ACK
TX ACK TIMEDOUT	Timeout khi chờ ACK

Bảng 3.1. mô tả các sự kiện xảy ra trong module khi hoạt động, chia ra thành 2 loại chính là các sự kiện khi nhận bản tin và các sự kiện khi truyền bản tin.

3.2.2 Xây dựng khối nhận lệnh từ UART và thực thi lệnh

3.2.2.1 Mục tiêu

Khởi nhận lệnh từ UART là thành phần hỗ trợ đọc và giải mã yêu cầu từ máy tính trung tâm. Sau đó dựa vào từng lệnh để thực thi lệnh. Kết quả được truyền qua máy tính trung tâm bằng UART khi thực hiện xong lệnh.

3.2.2.2 Cách triển khai

Khởi nhận lệnh từ máy tính trung tâm này được kết hợp với giao diện CLI của hệ thống. Cho phép người dùng sử dụng thêm các lệnh ngoài các lệnh có sẵn:

- Escan: Bắt đầu quét năng lượng trên các kênh, mỗi khi xong 1 kênh thì truyền qua UART giá trị mức năng lượng của kênh này.
- Getdata X: Lấy dữ liệu từ module với X là chỉ số của dữ liệu, module có thể lấy được các tham số: công suất phát trên kênh hiện tại, kênh hiện tại đang sử dụng, lấy ngưỡng ED-CCA(Energy Detection - Clear Channel Assessment) trên kênh hiện tại.
- Setdata X: Cho phép ghi dữ liệu vào module với X là chỉ số của dữ liệu, có thể thay đổi được các tham số: công suất phát trên kênh hiện tại, kênh hiện tại đang sử dụng, ngưỡng ED-CCA(Energy Detection - Clear Channel Assessment) trên kênh hiện tại.

3.3 Xây dựng phần mềm trên máy tính trung tâm

3.3.1 Xây dựng khối nhận và xử lý dữ liệu qua UART

3.3.1.1 Mục tiêu

Khởi nhận và xử lý dữ liệu qua UART là thành phần trung gian giữa phần cứng và phần mềm trung tâm. Nhiệm vụ của khối này là:

- Đọc liên tục dữ liệu gửi về từ KIT qua cổng UART (USB-to-Serial).
- Phân tích, nhận diện và phân loại các loại dữ liệu (gói tin Zigbee, EScan, BER test, trạng thái LED...).
- Giải mã, bóc tách thông tin theo từng lớp giao thức (IEEE 802.15.4, Zigbee Network, Security...).
- Lưu trữ dữ liệu vào cơ sở dữ liệu và cập nhật trạng thái cho các module khác (giao diện, báo cáo, điều khiển).

3.3.1.2 Cách triển khai

Khởi UART được triển khai trong lớp Node, chịu trách nhiệm quản lý kết nối với thiết bị phần cứng Zigbee qua cổng COM, khi khởi tạo node, hệ thống sẽ:

- Đóng kết nối cũ nếu đang mở để tránh xung đột.

- Mở kết nối mới với các tham số như cổng COM, tốc độ baud (mặc định 115200), và timeout.
- Tạo một daemon thread riêng để đọc dữ liệu liên tục từ cổng UART, đảm bảo luồng chính không bị block và thread này tự động dừng khi ứng dụng kết thúc.
- Khởi tạo cấu trúc dữ liệu lưu trữ tạm thời như dictionary cho kết quả quét năng lượng (EScan).

Quản lý buffer và phân tách dữ liệu:

- Dữ liệu nhận được từ UART là dạng byte stream liên tục, không có cấu trúc rõ ràng.
- Buffer kiểu bytearray được dùng để lưu trữ tạm thời dữ liệu nhận được.
- Dữ liệu được tách thành từng dòng dựa trên ký tự xuống dòng.
- Mỗi dòng dữ liệu sau khi tách được decode sang chuỗi và chuyển vào pipeline xử lý tiếp theo.

Sau khi tách dòng, hệ thống sử dụng các hàm pattern recognition) để xác định loại dữ liệu nhận được từ UART, giúp phân loại và xử lý đúng chức năng cho từng loại thông tin truyền về từ KIT Zigbee.

Dữ liệu sự kiện (Event data) là các gói tin Zigbee chứa thông tin về trạng thái mạng, dữ liệu cảm biến hoặc các sự kiện truyền thông, được nhận diện dựa trên từ khóa "Event" xuất hiện trong chuỗi dữ liệu. Khi phát hiện, hệ thống sẽ tạo một đối tượng ZigbeePacket mới, trích xuất các trường thông tin như thời gian, RSSI, LQI, ID64, kênh, payload, sau đó giải mã đa lớp theo chuẩn IEEE 802.15.4 và Zigbee. Quá trình giải mã này bao gồm phân tích Frame Control Field, địa chỉ nguồn và đích, loại khung (Beacon/Data/ACK/Command), sequence number, tiếp tục bóc tách lớp mạng Zigbee (routing, bảo mật, extended addressing) và cuối cùng là lớp payload ứng dụng. Tất cả dữ liệu đã giải mã sẽ được lưu vào cơ sở dữ liệu SQLite với timestamp chuẩn, đồng thời cập nhật vào danh sách packets trong bộ nhớ để phục vụ các module khác.

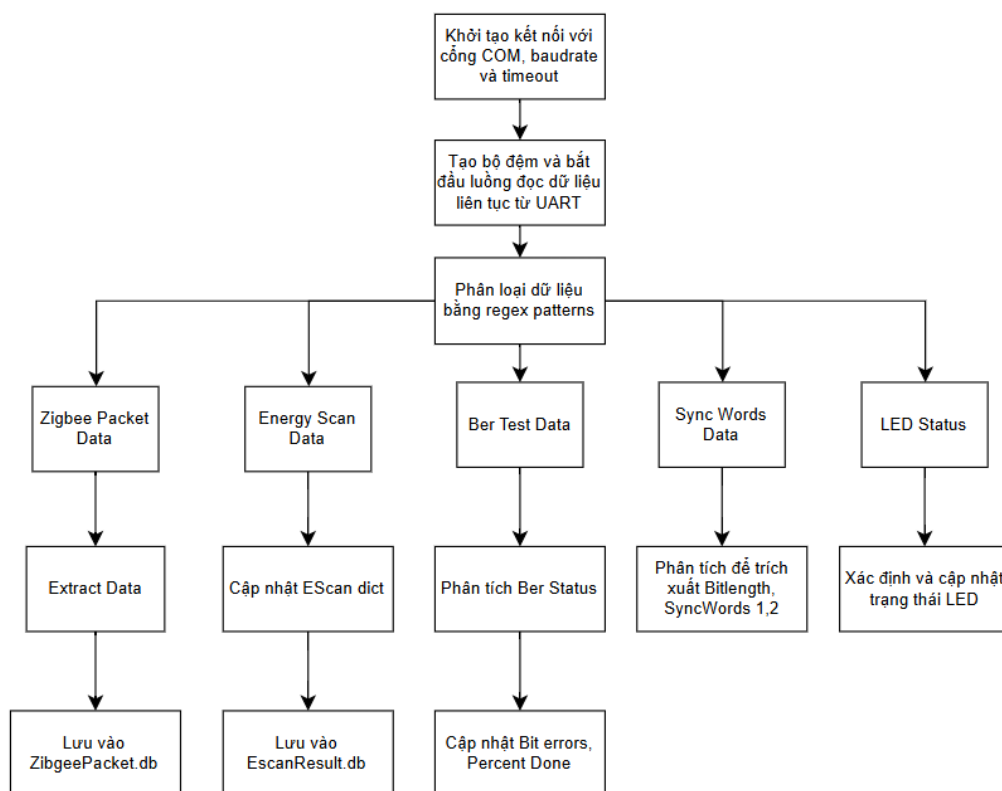
Dữ liệu quét năng lượng (EScan) là kết quả đo phổ tần trên từng kênh Zigbee, nhận diện qua từ khóa "EScan" và sử dụng regex để trích xuất số kênh và giá trị năng lượng. Sau khi nhận diện, hệ thống xác định ID node (ưu tiên lấy từ packet gần nhất hoặc tạo từ tên cổng), cập nhật giá trị vào dictionary EScan của node và lưu vào bảng escan results trong database. Điều này cho phép người dùng theo dõi chất lượng kênh, phân tích nhiễu và lựa chọn kênh tối ưu cho truyền thông Zigbee.

Dữ liệu kiểm thử BER (Bit Error Rate)[22] là thông tin về tiến trình kiểm thử tỷ lệ lỗi bit giữa hai node, nhận diện qua từ khóa "berstatus". Hệ thống sẽ phân tích các

thông số như RSSI, BitErrors, PercentDone, BitsTested, PercentBitError và cập nhật trực tiếp vào thuộc tính của node để phục vụ việc theo dõi tiến trình kiểm thử BER real-time trên dashboard.

Trạng thái đèn LED được nhận diện từ các dòng dữ liệu chứa từ khóa "LedStt" hoặc các thông báo toggle on/off. Hệ thống sẽ cập nhật trạng thái ON/OFF cho node, giúp người dùng kiểm soát và theo dõi trạng thái đèn trên thiết bị từ xa thông qua giao diện web.

Thông tin cấu hình Sync Words là các giá trị đồng bộ tần số radio, nhận diện qua từ khóa "getSyncWords". Sau khi nhận diện, hệ thống sẽ phân tích và cập nhật các giá trị cấu hình này vào node, đảm bảo đồng bộ hóa giữa các thiết bị trong mạng Zigbee.



Hình 3.2. Quy trình xử lý dữ liệu UART

3.3.2 Xây dựng hệ thống cơ sở dữ liệu

3.3.2.1 Giới thiệu và lựa chọn công nghệ

Để đảm bảo hệ thống Testbed có khả năng lưu trữ, truy xuất và phân tích dữ liệu một cách hiệu quả và bền vững, việc xây dựng một hệ thống cơ sở dữ liệu là một yêu cầu bắt buộc. Dữ liệu từ các kịch bản thử nghiệm, bao gồm hàng nghìn gói tin Zigbee và kết quả đo lường, cần được lưu trữ một cách có cấu trúc để phục vụ cho các phân tích sau này cũng như đảm bảo tính toàn vẹn của kết quả nghiên cứu.

Sau khi phân tích các yêu cầu của đề án, công nghệ SQLite[23] đã được lựa chọn làm hệ quản trị cơ sở dữ liệu chính. Quyết định này dựa trên những ưu điểm vượt trội

của SQLite trong bối cảnh một ứng dụng chạy trên máy tính cá nhân như hệ thống Testbed này:

- Kiến trúc không máy chủ (Serverless): Không giống như các hệ CSDL khác như MySQL hay PostgreSQL, SQLite không yêu cầu một tiến trình máy chủ (server process) riêng biệt để hoạt động. Nó được tích hợp trực tiếp vào ứng dụng, giúp hệ thống trở nên gọn nhẹ, giảm thiểu độ phức tạp trong cài đặt và triển khai.
- Lưu trữ trên một tệp tin duy nhất: Toàn bộ CSDL, bao gồm các bảng, chỉ mục và dữ liệu, được lưu trữ trong một tệp tin duy nhất trên đĩa. Điều này làm cho việc sao lưu, di chuyển và quản lý CSDL trở nên vô cùng đơn giản, chỉ cần sao chép một tệp tin là đủ.
- Tích hợp sẵn và dễ sử dụng trong Python: Ngôn ngữ Python đã tích hợp sẵn thư viện sqlite3, cho phép các nhà phát triển tương tác với CSDL SQLite một cách tự nhiên mà không cần cài đặt thêm bất kỳ driver hay thư viện phụ thuộc nào từ bên ngoài.
- Hiệu năng cao cho ứng dụng Desktop: Đối với các ứng dụng có lượng truy cập đồng thời ở mức thấp đến trung bình, SQLite cung cấp hiệu năng đọc và ghi rất cao, hoàn toàn đáp ứng được nhu cầu ghi log gói tin theo thời gian thực của hệ thống.

3.3.2.2 *Thiết kế cấu trúc cơ sở dữ liệu*

Hệ thống sử dụng 2 file cơ sở dữ liệu sau:

- zigbee_packets.db: Đây được xem là CSDL chính và quan trọng nhất của hệ thống. Nó chịu trách nhiệm lưu trữ thông tin chi tiết của mọi gói tin Zigbee mà hệ thống thu thập được trong quá trình hoạt động. Đây là nguồn dữ liệu gốc để phân tích hiệu năng mạng, gỡ lỗi giao thức và kiểm tra tính đúng đắn của quá trình truyền nhận.
- escan_results.db: Đây là CSDL phụ trợ, được thiết kế chuyên biệt để lưu trữ kết quả từ chức năng quét năng lượng (Energy Scan). Việc tách riêng dữ liệu này giúp cho các truy vấn liên quan đến phân tích phổ tần được thực hiện nhanh chóng và không làm ảnh hưởng đến hiệu suất của CSDL chính.

Bảng 3.2. Cấu trúc bảng zigbee_packets

Tên cột	Kiểu dữ liệu	Mô tả chi tiết và vai trò
ID	TEXT	Định danh của node đã nhận hoặc gửi gói tin. Trong hệ thống, đây là trường linh hoạt có thể chứa địa chỉ 64-bit (ID64) nếu có.
event	TEXT	Loại sự kiện liên quan đến gói tin, thường là rxPacket để chỉ một gói tin đã được nhận thành công.
data	TEXT	Chuỗi dữ liệu thô (payload) của gói tin dưới dạng hệ thập lục phân (hex), được phân tách bằng khoảng trắng. Đây là phần dữ liệu ứng dụng.
time	INTEGER	Dấu thời gian (timestamp) của gói tin do chính firmware trên KIT cung cấp, có độ chính xác cao đến microsecond.
rssi	INTEGER	Cường độ tín hiệu nhận được (Received Signal Strength Indicator), là một chỉ số quan trọng để đánh giá chất lượng tín hiệu, đơn vị dBm.
antena_id	INTEGER	ID của antenna vật lý đã được sử dụng để nhận gói tin này.
sync_word	INTEGER	Giá trị từ đồng bộ (sync word) của radio, hữu ích cho việc gỡ lỗi ở tầng vật lý.
is_ack	INTEGER	Cờ xác định đây có phải là một gói tin báo nhận (ACK) hay không (giá trị 1 hoặc 0).
crc_pass	INTEGER	Cờ xác định gói tin có vượt qua kiểm tra tính toàn vẹn dữ liệu (CRC) hay không (giá trị 1 hoặc 0).
lqi	INTEGER	Chỉ số chất lượng đường truyền (Link Quality Indicator), một thước đo khác về độ tin cậy của liên kết truyền thông.
channel	INTEGER	Kênh tần số mà gói tin được nhận, nằm trong dải tần 2.4 GHz của Zigbee (từ 11 đến 26).
created_at	TIMESTAMP	Thời gian khi bản ghi được thêm vào CSDL.

Bảng 3.2. là nơi lưu trữ chi tiết của mỗi gói tin Zigbee nhận được. Cấu trúc bảng được định nghĩa trong hàm create_database() của mã nguồn.

Bảng 3.3. Cấu trúc bảng *escan_data*

Tên cột	Kiểu dữ liệu	Mô tả chi tiết và vai trò
ID64	TEXT	Địa chỉ MAC 64-bit của node đã thực hiện quét năng lượng. Việc sử dụng NOT NULL đảm bảo mỗi kết quả đều được liên kết với một thiết bị cụ thể.
channel	INTEGER	Kênh tần số được quét (từ 11 đến 26).
energy_value	REAL	Giá trị năng lượng đo được trên kênh đó, đơn vị dBm. Kiểu REAL cho phép lưu trữ các giá trị số thực.
scan_timestamp	TIMESTAMP	Thời gian khi kết quả quét được ghi nhận.

Bảng 3.3. được tối ưu cho việc lưu trữ kết quả quét năng lượng, được định nghĩa trong hàm `create_escan_database()` của mã nguồn.

3.3.2.3 Triển khai và tích hợp vào hệ thống

Các database được khởi tạo ngay sau khi chạy chương trình lần đầu tiên, việc lưu dữ liệu vào CSDL được tích hợp chặt chẽ trong luồng xử lý dữ liệu UART, đảm bảo không có độ trễ và không bỏ sót thông tin.

Đối với gói tin Zigbee:

- Lớp ZigbeePacket có phương thức `save_to_db()` được gọi ngay sau khi dữ liệu gói tin được trích xuất thành công qua `ExtractData()`.
- Phương thức này thực hiện kết nối tới `zigbee_packets.db`, chèn dữ liệu gói tin vào bảng `zigbee_packets` với đầy đủ các trường như ID, event, data, time, rssi, lqi, channel, ...
- Sau khi ghi, kết nối được đóng lại để tránh giữ tài nguyên lâu, đồng thời có xử lý ngoại lệ để đảm bảo hệ thống không bị crash khi gặp lỗi ghi dữ liệu.

Đối với kết quả EScan:

- Hàm `save_escan_result()` được gọi trực tiếp trong phương thức `IsEScan()` của lớp Node khi phát hiện dòng dữ liệu quét năng lượng.
- Hàm này trích xuất ID node, kênh, giá trị năng lượng, và lưu vào bảng `escan_data` trong `escan_results.db`.
- Việc lưu dữ liệu được thực hiện ngay khi nhận được dữ liệu, giúp dữ liệu quét năng lượng luôn được cập nhật tức thời.

Tất cả các thao tác tương tác với CSDL, đặc biệt là các lệnh INSERT, đều được đặt trong khối try...except để bắt lỗi và xử lý ngoại lệ. Khi có lỗi như tệp CSDL bị khóa, lỗi phân quyền, hay lỗi cú pháp SQL, hệ thống sẽ ghi lại thông báo lỗi chi tiết trên console hoặc log file, giúp việc gỡ lỗi trở nên thuận tiện. Ngoài ra, việc sử dụng with hoặc context manager trong Python giúp tự động đóng kết nối CSDL sau khi thực hiện xong thao tác, tránh rò rỉ tài nguyên.

3.3.3 Xây dựng server Flask

3.3.3.1 Giới thiệu và lựa chọn công nghệ

Để điều khiển, giám sát và tương tác với toàn bộ hệ thống Testbed một cách hiệu quả, việc xây dựng một máy chủ backend (backend server) là yếu tố không thể thiếu. Máy chủ này đóng vai trò là "bộ não" trung tâm, là cầu nối giữa giao diện người dùng (frontend) và lớp logic điều khiển phần cứng. Sau khi phân tích các yêu cầu, công nghệ Flask[24] - một micro-framework mạnh mẽ và linh hoạt của Python - đã được lựa chọn để xây dựng máy chủ này.

Lý do lựa chọn Flask:

- Gọn nhẹ và linh hoạt: Flask cung cấp các công cụ cốt lõi để xây dựng ứng dụng web mà không áp đặt một cấu trúc cứng nhắc, cho phép nhà phát triển toàn quyền quyết định kiến trúc của ứng dụng.
- Dễ học và phát triển nhanh: Cú pháp của Flask đơn giản, dễ tiếp cận, giúp đẩy nhanh quá trình phát triển và gỡ lỗi.
- Hệ sinh thái mạnh mẽ: Flask có một cộng đồng lớn và nhiều tiện ích mở rộng có thể được tích hợp khi cần.
- Tương thích hoàn hảo với Python: Vì toàn bộ lớp logic (Node class, Packet class) đã được viết bằng Python, việc sử dụng Flask tạo ra một hệ thống đồng nhất, dễ dàng tích hợp và chia sẻ dữ liệu giữa các thành phần.

Máy chủ Flask trong đồ án này đảm nhiệm ba vai trò chính:

- Cung cấp các điểm cuối (endpoints) theo chuẩn RESTful để giao diện người dùng có thể gửi lệnh và yêu cầu dữ liệu.
- Quản lý trạng thái của toàn bộ hệ thống, bao gồm danh sách các node đang kết nối, trạng thái của các bài kiểm thử,...
- Điều phối các hoạt động, nhận lệnh từ người dùng và ra lệnh cho các Node object tương ứng để thực thi trên phần cứng.

3.3.3.2 Xây dựng API theo chuẩn RESTful

Nhóm API Quản lý kết nối và node:

- GET /refresh_ports: Cung cấp một phương thức để frontend có thể lấy danh sách các cổng COM đang có sẵn trên máy tính, giúp người dùng dễ dàng lựa chọn cổng để kết nối.
- POST /open_port: Đây là một API có logic xử lý mạnh mẽ. Khi nhận yêu cầu kết nối, nó không chỉ đơn thuần tạo một kết nối mới mà còn kiểm tra xem node với node_id đó đã tồn tại hay chưa. Nếu đã tồn tại, nó sẽ thực hiện quy trình "dọn dẹp" (cleanup): đóng kết nối serial cũ, xóa đối tượng Node cũ ra khỏi bộ nhớ, và chờ một khoảng thời gian ngắn để đảm bảo cổng COM được giải phóng hoàn toàn trước khi tạo kết nối mới. Điều này giúp hệ thống cực kỳ ổn định, tránh các lỗi liên quan đến xung đột tài nguyên.
- POST /close_port và POST /remove_device: Cung cấp các phương thức để người dùng ngắt kết nối và xóa một thiết bị ra khỏi giao diện quản lý một cách an toàn.

Nhóm API điều khiển chức năng Zigbee:

- POST /zigbee_action: Một endpoint đa năng được thiết kế theo mẫu "Command Pattern". Frontend chỉ cần gửi một action (ví dụ: reset, create_network, toggle_led, và API này sẽ gọi đến phương thức tương ứng của đối tượng Node. Thiết kế này giúp mã nguồn phía backend trở nên gọn gàng, dễ bảo trì và mở rộng.
- GET /get_led_state/<node_id>: Một API chuyên biệt để lấy trạng thái đèn LED của một node, phục vụ cho việc cập nhật giao diện theo thời gian thực.

Nhóm API truy xuất và hiển thị dữ liệu:

- GET /get_all_packages/<node_id>: API này được frontend gọi định kỳ (polling) để lấy danh sách tất cả các gói tin đã được thu thập. Nó không chỉ trả về dữ liệu thô mà còn bổ sung thêm các thông tin hữu ích như TypePackage (loại gói tin đã được giải mã) và RealTime (thời gian thực khi API được gọi) để hiển thị.
- POST /get_packet_detail/<node_id>: Một API rất tinh vi, thể hiện độ sâu kỹ thuật của hệ thống. Khi người dùng nhấp vào một gói tin trên giao diện, frontend sẽ gửi thông tin cơ bản của gói tin đó đến API này. Backend sẽ tìm kiếm đối tượng ZigbeePacket đầy đủ trong bộ nhớ và sau đó gọi hàm create_packet_detail() để tạo ra một cấu trúc JSON phân cấp, chi tiết, mô phỏng lại cách mà công cụ phân tích mạng chuyên nghiệp Wireshark hiển thị thông tin. Điều này cho phép người dùng xem chi tiết từng bit của các trường điều khiển, rất hữu ích cho việc gỡ lỗi và nghiên cứu sâu về giao thức.

Nhóm API tích hợp chức năng chuyên biệt (EScan, BER Test):

- EScan: POST /start_escan để bắt đầu quá trình quét và GET /get_escan_data để lấy kết quả và vẽ biểu đồ.
- bertest_open_port, bertest_close_port, bertest_config_sync: Để quản lý kết nối và cấu hình cho các node tham gia bài kiểm thử.
- bertest_start: API quan trọng nhất, nhận các tham số như node phát (TX), node nhận (RX), số lượng bit, tần số. Nó khởi tạo một thread riêng (run_bertest) để chạy tác vụ kiểm thử nặng này trong nền. Đây là một quyết định thiết kế kiến trúc then chốt, giúp giao diện người dùng không bị "đóng băng" trong suốt quá trình kiểm thử có thể kéo dài.
- bertest_status: Một API thông minh để frontend có thể polling. Nó có logic để kiểm tra xem bài test có đang chạy hay không. Nếu đang chạy, nó sẽ lấy dữ liệu live data từ các thuộc tính của node RX. Nếu đã kết thúc, nó sẽ trả về kết quả cuối cùng đã được lưu trữ.
- bertest_stop, bertest_clear_results: Cung cấp các tiện ích để quản lý vòng đời của bài kiểm thử.

3.3.3.3 Quản lý trạng thái và xử lý đồng thời

Quản lý trạng thái toàn cục: Server sử dụng các dictionary toàn cục (nodes, bertest_nodes) và các biến (bertest_running) để lưu trữ trạng thái của toàn bộ hệ thống. Mọi thông tin về các node đang kết nối, trạng thái của chúng, và tiến trình của các bài test đều được lưu trữ ở đây. Đây là một phương pháp quản lý trạng thái đơn giản nhưng hiệu quả cho quy mô của ứng dụng này.

Đảm bảo tính đồng thời: Việc sử dụng threaded=True và các thread riêng cho các tác vụ dài hơi (đọc UART, chạy BER test) là nền tảng cho khả năng xử lý đồng thời của hệ thống. Nó cho phép server có thể vừa chạy một bài test BER, vừa nhận và xử lý các gói tin Zigbee, vừa trả lời các yêu cầu cập nhật giao diện từ nhiều người dùng (nếu có) mà không bị xung đột.

3.4 Xây dựng Engine giải mã các loại gói tin

Trong một hệ thống kiểm thử mạng không dây, việc chỉ thu thập dữ liệu thô là chưa đủ. Để thực sự hiểu được hoạt động của mạng, cần phải có một cơ chế phân tích, từng gói tin được truyền đi. Nó được xây dựng với mục tiêu biến những chuỗi byte hex nhận được từ UART thành thông tin có cấu trúc, phân cấp theo từng lớp giao thức.

3.4.0.1 Tầng IEEE 802.15.4

Beacon Frame được phát ra định kỳ bởi các thiết bị có vai trò Coordinator hoặc Router. Chúng có nhiệm vụ quảng bá sự tồn tại của mạng PAN, cho phép các thiết bị

mới khám phá và xin gia nhập mạng. Engine nhận diện Beacon Frame thông qua 3 bit đầu tiên của Frame Control Field có giá trị 000. Sau khi xác định, quá trình giải mã diễn ra như sau:

- **Frame Control Field Analysis:** Engine phân tích đầy đủ 16 bit FCF, trích xuất các thông tin như Security Enabled, Frame Pending, ACK Request, PANID Compression, Frame Version, và các chế độ địa chỉ.
- **Sequence Number:** Đọc số thứ tự của gói tin để theo dõi và phát hiện mất gói.
- **Addressing Information:** Tùy theo SrcAddrMode, engine sẽ đọc Source PAN ID và địa chỉ nguồn (có thể là địa chỉ ngắn 16-bit hoặc mở rộng 64-bit).
- **Payload Processing:** Phần payload chứa Superframe Specification và các thông tin cấu hình mạng khác.

Data Frame (Frame Type: 001) là loại gói tin quan trọng và phức tạp nhất, có nhiệm vụ vận chuyển dữ liệu từ tầng ứng dụng qua mạng không dây. Mọi thông tin từ cảm biến, lệnh điều khiển thiết bị, hay dữ liệu trạng thái đều được đóng gói trong Data Frame. Quy trình giải mã đa lớp như sau:

- Tầng IEEE 802.15.4:

- * **Frame Control Analysis:** Giải mã đầy đủ FCF với 10 trường thông tin chi tiết.
- * **Addressing Complete:** Xử lý addressing phức tạp bao gồm Destination PAN ID, Destination Address, Source Address với hỗ trợ đầy đủ các chế độ địa chỉ (None, Reserved, Short, Extended).
- * **PAN ID Compression Handling:** Logic thông minh xử lý việc có/không có Source PAN ID dựa trên cờ PANIDCompression.

- Tầng Zigbee Network: Nếu payload còn đủ dữ liệu, engine sẽ gọi ParseZigbeeNetwork() để bóc tách lớp mạng ZigBee:

- * **Network Frame Control:** Phân tích 16-bit NWK FCF để xác định FrameType (Data/Command), ProtocolVersion, DiscoverRoute, Multicast, Security flags.
- * **Network Addressing:** Đọc Destination và Source Network Address (16-bit).
- * **Routing Information:** Trích xuất Radius (TTL) và Sequence Number của lớp mạng.
- * **Extended Features:** Xử lý Extended Destination/Source Address (64-bit), Multicast Control, Source Route List nếu có.

- Tầng Zigbee Security: Nếu cờ Security được bật, engine tiếp tục với ParseZigbeeNetworkSecu

- * Security Control Analysis: Phân tích SecurityLevel (0-7), KeyIdentifier, ExtendedNonce flags.
 - * Frame Counter Processing: Đọc 4-byte Frame Counter (Little Endian) để chống replay attack.
 - * Extended Nonce Handling: Nếu ExtendedNonce=true, đọc thêm 8-byte Source Address.
 - * Key Sequence: Xử lý Key Sequence Number nếu KeyIdentifier=1.
 - * MIC Processing: Tính toán và xử lý Message Integrity Code dựa trên Security Level: level 0: không mã hóa, không MIC; level 1,5: MIC 32 bit; level 2,6: MIC 64 bit; level 3,7: MIC 128 bit
- Application payload: Sau khi bóc tách tất cả các lớp header, phần dữ liệu còn lại chính là payload ứng dụng (có thể đã được mã hóa).

ACK Frame (Frame Type: 010) là cơ chế nền tảng đảm bảo độ tin cậy trong truyền thông không dây. Khi một thiết bị gửi gói tin có bật cờ "ACK Request", thiết bị nhận sẽ gửi lại ACK để xác nhận đã nhận thành công:

- Chỉ chứa FCF và Sequence Number để tiết kiệm băng thông.
- Không có addressing, ACK được gửi ngay lập tức bởi thiết bị nhận, không cần thông tin địa chỉ phức tạp.
- Xử lý nhanh với việc reset các trường không sử dụng.

Command Frame (Frame Type: 011) được sử dụng cho các tác vụ quản lý và điều khiển ở tầng MAC, không dùng để truyền dữ liệu ứng dụng mà để quản lý kết nối và hoạt động của mạng. Quy trình giải mã:

- Standard Addressing: Xử lý đầy đủ addressing như Data Frame.
- Command ID Recognition: Engine có khả năng nhận diện các lệnh cụ thể:

Bảng 3.4. Các loại Command Frame được hỗ trợ trong hệ thống

Command ID (Hex)	Tên Lệnh	Mô tả Chức năng và Vai trò
0x01	Association Request	Yêu cầu gia nhập mạng được gửi từ thiết bị muốn tham gia vào mạng PAN hiện có. Đây là bước đầu tiên trong quá trình thiết lập kết nối giữa thiết bị cuối (End Device/Router) và Coordinator/Router. Gói tin này chứa thông tin về khả năng của thiết bị (device capabilities) và yêu cầu được cấp phát một địa chỉ mạng ngắn (16-bit).
0x02	Association Response	Phản hồi từ Coordinator/Router đối với Association Request. Gói tin này thông báo kết quả của quá trình gia nhập mạng - thành công hay thất bại. Nếu thành công, nó sẽ chứa địa chỉ mạng ngắn (short address) được cấp phát cho thiết bị mới. Nếu thất bại, nó sẽ chứa mã lỗi (error code) giải thích lý do từ chối.
0x04	Data Request	Yêu cầu dữ liệu được gửi từ thiết bị con (child device) đến thiết bị cha (parent device - thường là Coordinator hoặc Router). Lệnh này được sử dụng trong cơ chế <i>indirect transmission</i> , khi thiết bị cha có dữ liệu chờ gửi cho thiết bị con nhưng thiết bị con đang ở chế độ tiết kiệm năng lượng (sleep mode). Thiết bị con sẽ thức dậy định kỳ và gửi Data Request để kiểm tra có dữ liệu chờ hay không.
0x07	Beacon Request	Yêu cầu phát Beacon được gửi để khám phá các mạng PAN có sẵn trong vùng lân cận. Khi nhận được lệnh này, tất cả các Coordinator và Router trong tầm nghe sẽ phản hồi bằng cách phát gói tin Beacon chứa thông tin về mạng của họ (PAN ID, địa chỉ, khả năng chấp nhận thiết bị mới, v.v.). Đây là cơ chế quan trọng cho quá trình <i>network discovery</i> và <i>network selection</i> .

- Intelligent Typing: Engine không chỉ xác định là "Command" mà còn phân loại chính xác loại lệnh cụ thể, ví dụ "Association Request" thay vì "Command(01)".

3.4.0.2 Tầng Zigbee Network

Network Data Frame (NWK Frame Type: 00):

- Routing Management: Xử lý Discover Route với 2-bit value (not just boolean).
- Multicast Support: Phân tích Multicast Control field.
- Source Routing: Giải mã Source Route List với RelayCount và RelayIndex.
- Extended Addressing: Hỗ trợ 64-bit extended addresses cho cả source và destination.

Network Command Frame (NWK Frame Type: 01): Các lệnh quản lý mạng ZigBee như route discovery, leave notification, route record được xử lý với cùng header format như Network Data nhưng có payload chứa các lệnh điều khiển mạng cụ thể.

3.4.0.3 Tầng Zigbee security

Engine hỗ trợ đầy đủ 8 mức bảo mật ZigBee (0-7) với khả năng xử lý:

- Encryption Detection: Phân biệt giữa các gói tin có/không có mã hóa AES.
- MIC Validation: Xử lý Message Integrity Code với độ dài khác nhau (32/64/128-bit).
- Key Management: Hỗ trợ các loại key khác nhau (Network Key, Link Key, etc.).
- Nonce Handling: Xử lý Extended Nonce với 64-bit Source Address.
- Replay Protection: Sử dụng Frame Counter để chống tấn công lặp lại.

3.4.0.4 Kết quả giải mã các loại gói tin

Zigbee Data là loại gói tin cơ bản dùng để truyền dữ liệu ứng dụng giữa các thiết bị trong mạng Zigbee. Các gói tin này có thể được gửi theo dạng unicast (từ một nguồn đến một đích cụ thể), multicast (từ một nguồn đến một nhóm thiết bị) hoặc broadcast (từ một nguồn đến tất cả các thiết bị trong mạng). Zigbee Data đóng vai trò trung tâm trong việc truyền tải thông tin cảm biến, điều khiển hoặc các dữ liệu ứng dụng khác giữa các thiết bị, đảm bảo mạng hoạt động đúng chức năng mong muốn. Hình 3.3. đến 3.6. là kết quả phân tích gói tin Zigbee data với raw data là "32 41 88 5B 33 03 FF FF 8A 71 09 12 FC FF 8A 71 01 14 6B 4B F6 FE FF 14 2E 84 28 01 D0 02 00 6B 4B F6 FE FF 14 2E 84 00 13 19 F9 93 06 F1 B5 D5 02"

PHY HEADER

Packet Length:

50 Bytes (0x32)

FRAME CONTROL FIELD

Frame Control:

0x8841

.....001

Frame Type: Data (1)

.....0...

Security Enabled: False

.....0...

Frame Pending: False

.....0...

ACK Required: False

.....1...

PAN ID Compression: True

..10.....

Destination Address Mode: Short (2)

..00.....

Frame Version: 802.15.4-2003 (0)

10.....

Source Address Mode: Short (2)

ADDRESSING FIELDS

Sequence:

5B

Destination PAN ID:

0333

Destination Address:

FFFF

Source Address:

718A

Hình 3.3. Kết quả giải mã tầng IEEE 802.15.4

NETWORK FRAME CONTROL

Frame Control:

0x1209

.....01

Frame Type: Command (1)

.....0010...

Protocol Version: 2

.....00.....

Discover Route: False

.....0...

Multicast: False

.....1.....

Security: True

NETWORK ADDRESSING

Destination Address:

FFFC

Source Address:

718A

Radius:

01

Sequence:

14

Hình 3.4. Kết quả giải mã tầng Zigbee Network

SECURITY CONTROL	
Security Level:	0
Key Identifier:	1
Extended Nonce:	True
SECURITY FIELDS	
Frame Counter:	0002D001
Source Address:	842E14FFFFF64B6B
Key Sequence:	00

Hình 3.5. Kết quả giải mã tầng Zigbee Security

APPLICATION PAYLOAD [9 BYTES]	
PAYLOAD DATA	
0000:	13 19 F9 93 06 F1 B5 D5 02
RAW DATA [49 BYTES]	
COMPLETE PACKET HEX DUMP	
0000:	32 41 88 5B 33 03 FF FF 8A 71 09 12 FC FF 8A 71 2A [3....q....q
0010:	01 14 6B 4B F6 FE FF 14 2E 84 28 01 D0 02 00 6B ..kK.....k
0020:	4B F6 FE FF 14 2E 84 00 13 19 F9 93 06 F1 B5 D5 K.....
0030:	02 .

Hình 3.6. Kết quả giải mã Application Payload

Gói tin ACK là gói xác nhận được gửi từ thiết bị nhận về thiết bị gửi để báo rằng một gói dữ liệu đã được nhận thành công. Trong Zigbee, ACK tồn tại ở lớp MAC (theo chuẩn IEEE 802.15.4) và có thể được bật mặc định cho các gói unicast, giúp tăng độ tin cậy truyền thông bằng cách giảm khả năng mất gói tin. Nếu thiết bị gửi không nhận được ACK trong một khoảng thời gian nhất định, nó sẽ hiểu là gói tin bị lỗi và có thể gửi lại. Hình 3.7. là kết quả phân tích gói tin ACK với raw data là " 05 12 00 6B"

PHY HEADER

Packet Length:

5 bytes (0x05)

FRAME CONTROL FIELD

Frame Control:

0x0012

.....010

Frame Type: ACK (2)

.....0...

Security Enabled: False

.....1.....

Frame Pending: True

.....0.....

ACK Required: False

.....0.....

PAN ID Compression: False

..00.....

Destination Address Mode: None (0)

..00.....

Frame Version: 802.15.4-2003 (0)

00.....

Source Address Mode: None (0)

ADDRESSING FIELDS

Sequence:

6B

Destination PAN ID:

N/A

Destination Address:

N/A

Source Address:

N/A

Hình 3.7. Kết quả giải mã gói ACK

Gói tin Beacon Request là gói broadcast do thiết bị mới (end device hoặc router) gửi ra khi muốn tìm kiếm các mạng Zigbee xung quanh. Khi nhận được Beacon Request, các coordinator hoặc router đang hoạt động trên kênh đó sẽ phản hồi bằng Beacon. Quá trình này giúp thiết bị mới phát hiện và lựa chọn mạng để gửi Association Request, đóng vai trò quan trọng trong quá trình khám phá và gia nhập mạng Zigbee. Hình 3.8. là kết quả phân tích gói tin Beacon Request với raw data là "0A 03 08 66 FF FF FF 07"

PHY HEADER

Packet Length:

10 bytes (0x0A)

FRAME CONTROL FIELD

Frame Control:

0x0803

.....011

Frame Type: Command (3)

.....0...

Security Enabled: False

.....0....

Frame Pending: False

.....0...

ACK Required: False

.....0....

PAN ID Compression: False

..10.....

Destination Address Mode: Short (2)

..00.....

Frame Version: 802.15.4-2003 (0)

00.....

Source Address Mode: None (0)

ADDRESSING FIELDS

Sequences:

64

Destination PAN ID:

FFFF

Destination Address:

FFFF

Source Address:

N/A

Hình 3.8. Kết quả giải mã gói tin Beacon Request

Gói tin Beacon là gói quảng bá được gửi định kỳ bởi coordinator hoặc router để thông báo sự tồn tại của mạng Zigbee trên một kênh tần số nhất định. Beacon chứa thông tin như PAN ID, thông số mạng, trạng thái cho phép thiết bị mới tham gia và các thông tin cấu hình khác. Các thiết bị mới quét kênh sẽ dựa vào Beacon để xác định mạng nào đang hoạt động và lựa chọn tham gia phù hợp. Hình 3.9. là kết quả phân tích gói tin Beacon với raw data là "1C 00 80 A6 E2 57 18 9C FF 0F 00 00 00 22 8C A6 4C FA F4 E3 A4 2A 3D FF FF FF 00"

PHY HEADER

Packet Length:

28 bytes (0x1C)

FRAME CONTROL FIELD

Frame Control:

0x8000

.....000

Frame Type: Beacon (0)

.....0...

Security Enabled: False

.....0

Frame Pending: False

.....0..

ACK Required: False

.....0...

PAN ID Compression: False

..00.....

Destination Address Mode: None (0)

..00.....

Frame Version: 802.15.4-2003 (0)

10.....

Source Address Mode: Short (2)

ADDRESSING FIELDS

Sequence:

A6

Destination PAN ID:

57E2

Destination Address:

N/A

Source Address:

9C18

Hình 3.9. Kết quả giải mã gói tin Beacon

Gói tin Association Request được gửi từ thiết bị muốn tham gia mạng (thường là end device hoặc router) đến coordinator hoặc router cha. Gói tin này chứa thông tin nhận diện thiết bị và yêu cầu được phép gia nhập vào mạng Zigbee. Association Request là bước đầu tiên trong quá trình thiết bị mới đăng ký tham gia mạng, đóng vai trò khởi tạo mối quan hệ giữa thiết bị con và thiết bị cha trong cấu trúc mạng Zigbee. Hình 3.10. là kết quả phân tích gói tin Association Request với raw data là "15 23 C8 6A 33 03 55 59 FF FF 6B 4B F6 FE FF 14 2E 84 01 8E"

PHY HEADER	
Packet Length:	21 bytes (0x15)

FRAME CONTROL FIELD	
Frame Control:	0xC823
.....011	Frame Type: Command (3)
.....0....	Security Enabled: False
.....0....	Frame Pending: False
.....1....	ACK Required: True
.....0....	PAN ID Compression: False
..10.....	Destination Address Mode: Short (2)
.00.....	Frame Version: 802.15.4-2003 (0)
11.....	Source Address Mode: Extended (3)

ADDRESSING FIELDS	
Sequence:	6A
Destination PAN ID:	0333
Destination Address:	5955
Source Address:	N/A

Hình 3.10. Kết quả giải mã gói tin Association Request

Sau khi nhận được Association Request, coordinator hoặc router sẽ gửi lại gói tin Association Response để trả lời cho thiết bị yêu cầu gia nhập. Association Response thông báo kết quả của quá trình đăng ký, bao gồm việc chấp nhận hoặc từ chối yêu cầu, cũng như cung cấp địa chỉ mạng (network address) cho thiết bị mới nếu được chấp nhận. Gói tin này hoàn tất quá trình kết nối thiết bị mới vào mạng Zigbee. Hình 3.11. là kết quả phân tích gói tin Association Response với raw data là "1B 63 CC 65 33 03 6B 4B F6 FE FF 14 2E 84 FE 4B F6 FE FF 14 2E 84 02 C6 7B 00"

PHY HEADER

Packet Length:

27 bytes (0x1B)

FRAME CONTROL FIELD

Frame Control:

0xCC63

.....011

Frame Type: Command (3)

.....0....

Security Enabled: False

.....0....

Frame Pending: False

.....1....

ACK Required: True

.....1....

PAN ID Compression: True

..11.....

Destination Address Mode: Extended (3)

..00.....

Frame Version: 802.15.4-2003 (0)

11.....

Source Address Mode: Extended (3)

ADDRESSING FIELDS

Sequence:

65

Destination PAN ID:

0333

Destination Address:

N/A

Source Address:

N/A

Hình 3.11. Kết quả giải mã gói tin *Association Response*

Gói tin Data Request thường được gửi bởi end device đến thiết bị cha (coordinator hoặc router) để hỏi xem có dữ liệu nào đang chờ gửi cho nó hay không. Cơ chế này đặc biệt quan trọng với các thiết bị tiết kiệm năng lượng (sleepy end device), bởi chúng chỉ thức dậy định kỳ để gửi Data Request và nhận dữ liệu nếu có. Data Request giúp giảm tiêu thụ năng lượng và đảm bảo thiết bị vẫn nhận được thông tin cần thiết từ mạng. Hình 3.12. là kết quả phân tích gói tin Data Request với raw data là "12 63 C8 6B 33 03 55 59 6B 4B F6 FE FF 14 2E 84 04"

PHY HEADER

Packet Length:

18 bytes (0x12)

FRAME CONTROL FIELD

Frame Control:

0xC863

.....011

Frame Type: Command (3)

.....0---

Security Enabled: False

.....0----

Frame Pending: False

.....1----

ACK Required: True

.....1----

PAN ID Compression: True

..10.....

Destination Address Mode: Short (2)

..00.....

Frame Version: 802.15.4-2003 (0)

11.....

Source Address Mode: Extended (3)

ADDRESSING FIELDS

Sequence:

6B

Destination PAN ID:

0333

Destination Address:

5955

Source Address:

N/A

Hình 3.12. Kết quả giải mã gói tin Data Request

3.5 Xây dựng giao diện hiển thị

3.5.1 Khái quát lại về luồng dữ liệu

Thu thập dữ liệu từ phần cứng: Khi người dùng thao tác trên giao diện web (ví dụ: gửi lệnh điều khiển, kiểm thử, hoặc đơn giản chỉ giám sát node), các node vật lý sẽ truyền dữ liệu gói tin Zigbee về máy chủ thông qua kết nối UART. Quá trình này được thực hiện liên tục nhờ một thread đọc dữ liệu của mỗi node (ReadFromUART). Mỗi dòng dữ liệu nhận được sẽ được kiểm tra định dạng, phân loại (gói tin, trạng thái, sự kiện, v.v.) và xử lý tương ứng.

Trích xuất và giải mã gói tin Zigbee: Khi nhận được một dòng dữ liệu chứa thông tin gói tin Zigbee (thường có định dạng Event: ...Data: ...ID64: ...), hệ thống sẽ sử dụng class ZigbeePacket để trích xuất các trường dữ liệu và giải mã từng lớp giao thức.

Lưu trữ dữ liệu vào cơ sở dữ liệu: Sau khi giải mã, mỗi đối tượng ZigbeePacket sẽ được lưu vào cơ sở dữ liệu SQLite thông qua hàm save_to_db. Các trường quan trọng như ID64, event, data, thời gian, RSSI, LQI, CRCPass, channel, v.v. sẽ được lưu vào bảng zigbee_packets cùng với timestamp thực tế.

Truy xuất và phân tích dữ liệu trên giao diện web:

- Lấy danh sách gói tin: Khi người dùng yêu cầu trên giao diện (ví dụ: xem tất cả gói tin của một node), backend sẽ lấy danh sách các đối tượng ZigbeePacket từ node tương ứng, chuyển đổi ra định dạng bảng (dictionary) với các trường như

LQI, ID64, CRCPass, SyncWord, RSSI, Channel, Payload, TypePackage, thời gian, v.v., rồi trả về frontend dưới dạng JSON để hiển thị dạng bảng hoặc biểu đồ.

- Xem chi tiết gói tin: Khi người dùng chọn một gói tin cụ thể, backend sẽ tìm đúng đối tượng ZigbeePacket, tạo ra cấu trúc dữ liệu chi tiết theo từng lớp giao thức, bao gồm đầy đủ các trường của IEEE 802.15.4, Zigbee Network, Zigbee Security và Application Payload. Thông tin này được trả về frontend để hiển thị trực quan, giúp người dùng dễ dàng phân tích, debug hoặc nghiên cứu sâu về hoạt động mạng.

3.5.2 Các chức năng của giao diện Web

3.5.2.1 Quản lý node và kết nối phần cứng

Giao diện web cho phép người dùng trực tiếp thao tác với hệ thống phần cứng Zigbee một cách trực quan và thuận tiện nhất. Người dùng có thể:

- Tự động phát hiện và hiển thị các cổng COM khả dụng: Khi mở giao diện, người dùng có thể nhấn nút "Làm mới cổng" để hệ thống tự động quét các cổng serial đang kết nối với máy tính. Danh sách này luôn được cập nhật, giúp người dùng dễ dàng nhận biết các thiết bị mới vừa được cắm vào hoặc đã tháo ra.
- Thêm và khởi tạo node mới: Người dùng có thể chọn cổng COM, đặt tên node, cấu hình baudrate và nhấn "Kết nối" để tạo mới một node. Hệ thống sẽ tự động khởi tạo đối tượng node, mở kết nối UART, khởi động thread đọc dữ liệu và cập nhật trạng thái node lên giao diện.
- Theo dõi trạng thái kết nối real-time: Mỗi node được hiển thị dưới dạng một card riêng biệt trên dashboard, với các chỉ báo trạng thái như đang kết nối, đã kết nối, mất kết nối, hoặc đang thực hiện thao tác. Các trạng thái này được mã hóa màu sắc (xanh lá, vàng, đỏ, xanh dương) và có thể đi kèm hiệu ứng động để tăng tính trực quan.
- Đóng/mở kết nối node: Người dùng có thể đóng port, xóa node khỏi hệ thống hoặc khởi tạo lại node chỉ với một thao tác đơn giản trên giao diện mà không cần thiệp thủ công vào phần cứng.

3.5.2.2 Điều khiển và giám sát thiết bị Zigbee

Giao diện web cung cấp bộ công cụ điều khiển mạnh mẽ cho từng node, giúp người dùng có thể thao tác từ xa với thiết bị Zigbee:

- Gửi lệnh điều khiển trực tiếp: Thông qua các nút chức năng, người dùng có thể gửi các lệnh như reset thiết bị, tạo mạng Zigbee mới, tham gia mạng, rời mạng, mở mạng cho phép thiết bị khác gia nhập, bật/tắt/toggle đèn LED trên node, hoặc cấu hình các tham số đặc biệt như SyncWords.

- Theo dõi trạng thái LED và các thông số thiết bị: Giao diện luôn cập nhật trạng thái LED của từng node (ON/OFF), đồng thời hiển thị các thông số như RSSI, LQI, kênh hoạt động, số lượng gói tin đã nhận, v.v. Mọi thay đổi trạng thái đều được phản ánh tức thời lên giao diện.
- Thực hiện các thao tác đặc thù Zigbee: Chẳng hạn, người dùng có thể yêu cầu node thực hiện Energy Scan, cấu hình lại thông số đồng bộ, hoặc gửi các lệnh đặc biệt phục vụ kiểm thử và gỡ lỗi.

3.5.2.3 *Quản lý, hiển thị và phân tích gói tin realtime*

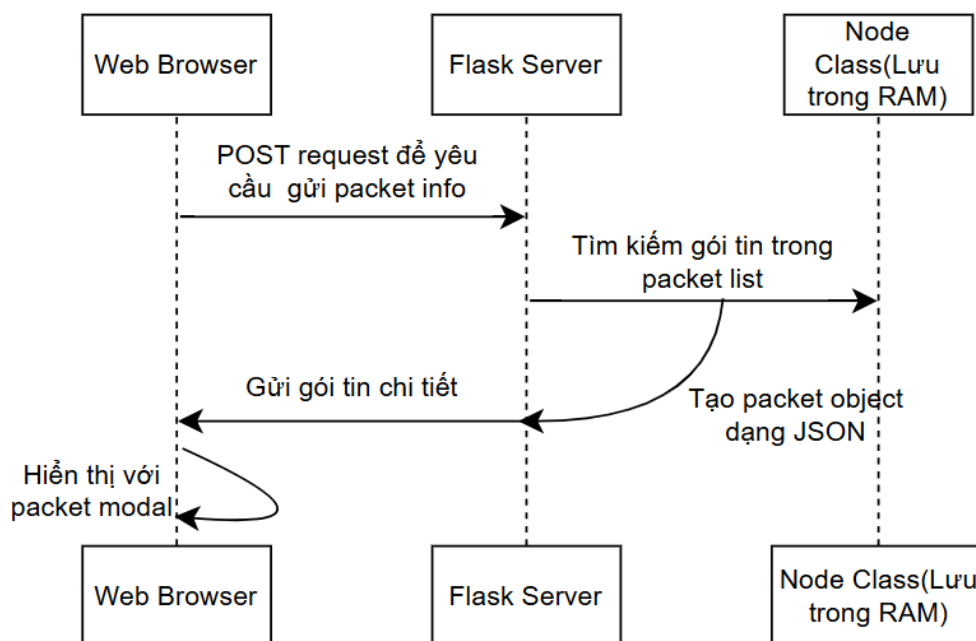
Bảng gói tin cập nhật liên tục: Tất cả các gói tin Zigbee nhận được từ các node sẽ được hiển thị trên một bảng dữ liệu lớn, với các trường thông tin như thời gian nhận, ID thiết bị, loại gói tin, giá trị RSSI, LQI, kênh, độ dài payload, trạng thái CRC, v.v. Bảng này có khả năng cập nhật real-time, đảm bảo người dùng luôn theo dõi được dữ liệu mới nhất.

Phân loại và mã màu gói tin: Hệ thống tự động nhận diện loại gói tin (Beacon, Data, ACK, Command) dựa trên giải mã frame control field, và hiển thị chúng với màu sắc hoặc biểu tượng riêng biệt giúp người dùng dễ dàng nhận biết.

Lọc và tìm kiếm nâng cao: Người dùng có thể lọc bảng theo loại gói tin, giá trị RSSI, kênh hoạt động, hoặc tìm kiếm nhanh theo ID thiết bị, hỗ trợ phân tích hiệu quả khi số lượng gói tin lớn.

Xem chi tiết gói tin: Khi nhấn vào một gói tin bất kỳ, giao diện sẽ mở một modal hiển thị chi tiết từng trường của gói tin theo cấu trúc phân lớp (IEEE 802.15.4, Zigbee Network, Security, Payload), phân tích từng bit, từng trường, mô phỏng cách hiển thị của Wireshark. Người dùng có thể xem breakdown từng bit của frame control, các trường địa chỉ, thông tin bảo mật, và cả payload dưới dạng hexdump.

Chức năng export dữ liệu: Người dùng có thể xuất dữ liệu gói tin ra các định dạng CSV, JSON hoặc PDF phục vụ cho việc phân tích ngoài hệ thống hoặc lưu trữ lâu dài.



Hình 3.13. Sơ đồ luồng xử lý ZigBee packet detail modal

Hình 3.13. mô tả luồng xử lý chi tiết của tính năng Packet Detail Modal trong hệ thống phân tích ZigBee:

- Khởi tạo yêu cầu từ Web Browser: Quá trình bắt đầu khi người dùng thực hiện một hành động trên Web Browser, cụ thể là click vào một packet trong bảng danh sách. Trình duyệt sẽ gửi một POST request đến Flask Server với thông tin chi tiết về packet cần xem. Request này chứa các thông tin định danh quan trọng như Event, TimeUs, ID64 và các thông số kỹ thuật khác của packet đã được chọn.
- Xử lý tìm kiếm tại Flask Server: Flask Server nhận được POST request và tìm kiếm gói tin trong packet list bằng cách truy cập trực tiếp vào Node Class đang lưu trữ trong RAM. Server sẽ duyệt qua danh sách các packet đã được thu thập và phân tích để tìm ra packet chính xác dựa trên các thông tin định danh như Event và TimeUs. thiết.
- Cơ chế fallback và tạo packet object: Trong trường hợp không tìm thấy packet trong bộ nhớ (có thể do packet đã bị xóa khỏi memory buffer), Flask Server sẽ thực hiện cơ chế "Tạo packet object dạng JSON" như một giải pháp dự phòng. Quá trình này tạo ra một packet object mới từ dữ liệu được gửi lên từ Web Browser, đảm bảo người dùng vẫn có thể xem được thông tin cơ bản của packet dù không có phân tích protocol đầy đủ.
- Phản hồi và hiển thị kết quả: Sau khi hoàn tất quá trình xử lý, Flask Server sẽ gửi gói tin cho Web Browser dưới dạng JSON response. Dữ liệu này chứa thông tin chi tiết về packet bao gồm phân tích 3 lớp protocol: IEEE 802.15.4, ZigBee

Network và ZigBee Security. Cuối cùng, Web Browser nhận được dữ liệu và hiển thị với packet modal, tạo ra một cửa sổ popup chi tiết cho phép người dùng xem đầy đủ thông tin kỹ thuật của packet đã chọn.

3.5.2.4 *Quản lý và hiển thị kết quả EScan*

Khởi động Energy Scan cho từng node: Người dùng có thể chọn node và bắt đầu quá trình quét năng lượng trên các kênh Zigbee chỉ với một thao tác trên giao diện.

Hiển thị kết quả EScan trực quan: Kết quả quét năng lượng được cập nhật real-time và hiển thị dưới dạng bảng hoặc biểu đồ trực quan (bar chart/line chart), giúp người dùng nhận biết ngay các kênh bị nhiễu, từ đó chọn kênh tối ưu cho mạng Zigbee.

Lưu trữ và truy xuất dữ liệu EScan: Toàn bộ kết quả quét được lưu trữ vào cơ sở dữ liệu, cho phép người dùng xem lại lịch sử quét, so sánh các lần quét khác nhau, hoặc xuất dữ liệu để phân tích sâu hơn.

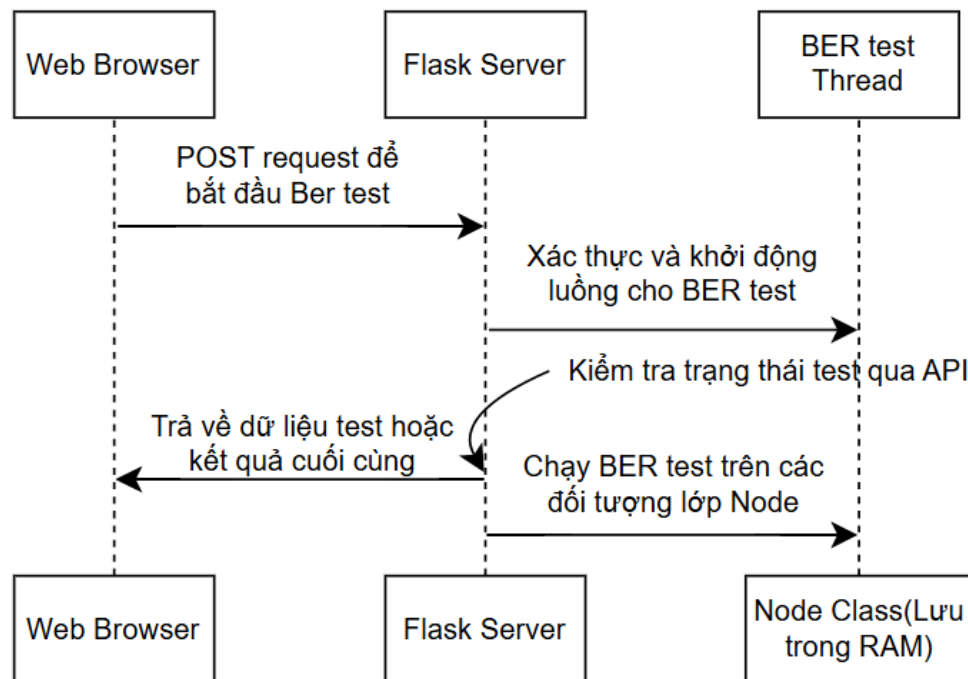
3.5.2.5 *Chức năng test BER*

Giao diện cấu hình kiểm thử BER: Cho phép người dùng chọn node phát (TX), node thu (RX), thiết lập các tham số kiểm thử như số bit, tần số, timeout, cấu hình SyncWords. Giao diện hỗ trợ kiểm tra trạng thái kết nối của từng node, cảnh báo khi cấu hình chưa hợp lệ.

Theo dõi tiến trình kiểm thử BER real-time: Khi kiểm thử bắt đầu, giao diện hiển thị tiến trình kiểm thử với progress bar, số bit đã kiểm tra, số lỗi bit, tỷ lệ lỗi (BER), RSSI, cùng các chỉ báo trạng thái. Các thông số này được cập nhật liên tục, giúp người dùng theo dõi chất lượng đường truyền không dây trong thời gian thực.

Dừng, xóa và reset kiểm thử dễ dàng: Giao diện hỗ trợ các nút dừng kiểm thử, xóa kết quả, reset node, đảm bảo quá trình kiểm thử luôn kiểm soát được và tránh các lỗi ngoài ý muốn.

Hiển thị kết quả cuối cùng và lưu lịch sử: Sau khi kiểm thử hoàn thành, kết quả được lưu lại và hiển thị rõ ràng, có thể xem lại hoặc xuất báo cáo phục vụ cho việc đánh giá hiệu năng hệ thống.



Hình 3.14. Sequence diagram mô tả quy trình thực hiện BER Test trong hệ thống

- Bước 1: Khởi tạo kiểm thử, Web Browser gửi POST request để bắt đầu Ber test đến Flask Server. Đây là yêu cầu từ giao diện web để bắt đầu quá trình kiểm thử BER
- Bước 2: Xử lý backend, Flask Server thực hiện "Xác thực và khởi động luồng cho BER test", tạo ra BER test Thread để xử lý kiểm thử trong nền
- Bước 3: Giám sát thời gian thực, có vòng lặp kiểm tra trạng thái test qua API giữa Web Browser và Flask Server. Điều này cho phép frontend theo dõi tiến trình test một cách real-time
- Bước 4: Trả kết quả, Flask Server trả về dữ liệu test hoặc kết quả cuối cùng cho Web Browser

3.6 Xây dựng phần cứng cho hệ thống Testbed

3.6.1 Vẽ sơ đồ nguyên lý và vẽ mạch in

3.6.1.1 Mục tiêu

Thiết kế và phát triển sơ đồ nguyên lý cùng bảng mạch in (PCB) chuyên dụng cho hệ thống Testbed mạng Zigbee, đảm bảo tính ổn định, khả năng mở rộng và hiệu suất tối ưu cho các hoạt động thử nghiệm và đánh giá mạng không dây.

Thiết kế sơ đồ nguyên lý: Xây dựng sơ đồ nguyên lý hoàn chỉnh thể hiện rõ ràng các kết nối giữa vi mạch RF (như EFR32), mạch phối hợp trở kháng, anten Inverted-F tùy chỉnh, và các thành phần hỗ trợ khác. Sơ đồ phải tuân thủ các tiêu chuẩn kỹ thuật và ký hiệu điện tử chuẩn hóa.

Tích hợp giao diện kiểm tra: Thiết kế các điểm test point và giao diện kết nối cần thiết để hỗ trợ việc đo lường, giám sát và kiểm tra hiệu suất mạng trong quá trình vận hành TestBeds.

3.6.1.2 Cách triển khai

Giai đoạn 1: Định nghĩa yêu cầu và thông số kỹ thuật. Giai đoạn này tập trung vào việc xác định rõ ràng các yêu cầu kỹ thuật và chức năng của bộ công cụ phát triển EFR32MG24, bao gồm lựa chọn vi mạch chính EFR32MG24B020F1024IM48 với ARM Cortex-M33 78MHz, định nghĩa các tính năng cốt lõi như giao diện USB-C, anten tích hợp, và các chân mở rộng cho việc bổ sung chức năng. Các chỉ tiêu kỹ thuật cụ thể bao gồm: kích thước mạch giới hạn dưới 10cm x 10cm để đảm bảo tính nhỏ gọn và dễ tích hợp, hệ thống nguồn cung cấp có khả năng cung cấp dòng tối thiểu 150mA để đáp ứng nhu cầu hoạt động của EFR32MG24 và các thiết bị ngoại vi, tích hợp mạch chuyển đổi USB sang UART trên mạch để hỗ trợ giao tiếp và gỡ lỗi trực tiếp qua USB mà không cần bộ chuyển đổi ngoài, các cổng hỗ trợ nạp mã lệnh (giao diện SWD/JTAG) và các chân kết nối chuẩn cho việc mở rộng với cảm biến ngoài, cùng với các thành phần tương tác người dùng bao gồm đèn LED (ít nhất 2 LED) để hiển thị trạng thái hoạt động và các nút bấm (tối thiểu 2 nút: khởi động lại và nút người dùng) để điều khiển và kiểm tra chức năng.

Giai đoạn 2: Thiết kế sơ đồ nguyên lý. Giai đoạn này bao gồm việc phát triển các khối chức năng chính: khối nguồn cung cấp từ USB-C 5V xuống 3.3V với nguồn ổn áp tuyến tính, khối antena với mạch phối hợp trở kháng dạng M tuân thủ hướng dẫn của Silicon Labs, khối vi điều khiển EFR32MG24B020F1024IM48 kết nối với bộ dao động thạch anh cùng với khối đầu vào/đầu ra tích hợp vi mạch chuyển đổi USB sang UART CH340 để hỗ trợ giao tiếp nối tiếp, các đèn LED để hiển thị trạng thái hoạt động và các nút bấm để điều khiển và kiểm tra chức năng của hệ thống.

Giai đoạn 3: Thiết kế bố trí mạch in. Thiết kế bố trí mạch in áp dụng cấu trúc hai lớp với mặt phẳng đất liên tục và hệ thống phân phối nguồn tối ưu, tuân thủ các nguyên tắc thiết kế tần số vô tuyến cho vi mạch EFR32MG24. Chiến lược phân bố linh kiện được tối ưu hóa với vi mạch EFR32MG24 đặt tại vị trí trung tâm mạch để tối thiểu hóa độ dài đường dẫn đến các linh kiện ngoại vi, vi mạch CH340 được bố trí gần cổng USB để giảm nhiễu trên đường truyền dữ liệu, các tụ điện khử gợn được đặt sát chân nguồn của từng vi mạch để đảm bảo ổn định nguồn cung cấp. Các đường nguồn được thiết kế rộng để giảm sụt áp và tổn hao, các đường dẫn tín hiệu tốc độ cao được định tuyến với độ dài tối thiểu và tránh các góc vuông, đồng thời triển khai kỹ thuật nối via để đảm bảo tính toàn vẹn của mặt phẳng đất.

3.6.2 Thiết kế antena cho mạch

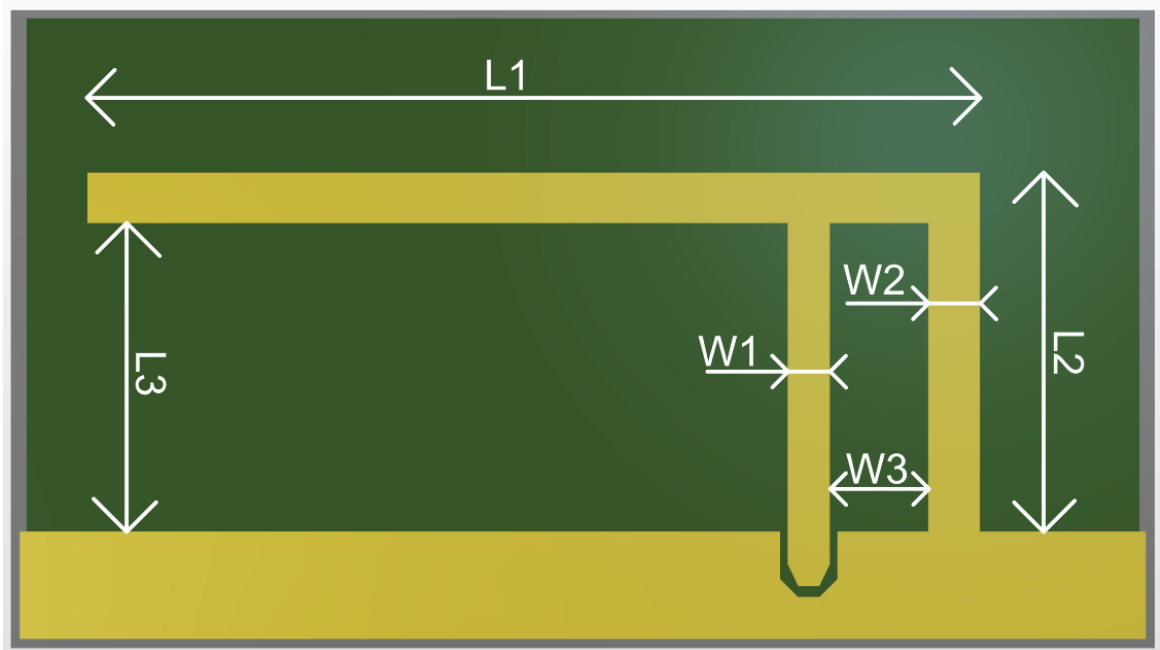
3.6.2.1 Mục tiêu

Antena được thiết kế với các chỉ tiêu sau:

- Trở kháng nhìn vào antenna là 50Ohm ở tần số 2.45GHz. Tần số hoạt động từ 2.4GHz đến 2.5Ghz
- Hiệu suất antenna trên 85/
- Hệ số phản xạ S_{11} dưới -10dBm trong toàn dải tần.
- Dạng đồ thị bức xạ là đa hướng

3.6.2.2 Cách triển khai

Dựa trên các yêu cầu trên em chọn loại anten là Inverted-F (IFA) được in trực tiếp trên mạch với cấu trúc phẳng để đảm bảo tính nhỏ gọn và hiệu suất bức xạ ổn định cho hệ thống. Kích thước antenna được mô tả như hình sau:



Hình 3.15. Cấu trúc và kích thước antenna

Bảng 3.5. Các thông số về kích thước của antenna

Thông số	Kích thước (mm)	Kích thước (mil)
L1	22.862	900.079
L2	9.190	361.811
L3	7.900	311.024
W1	1.076	42.362
W2	1.316	51.811
W3	2.528	99.528

Các kích thước của antenna phía trên là kích thước cuối cùng của antenna, sau khi đã tích hợp vào mạch, các thông số khác của antenna được mô tả ở phần kết quả.

3.6.3 Tính toán, mô phỏng giá trị các linh kiện cho mạch phối hợp trở kháng

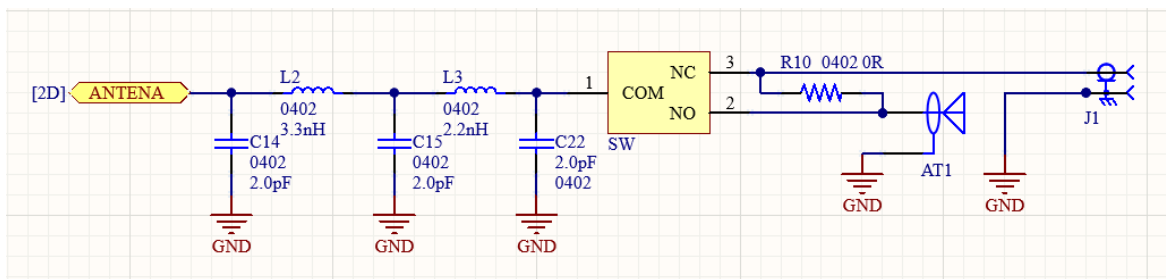
3.6.3.1 Mục tiêu

Mạch phối hợp trở kháng được thiết kế với các chỉ tiêu sau:

- Trở kháng của antenna là 50Ω ở tần số 2.45GHz.
- Trở kháng nhìn vào mạch phối hợp trở kháng là $17.7 - j*2.3(\Omega)$ ở tần số 2.45GHz.
- Hệ số phản xạ S_{11} dưới -10dBm trong toàn dải tần..

3.6.3.2 Cách triển khai

Dựa trên yêu cầu trên, cùng với tham khảo đề xuất của silabs, em đã chọn mạch phối hợp trở kháng với mạng hình M, có cấu trúc và giá trị linh kiện như hình sau:



Hình 3.16. Sơ đồ nguyên lý mạch phối hợp trở kháng

Hình 3.16. là sơ đồ nguyên lý của mạch phối hợp trở kháng, SW cho phép người dùng chọn loại antenna được sử dụng lên mạch, là antenna PCB tích hợp (AT_1) hoặc 1 antenna khác bên ngoài được kết nối bằng cổng U.FL (J_1). Cả 2 trường hợp trên, người dùng phải gỡ bỏ R_{10} . Ngoài ra, nếu người dùng muốn đo hệ số phản xạ của antenna thì có thể gỡ bỏ SW, và gắn vào điện trở R_{10} có giá trị $0(\Omega)$ để kết nối antenna với U.FL, sau đó kết nối U.FL này với VNA để đo hệ số phản xạ của antenna. Các giá trị linh kiện được tính toán bằng ADS, được mô phỏng bằng CST và được hiệu chỉnh bằng CST, kết quả của mạch phối hợp trở kháng được đưa ra ở phần sau.

Kết luận chương

Chương 3 đã trình bày chi tiết về cách xây dựng các thành phần chính của hệ thống. Từ đó có thể hiểu được các công cụ sử dụng, cách triển khai và cách các thành phần hoạt động cũng như tương tác với nhau trong hệ thống. Tiếp theo chương 4 sẽ là các kết quả đạt được và các kịch bản để kiểm thử hệ thống.

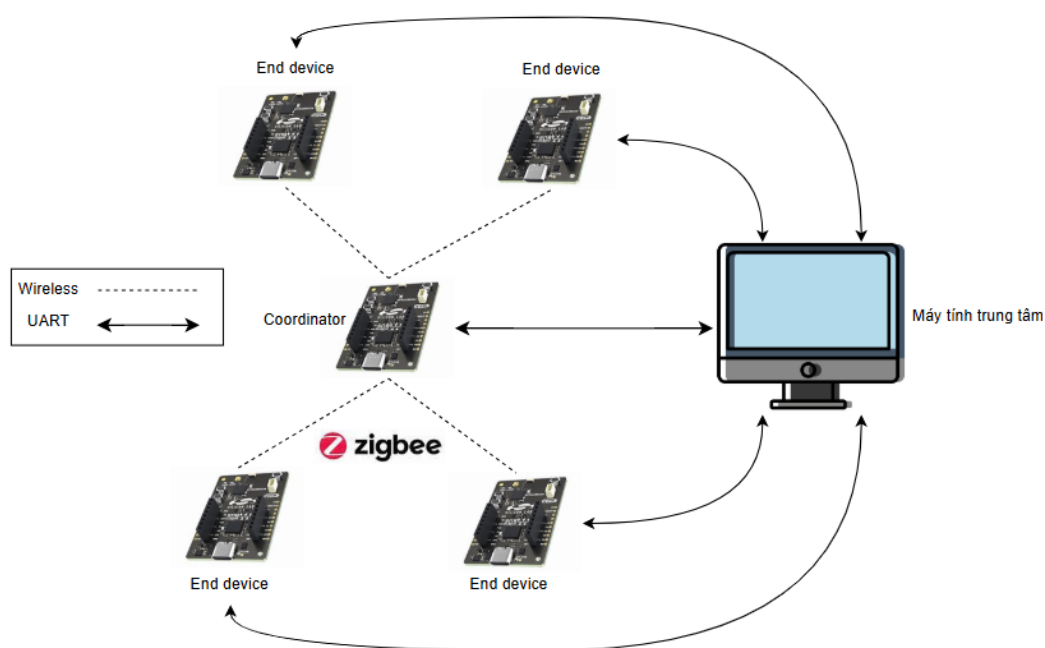
CHƯƠNG 4. KẾT QUẢ THỰC NGHIỆM

Mở đầu chương

Chương 4 trình bày về cách nhóm xây dựng các kịch bản ứng dụng để kiểm thử hệ thống và mô tả cách triển khai các kịch bản đó. Các kịch bản này gồm 2 nhóm là kịch bản kiểm thử phần cứng và kịch bản ứng dụng của người dùng. Sau đó là các kết quả đạt được.

4.1 Xây dựng kịch bản ứng dụng người dùng để kiểm thử hệ thống

4.1.1 Mô hình kịch bản ứng dụng

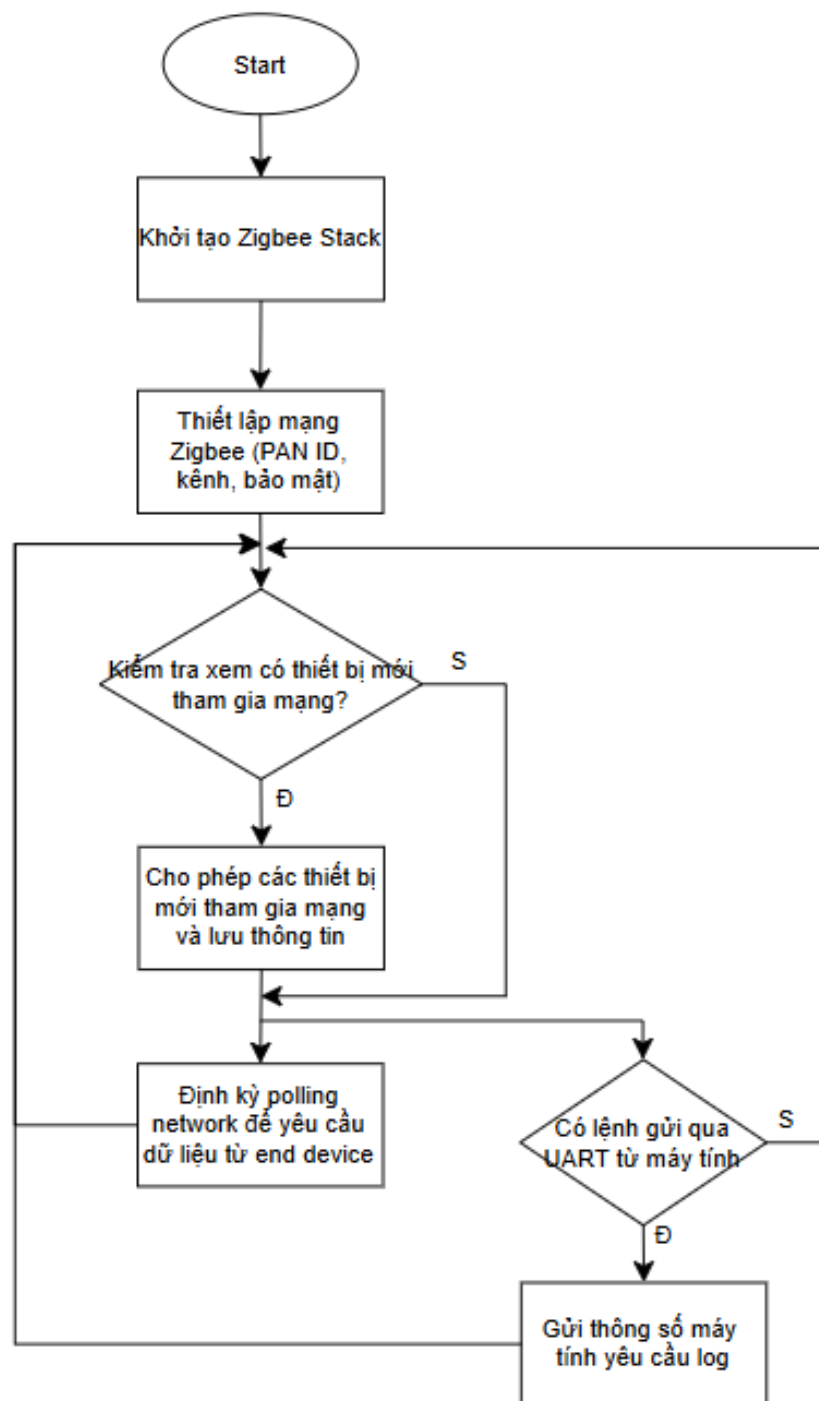


Hình 4.1. Sơ đồ mô hình kịch bản ứng dụng

Kịch bản ứng dụng người dùng được xây dựng là một mạng Zigbee hình sao gồm Coordinator ở trung tâm và 4 End device. Tất cả đều được kết nối UART đến máy tính trung tâm. Kết nối này là 2 chiều:

- Các thiết bị sẽ gửi dữ liệu các thông số lớp PHY và MAC về máy tính trung tâm để xử lý. Các thông số này gồm 2 loại: thông số tự động log và thông số log bởi người dùng khi cần
- Các thiết bị cũng sẽ nhận lệnh điều khiển từ máy tính trung tâm qua kết nối UART này

4.1.2 Xây dựng lưu đồ hoạt động cho các thành phần trong mạng



Hình 4.2. Lưu đồ hoạt động của Coordinator

Giai đoạn khởi tạo và thiết lập ban đầu: Coordinator bắt đầu quá trình hoạt động từ điểm start. Ngay sau đó, thiết bị tiến hành "Khởi tạo Zigbee Stack" - một bước quan trọng để thiết lập tất cả các lớp giao thức Zigbee cần thiết cho hoạt động mạng. Quá trình này bao gồm việc chuẩn bị các thành phần cốt lõi của stack protocol để Coordinator có thể đảm nhận vai trò trung tâm trong mạng.

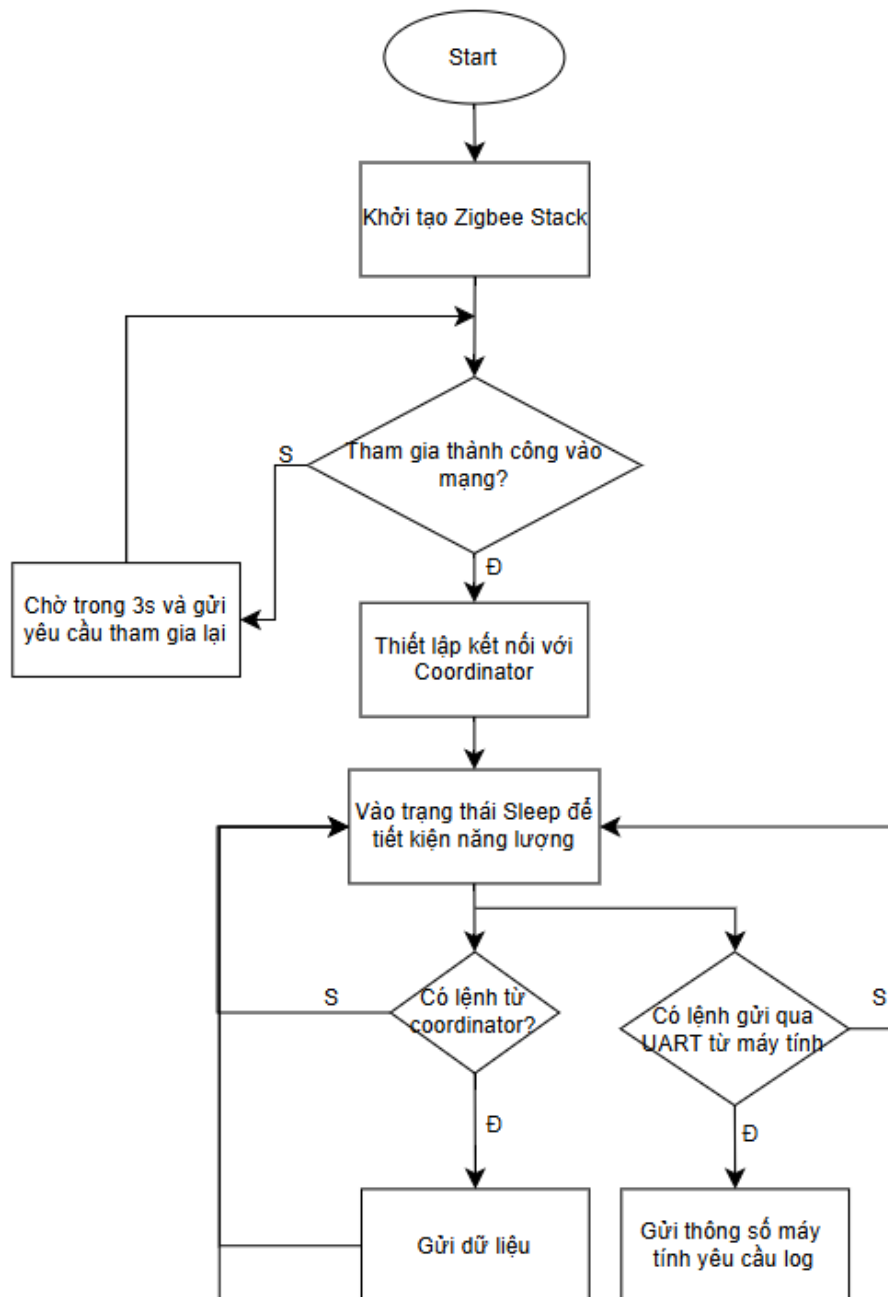
Quá trình thiết lập mạng Zigbee: Sau khi hoàn tất khởi tạo Stack, Coordinator

tiến hành "Thiết lập mạng Zigbee (PAN ID, kênh, bảo mật)". Đây là giai đoạn quan trọng nhất trong việc tạo lập mạng, bao gồm việc chọn PAN ID duy nhất, lựa chọn kênh hoạt động phù hợp và thiết lập các chính sách bảo mật cho toàn bộ mạng. Coordinator đóng vai trò là người khởi tạo và quản lý tất cả các thông số mạng này.

Cơ chế quản lý thiết bị tham gia mạng: Sau khi thiết lập mạng hoàn tất, Coordinator kiểm tra xem có thiết bị nào tham gia mạng?". Nếu không có thiết bị mới, hệ thống tạo thành một vòng lặp để tiếp tục giám sát. Ngược lại, Coordinator sẽ thực hiện "Cho phép các thiết bị mới tham gia mạng và lưu thông tin". Quá trình này đảm bảo Coordinator có thể tiếp nhận và quản lý các End device mới một cách liên tục.

Hoạt động polling và quản lý dữ liệu: Một khi đã có thiết bị tham gia mạng, Coordinator chuyển sang chế độ hoạt động chính với việc "Định kỳ polling network để yêu cầu dữ liệu từ end device". Đây là cơ chế quan trọng trong mạng Zigbee, cho phép Coordinator chủ động thu thập dữ liệu từ các End device, đặc biệt là những thiết bị hoạt động ở chế độ tiết kiệm năng lượng và không thể gửi dữ liệu một cách chủ động.

Giao tiếp với máy tính thông qua UART: Đồng thời với hoạt động polling, Coordinator cũng kiểm tra xem có lệnh gửi qua UART từ máy tính. Nếu không có lệnh, hệ thống quay lại chu trình polling network. Ngược lại, Coordinator sẽ thực hiện "Gửi thông số máy tính yêu cầu log", cho phép giao tiếp và truyền dữ liệu giữa mạng Zigbee và hệ thống máy tính trung tâm.



Hình 4.3. Lưu đồ hoạt động của End device

Giai đoạn khởi động và thiết lập ban đầu: Khi end device được khởi động, thiết bị sẽ bắt đầu từ trạng thái Start và ngay lập tức tiến hành khởi tạo Zigbee Stack. Đây là bước quan trọng để thiết lập tất cả các lớp giao thức cần thiết cho hoạt động Zigbee, bao gồm các lớp vật lý, MAC, mạng và ứng dụng.

Quá trình gia nhập mạng: Sau khi khởi tạo Zigbee Stack hoàn tất, end device sẽ kiểm tra xem có tham gia thành công vào mạng hay không. Nếu quá trình tham gia mạng chưa thành công, thiết bị sẽ chờ trong vòng 3 giây và sau đó gửi yêu cầu tham gia lại. Chu trình này sẽ được lặp lại liên tục cho đến khi thiết bị có thể tham gia thành công vào mạng Zigbee.

Thiết lập kết nối với Coordinator: Một khi end device đã tham gia thành công

vào mạng, bước tiếp theo là thiết lập kết nối trực tiếp với Coordinator. Đây là mối quan hệ cha-con quan trọng trong cấu trúc mạng Zigbee, cho phép End device giao tiếp và trao đổi dữ liệu thông qua Coordinator.

Chế độ tiết kiệm năng lượng: Sau khi hoàn tất quá trình thiết lập, End device sẽ chuyển vào trạng thái sleep để tiết kiệm năng lượng. Đây là trạng thái mặc định của thiết bị nhằm tối ưu hóa tuổi thọ pin, đặc biệt quan trọng đối với các thiết bị hoạt động bằng pin trong thời gian dài.

Cơ chế xử lý sự kiện song song: Từ trạng thái sleep, End device sẽ liên tục theo dõi hai loại sự kiện khác nhau một cách song song. Thứ nhất, thiết bị kiểm tra xem có lệnh nào từ Coordinator hay không. Nếu có lệnh từ coordinator, end device sẽ thực hiện việc gửi dữ liệu theo yêu cầu, sau đó quay lại trạng thái sleep. Đồng thời, End device cũng kiểm tra xem có lệnh nào được gửi qua UART từ máy tính hay không. Khi phát hiện có lệnh từ máy tính, thiết bị sẽ thực hiện việc gửi thông số mà máy tính yêu cầu log. Sau khi hoàn thành nhiệm vụ này, thiết bị cũng sẽ quay lại trạng thái sleep để tiết kiệm năng lượng.

4.2 Tính toán PER và PLR từ các event trong database

4.2.1 PER

Packet Error Rate (PER) là tỷ lệ giữa số lượng gói tin bị nhận sai (gói có lỗi) so với tổng số gói tin đã truyền đi trong một khoảng thời gian nhất định. Gói tin lỗi là gói có nội dung bị thay đổi, hỏng hóc do nhiễu, suy hao, hoặc các yếu tố vật lý khác trong quá trình truyền dẫn, khiến dữ liệu không còn chính xác khi đến đích. Công thức tính PER:

$$PER = \frac{\text{Số lượng gói tin nhận bị lỗi}}{\text{Tổng số gói tin nhận được}} \quad (4.1)$$

Nguyên nhân chính gây ra PER:

- Nhiễu điện từ, fading, hoặc giao thoa sóng trong môi trường không dây.
- Cáp mạng, cổng kết nối, hoặc thiết bị phần cứng bị lỗi.
- Lỗi định dạng gói tin do phần mềm hoặc cấu hình sai.

Ảnh hưởng: PER cao có thể gây mất dữ liệu, giảm hiệu suất truyền thông, làm gián đoạn các ứng dụng đòi hỏi thời gian thực như điều khiển tự động hóa, truyền video, hoặc âm thanh trực tuyến.

PER cao có thể gây mất dữ liệu, giảm hiệu suất truyền thông, làm gián đoạn các ứng dụng đòi hỏi thời gian thực như điều khiển tự động hóa, truyền video, hoặc âm thanh trực tuyến.

Biện pháp giảm PER:

- Sử dụng các giao thức sửa lỗi tiên tiến (ví dụ: Forward Error Correction – FEC).

- Nâng cấp hạ tầng mạng, sử dụng thiết bị chất lượng cao.
- Bảo trì, kiểm tra định kỳ hệ thống mạng

4.2.2 PLR

Packet Loss Rate (PLR) là tỷ lệ phần trăm số lượng gói tin bị mất (không đến được đích) so với tổng số gói tin đã gửi đi. Gói tin bị mất có thể do bị loại bỏ tại các thiết bị mạng (switch, router) hoặc bị thất lạc trong quá trình truyền dẫn. Công thức tính PLR:

$$PLR = \frac{\text{Số lượng gói tin bị mất}}{\text{Tổng số gói tin truyền đi}} \quad (4.2)$$

Nguyên nhân chính gây mất gói tin:

- Tắc nghẽn mạng (network congestion) khiến bộ đệm đầy và gói tin bị loại bỏ.
- Lỗi phần cứng, lỗi phần mềm, hoặc cấu hình sai.
- Nhiều sóng, tín hiệu yếu trong môi trường không dây.
- Lỗi bit trong quá trình truyền dẫn, khi đó gói tin bị loại bỏ hoàn toàn.

Ảnh hưởng:

- Giảm thông lượng thực tế.
- Tăng độ trễ do phải truyền lại gói tin (đặc biệt với TCP).
- Ảnh hưởng nghiêm trọng đến chất lượng các ứng dụng thời gian thực như VoIP, video call, game online, truyền hình trực tuyến (streaming).

4.2.3 Đề xuất cách tính PER và PLR từ các sự kiện

Khi chạy thử hệ thống trong thời gian đủ lâu, nhóm đã thu thập được các sự kiện như sau:

- TX PENDED MAC: Sự kiện này báo hiệu rằng một gói tin truyền đã được lên lịch ở tầng MAC nhưng chưa thực sự bắt đầu truyền ở tầng vật lý (PHY). Gói tin đang chờ điều kiện thích hợp để được chuyển tiếp xuống tầng PHY.
- TX PENDED PHY: Gói tin đã vượt qua tầng MAC và đang chờ được truyền ở tầng vật lý (PHY). Gói tin đã sẵn sàng ở hàng đợi truyền của PHY nhưng chưa bắt đầu truyền thực tế.
- TX ENDED: Quá trình truyền gói tin đã kết thúc, toàn bộ dữ liệu đã được phát ra khỏi radio. Sự kiện này xác nhận quá trình truyền đã hoàn tất, có thể kiểm tra trạng thái thành công hoặc thất bại.

- **RX ENDED:** Quá trình nhận một gói tin đã kết thúc. Khi radio nhận đủ dữ liệu của một gói tin, sự kiện này sẽ được kích hoạt và thông tin về gói tin sẽ sẵn sàng xử lý.
- **RX FILTERED:** Radio nhận được một gói tin nhưng gói tin này không vượt qua được bộ lọc địa chỉ hoặc các tiêu chí lọc khác nên bị loại bỏ và không chuyển lên tầng trên.
- **TX IDLED:** Radio đã chuyển sang trạng thái nhàn rỗi sau khi hoàn tất quá trình truyền, có thể do truyền thành công hoặc thất bại.
- **RX IDLED:** Radio chuyển sang trạng thái nhàn rỗi sau khi hoàn tất quá trình nhận, thường xảy ra khi không còn dữ liệu nào để nhận hoặc sau khi nhận xong một gói tin.
- **RX ACKING:** Radio đang chuẩn bị gửi ACK (Acknowledgement) để xác nhận đã nhận thành công một gói tin.
- **RX ACK SENT:** Radio đã gửi xong gói ACK xác nhận đã nhận thành công gói tin từ thiết bị gửi.
- **TX ACK WAITING:** Radio đang chờ nhận ACK từ phía thiết bị nhận sau khi đã truyền xong một gói tin yêu cầu xác nhận.
- **TX ACK RECEIVED:** Radio đã nhận được ACK từ phía thiết bị nhận, xác nhận rằng gói tin vừa truyền đã được nhận thành công.
- **TX CCA BUSY:** Radio thực hiện kiểm tra kênh (Clear Channel Assessment) trước khi truyền và phát hiện kênh đang bận, quá trình truyền sẽ bị trì hoãn hoặc hủy.
- **TX BLOCKED:** Quá trình truyền bị chặn lại do không đủ tài nguyên, radio đang bận hoặc do cấu hình phần mềm không cho phép truyền tại thời điểm đó.
- **RX CORRUPTED:** Radio nhận được một gói tin nhưng phát hiện dữ liệu bị lỗi (ví dụ: CRC không đúng hoặc nhiễu sóng làm hỏng dữ liệu), gói tin này sẽ bị loại bỏ.

Từ những sự kiện trên, nhóm đưa ra công thức tính PER và PLR(cho toàn bộ mạng) như sau:

- Công thức tính PER:

$$PER = \frac{N_{RX \text{ CORRUPTED}}}{N_{RX \text{ ENDED}} + N_{RX \text{ CORRUPTED}}} \quad (4.3)$$

Trong đó:

* $N_{RX \text{ CORRUPTED}}$: Số lượng event RX CORRUPTED (gói nhận bị lỗi)

* $N_{RX \text{ ENDED}}$: Số lượng event RX ENDED (gói nhận thành công)

- Công thức tính PLR:

$$PLR = \frac{N_{RX \text{ FILTERED}}}{N_{RX \text{ ENDED}} + N_{RX \text{ CORRUPTED}} + N_{RX \text{ FILTERED}}} \quad (4.4)$$

Trong đó:

* $N_{RX \text{ FILTERED}}$: Số lượng event RX FILTERED (gói bị loại bỏ bởi bộ lọc)

* $N_{RX \text{ ENDED}}$: Số lượng event RX ENDED (gói nhận thành công)

* $N_{RX \text{ CORRUPTED}}$: Số lượng event RX CORRUPTED (gói nhận bị lỗi)

$$PLR = 1 - \frac{N_{RX \text{ ENDED}}}{N_{TX \text{ ENDED}}} \quad (4.5)$$

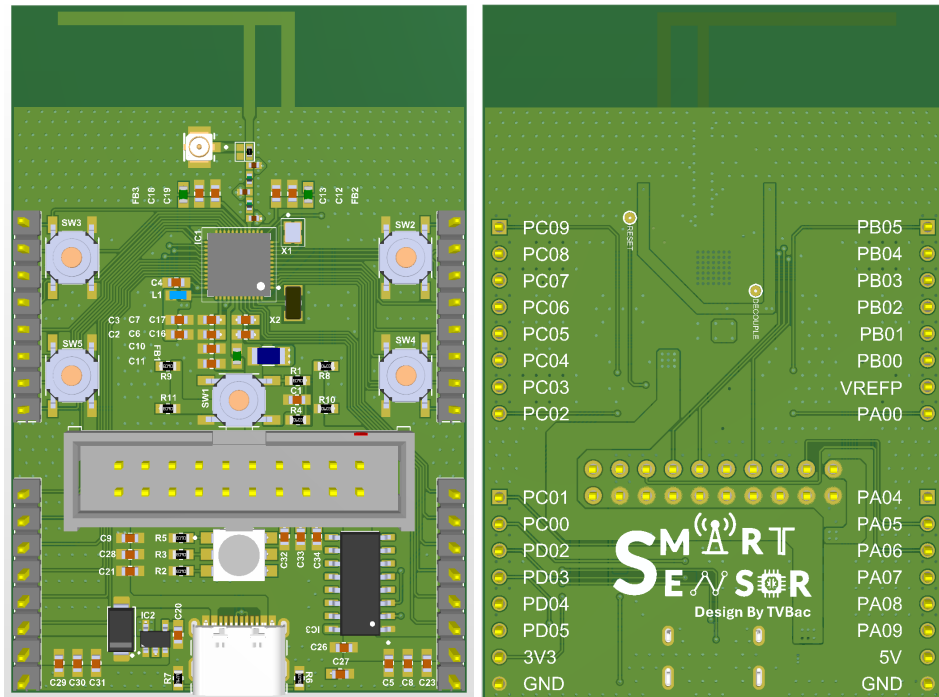
Trong đó:

* $N_{TX \text{ ENDED}}$: Số lượng gói đã truyền đi (TX ENDED)

* $N_{RX \text{ ENDED}}$: Số lượng gói nhận thành công (RX ENDED)

4.3 Kết quả phân cứng thu được

4.3.1 Mạch in PCB

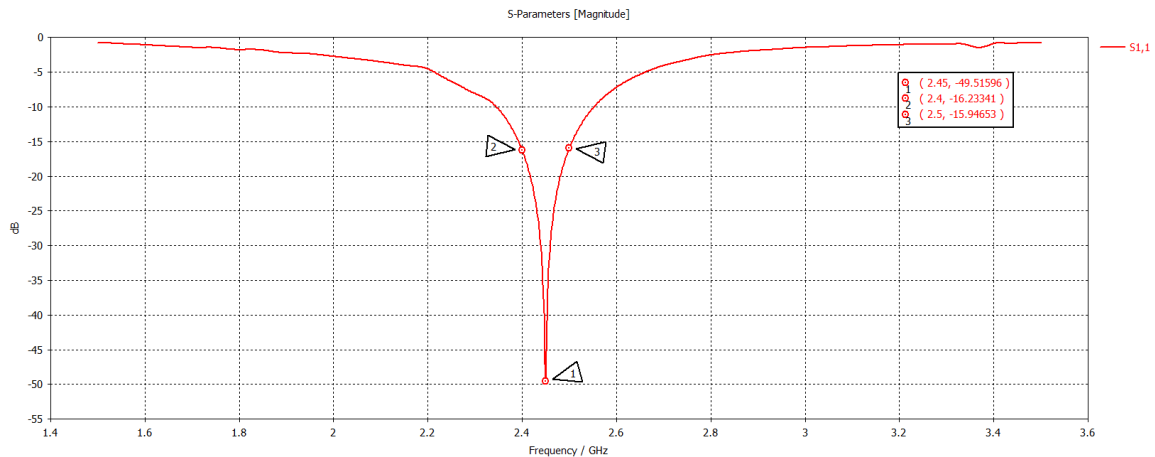


Hình 4.4. Hình ảnh 3D của mạch in

Mạch in được thiết kế với kích thước 44x66.5mm, có sẵn các cổng GPIO, được cấp nguồn qua cổng Type-C. Mạch có tích hợp bộ chuyển đổi USB to UART sử dụng IC CH340. Mạch sử dụng 4 nút bấm cùng với 1 đèn led RGB.

4.3.2 Kết quả thu được của Inverted-F antenna

4.3.2.1 Kết quả mô phỏng



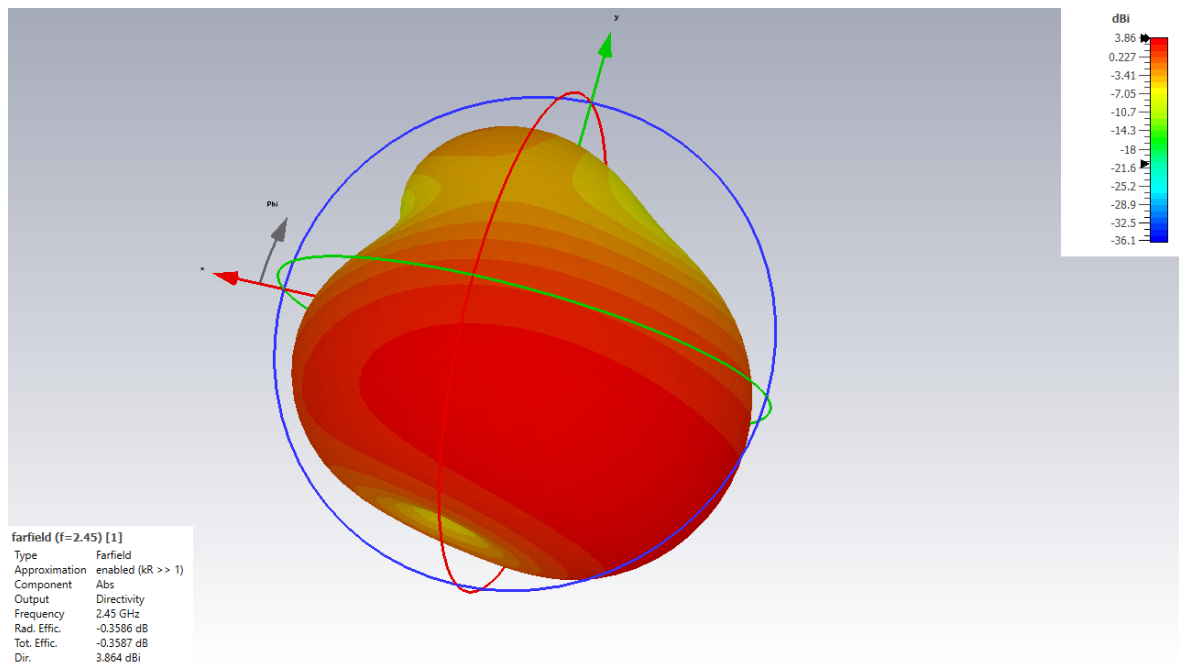
Hình 4.5. Đồ thị S_{11} của antenna

Dựa trên đồ thị ở hình 4.5., ta có thể thấy antenna đạt tần số cộng hưởng tối ưu tại 2.45GHz với giá trị $S_{11} = -49.55596\text{dB}$, cho thấy khả năng phối hợp trở kháng rất tốt tại tần số thiết kế. Giá trị S_{11} cực thấp này chứng tỏ gần như toàn bộ công suất đầu vào được truyền tải hiệu quả đến anten mà không bị phản xạ về nguồn.

Trong dải tần số quan tâm từ 2.4GHz đến 2.5GHz, anten duy trì hiệu suất tốt với:

- Tại 2.4GHz: $S_{11} = -16.23341\text{dB}$
- Tại 2.5GHz: $S_{11} = -15.94653\text{dB}$

Cả hai giá trị đều thỏa mãn yêu cầu thiết kế $S_{11} < -10\text{dB}$, đảm bảo ít nhất 90% công suất đầu vào được truyền tải hiệu quả. Băng thông -10dB của anten bao phủ hoàn toàn dải tần ISM 2.4GHz (2.4-2.485GHz) cần thiết cho các ứng dụng của kit. Kết quả mô phỏng chứng minh anten Inverted-F đã được thiết kế và tối ưu hóa thành công, đáp ứng đầy đủ các yêu cầu kỹ thuật đề ra về trở kháng 50Ω , hệ số phản xạ $S_{11} < -10\text{dB}$ trong toàn dải tần, và khả năng hoạt động ổn định từ 2.4GHz đến 2.5GHz. Giá trị S_{11} cực thấp tại tần số trung tâm 2.45GHz (-49.56dB) đảm bảo hiệu suất bức xạ tối ưu cho hệ thống testbeds.



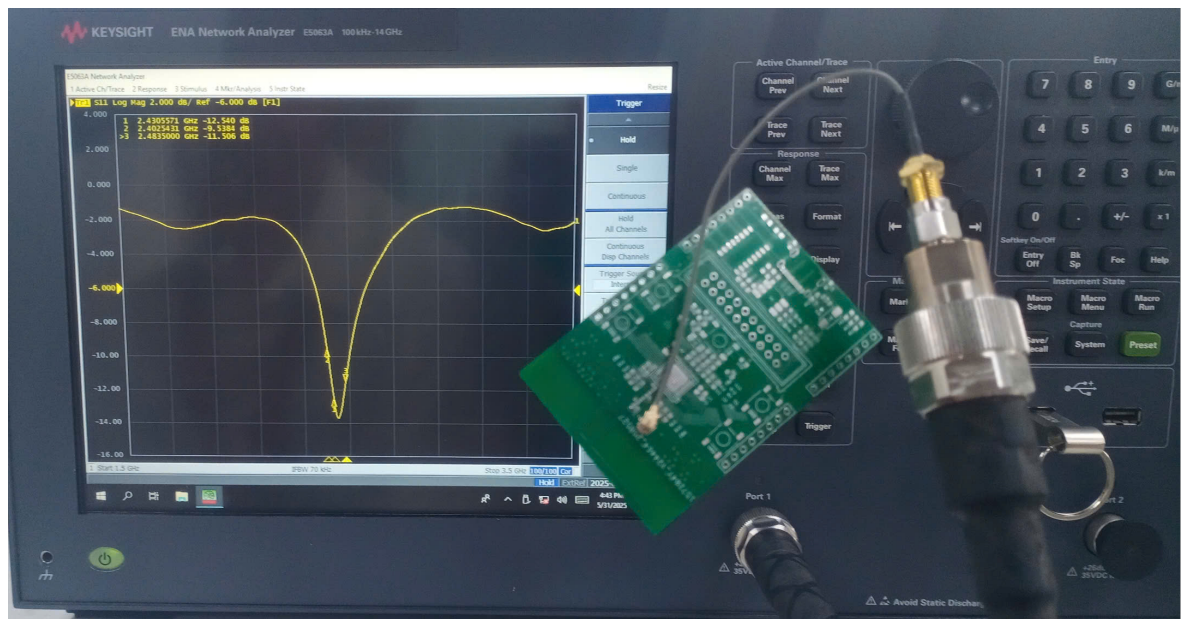
Hình 4.6. Kết quả farfield từ mô phỏng

Hình 4.6. cho thấy antenna thể hiện mẫu bức xạ gần như đa hướng (omnidirectional) với độ lợi tối đa 3.864 dBi tại tần số 2.45GHz. Hiệu suất bức xạ đạt -0.3586 dB (khoảng 92.07%) và tổng hiệu suất đạt -0.3587 dB (khoảng 92.07%), cho thấy anten hoạt động hiệu quả với tổn hao rất thấp.

Anten có phân cực thẳng đứng (vertical polarization) điển hình của anten Inverted-F. Mẫu bức xạ cho thấy có sự bất đối xứng nhẹ, điều này là bình thường đối với anten PIFA do ảnh hưởng của mặt phẳng đất hữu hạn và cấu trúc không đối xứng hoàn toàn.

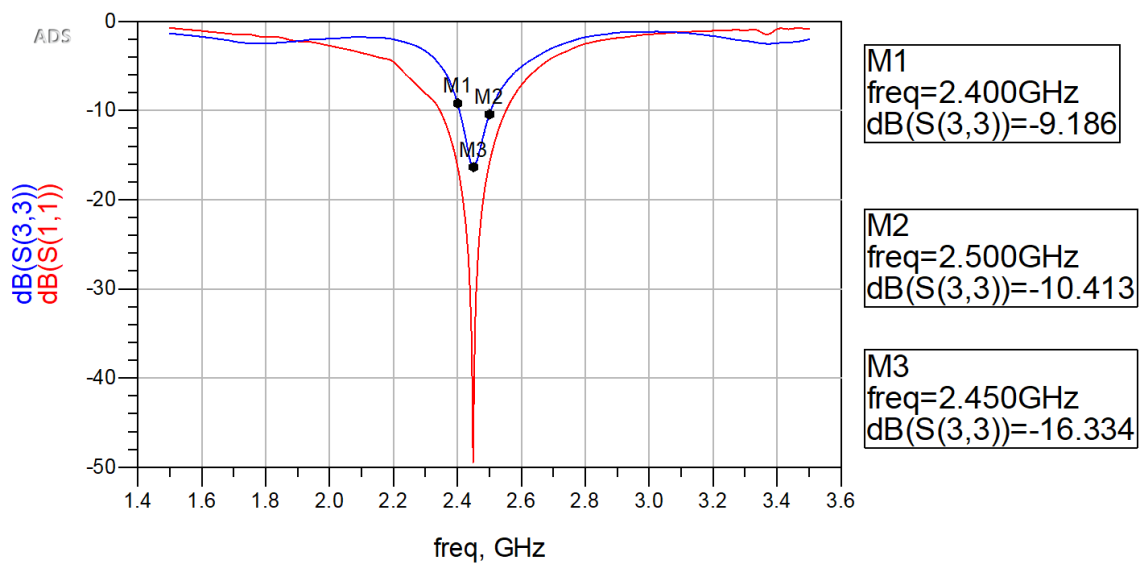
Độ lợi 3.864 dBi là giá trị hợp lý cho anten PIFA nhỏ gọn. Việc đạt được hiệu suất cao (>92%) đồng thời duy trì mẫu bức xạ đa hướng cho thấy thiết kế cân bằng tốt giữa kích thước, hiệu suất và đặc tính bức xạ.

4.3.2.2 Kết quả đo thực tế



Hình 4.7. Kết quả đo được từ máy VNA

Hình 4.7. là kết quả đo hệ số phản xạ S_{11} của antenna, được vẽ lại ở hình 4.6. bằng phần mềm ADS để tiện so sánh với kết quả mô phỏng.



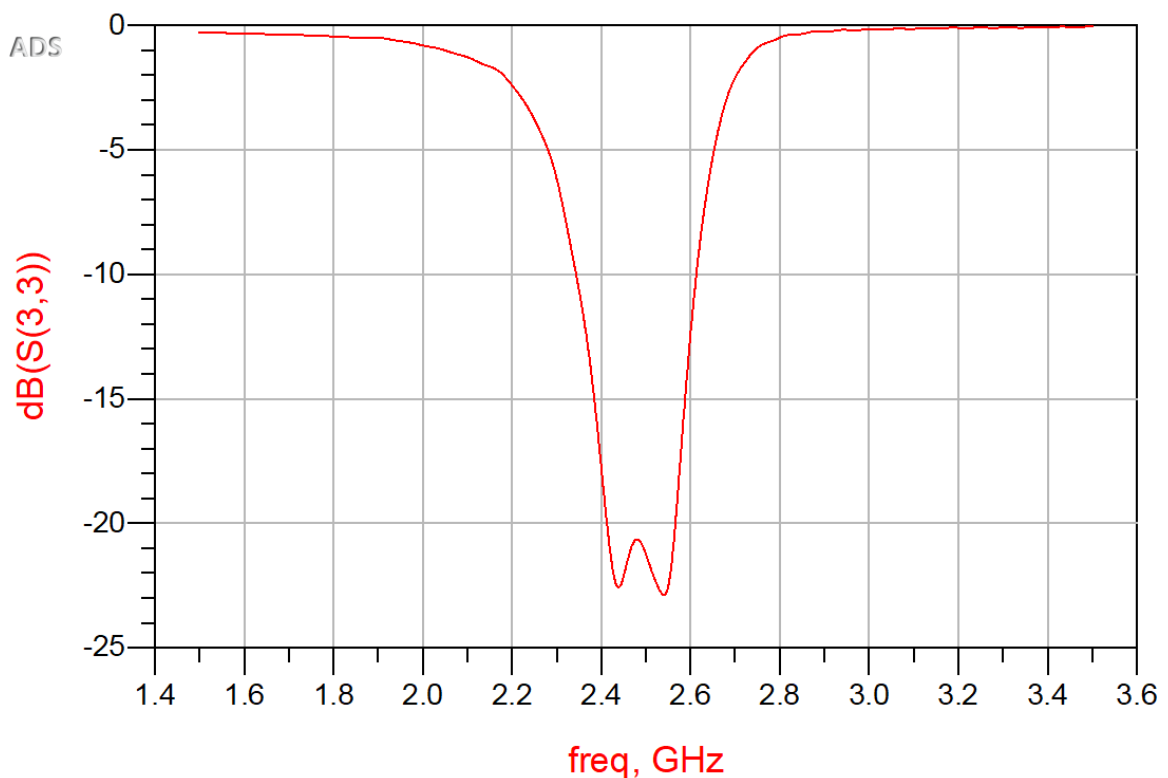
Hình 4.8. Kết quả đo được từ máy VNA (xanh) và từ mô phỏng trên CST (đỏ)

Kết quả đo thực tế (đường màu xanh) cho thấy anten có hiệu suất thấp hơn đáng kể so với mô phỏng. Tại các điểm đo quan trọng:

- Tại 2.4GHz: $S_{11} = -9.186\text{dB}$
- Tại 2.45GHz: $S_{11} = -16.334\text{dB}$
- Tại 2.5GHz: $S_{11} = -10.413\text{dB}$

Kết quả đo thực tế cho thấy hiệu suất anten thấp hơn đáng kể so với mô phỏng. Nguyên nhân chính gây ra sự khác biệt này bao gồm: sai số về thông số vật liệu PCB thực tế (hằng số điện môi và hệ số tổn hao khác với giá trị mô phỏng), ảnh hưởng của quá trình chế tạo PCB làm thay đổi kích thước anten, tác động của môi trường đo đạc (vật thể xung quanh và nguồn phản xạ), cùng với chất lượng kết nối và hàn tại các điểm nối. Ngoài ra, vị trí đặt U.FL để kết nối với máy đo cũng có sự khác biệt với vị trí đặt nguồn của mô phỏng do hạn chế về mặt vật lý. Những yếu tố này đều góp phần làm sai lệch giữa kết quả đo thực tế với kết quả mô phỏng lý tưởng.

4.3.3 Kết quả của mạch phối hợp trở kháng



Hình 4.9. Kết quả mô phỏng mạch phối hợp trở kháng cho antena

Hình 4.9. là kết quả mô phỏng hệ số phản xạ nhìn vào mạch phối hợp trở kháng, đồ thị này được lấy từ ADS với trở kháng nhìn về mạch lấy được từ mô phỏng trên CST với đặt cổng nguồn có trở kháng 50Ω . Sở dĩ cần sử dụng ADS bởi vì CST chỉ cho phép đặt các nguồn có trở kháng nguồn là số thực, trong khi trở kháng nhìn từ chân IC là $17.7 + j*2.3 (\Omega)$. Đồ thị cho thấy, công suất phát ra từ IC được truyền đi với ít nhất 90% công suất.

4.4 Thử nghiệm hệ thống trong các điều kiện khác nhau

4.4.1 Môi trường thường không có vật cản



Hình 4.10. Thử nghiệm hệ thống trong môi trường thường không có vật cản

Nhóm chọn môi trường để thử nghiệm hệ thống là phòng trọ kín, với nhiều sóng vô tuyến thấp, ngoài ra các thiết bị điện tử và các thiết bị phát sóng xung quanh cũng được tắt. Hệ thống triển khai gồm 1 Coordinator và 2 End device như hình 4.10., hoạt động từ 15:55:22 ngày 15/06/2025 đến 23:21:37 ngày 16/06/2025.

BẢNG 1: PER VÀ SUCCESS RATE:

Kit_ID	RX_CORRUPTED	RX_ENDED	Total_RX	PER_Percent	Success_Rate
842E14FFFEF64BFE	6	7402	7408	0.08099	99.91901
842E14FFFEF64B6B	1	712	713	0.14025	99.85975
84BA20FFFEAB417F	1	6737	6738	0.01484	99.98516

BẢNG 2: CCA COMPETITION (Cạnh tranh truy cập môi trường):

Kit_ID	TX_CCA_BUSY	TX_ENDED	Total_TX	CCA_Competition_Percent	CCA_Level
842E14FFFEF64BFE	135	6914	7049	1.95256	Good
842E14FFFEF64B6B	42	452	494	9.29204	Fair
84BA20FFFEAB417F	61	6797	6858	0.89745	Excellent

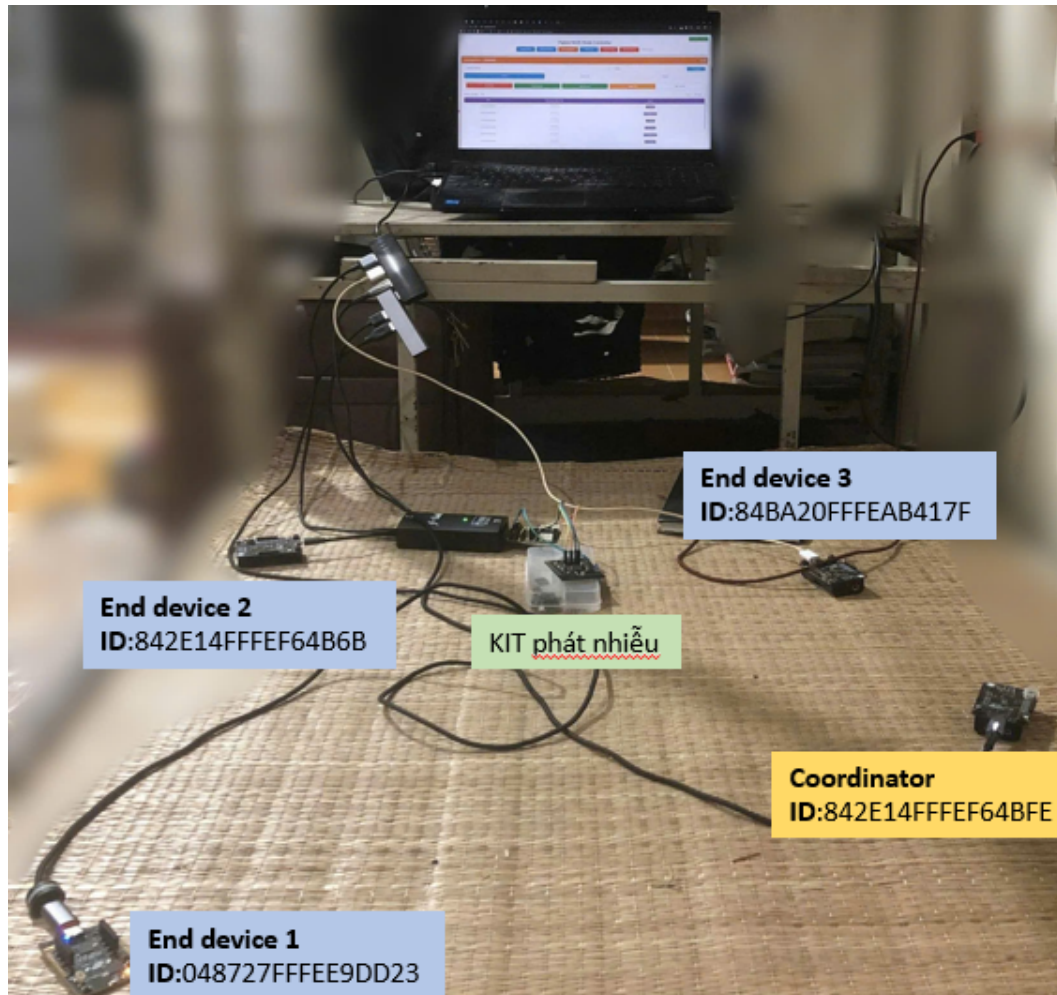
BẢNG 3: RF METRICS - CHỈ TỪ SUCCESSFUL PACKETS (RX_ENDED):

Kit_ID	RSSI_Min	RSSI_Avg	RSSI_Max	LQI_Min	LQI_Avg	LQI_Max
842E14FFFEF64BFE	-67	-23.98	-9	132	251.83	255
842E14FFFEF64B6B	-63	-26.19	-8	148	254.05	255
84BA20FFFEAB417F	-67	-22.48	-8	132	251.95	255

Hình 4.11. Kết quả đạt được trong môi trường ít nhiễu

Hình 4.11. là kết quả phân tích và tính toán PER cho từng thiết bị, ngoài ra còn có kết quả về RSSI và LQI và kết quả cạnh tranh truy cập môi trường trong môi trường ít nhiễu. Vì các node để gần nhau nên các chỉ số RSSI đều khá cao, chỉ số LQI trung bình cũng ở mức cao. Mức độ cạnh tranh truy cập môi trường ở mức thấp(số lượng event TX CCA BUSY khá nhỏ so với số lượng TX ENDED).

4.4.2 Môi trường có nhiễu tín hiệu



Hình 4.12. Thử nghiệm hệ thống trong môi trường có nhiễu

Hình 4.12. là hình ảnh bố trí thử nghiệm hệ thống trong môi trường được chủ động phát nhiễu bởi KIT sử dụng tính năng Tone Phase Noise[25] của RAIL. Hệ thống gồm 1 Coordinator và 3 End device hoạt động từ 21:45:03 ngày 20/06/2025 đến 08:26:12 ngày 21/06/2025.

Tone Phase Noise trong Railtest là tính năng phát sóng mang liên tục của chip EFR32 nhằm mục đích tạo ra một nguồn nhiễu RF chủ động trong môi trường thử nghiệm. Cụ thể trong trường hợp này, sau khi gửi lệnh reset để đưa thiết bị về trạng thái mặc định, em sử dụng lệnh settxstream 1 3 để kích hoạt chế độ phát TonePhaseNoise. Lúc này, thiết bị sẽ phát ra một tín hiệu sóng mang thuần tại tần số đã chọn, không

có dữ liệu hay điều chế nào, nhưng tín hiệu này vẫn chiếm dụng phổ tần số và tạo ra nhiễu nền đáng kể cho các thiết bị khác hoạt động cùng kênh.

Việc tạo ra nhiễu bằng Tone Phase Noise rất hữu ích trong các kịch bản kiểm thử khả năng chống nhiễu, đánh giá độ nhạy, hoặc kiểm tra hiệu năng truyền nhận của các thiết bị khác trong cùng môi trường RF. Khi một thiết bị phát sóng mang liên tục ở chế độ này, nó sẽ làm tăng mức nhiễu nền trên phổ tần số, khiến các thiết bị khác khó truyền nhận dữ liệu hoặc có thể làm tăng tỷ lệ lỗi gói tin. Ngoài ra, việc sử dụng Tone Phase Noise để tạo nhiễu còn giúp kiểm tra khả năng phát hiện và xử lý kênh bận của các thiết bị khác, cũng như đánh giá hiệu quả của các thuật toán tránh nhiễu hoặc chuyển kênh tự động.

 BẢNG 1: PER VÀ SUCCESS RATE:

Kit_ID	RX_CORRUPTED	RX_ENDED	Total_RX	PER_Percent	Success_Rate
842E14FFFEF64BFE	1	47	48	2.08333	97.91667
842E14FFFEF64B6B	6	2528	2534	0.23678	99.76322
048727FFFE9DD23	0	2550	2550	0.00000	100.00000
84BA20FFFEAB417F	4	2584	2588	0.15456	99.84544

 BẢNG 2: CCA COMPETITION (Cạnh tranh truy cập môi trường):

Kit_ID	TX_CCA_BUSY	TX_ENDED	Total_TX	CCA_Competition_Percent	CCA_Level
842E14FFFEF64BFE	876	2448	3324	35.78431	Critical
842E14FFFEF64B6B	57	23	80	247.82609	Critical
048727FFFE9DD23	10	23	33	43.47826	Critical
84BA20FFFEAB417F	39	7	46	557.14286	Critical

 BẢNG 3: RF METRICS - CHỈ TỪ SUCCESSFUL PACKETS (RX_ENDED):

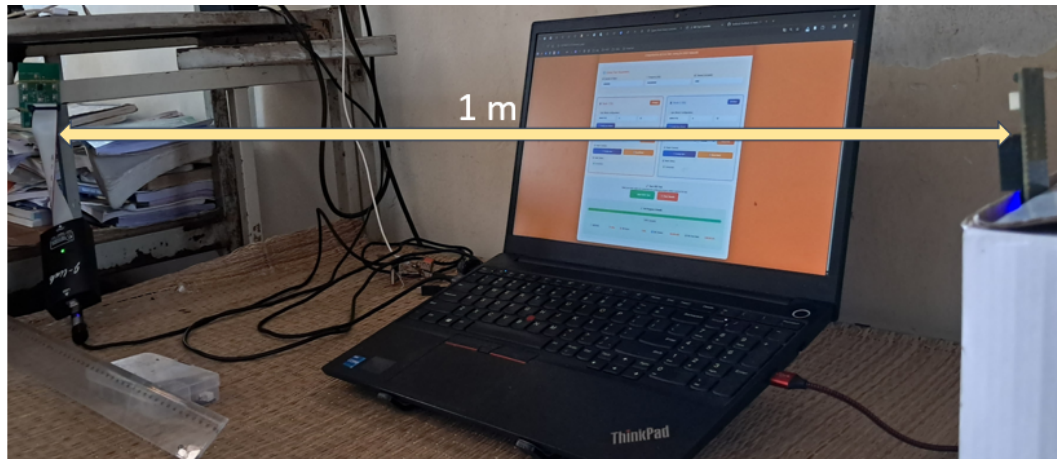
Kit_ID	RSSI_Min	RSSI_Avg	RSSI_Max	LQI_Min	LQI_Avg	LQI_Max
842E14FFFEF64BFE	-40	-33.43	-24	240	253.64	255
842E14FFFEF64B6B	-38	-29.07	-23	248	254.99	255
048727FFFE9DD23	-55	-32.09	-27	180	254.96	255
84BA20FFFEAB417F	-54	-22.16	-13	184	254.96	255

Hình 4.13. Kết quả đạt được trong môi trường chủ động tạo nhiễu

Hình 4.13. là kết quả phân tích được khi thử nghiệm hệ thống trong môi trường phát nhiễu chủ động. Các thông số PER, RSSI và LQI không có quá nhiều khác biệt so với thử nghiệm trước. Điểm đáng chú ý ở đây là mức cạnh tranh truy cập môi trường đã tăng lên rất cao ở mỗi thiết bị.

4.5 Kết quả test Bit Error Rate(BER)

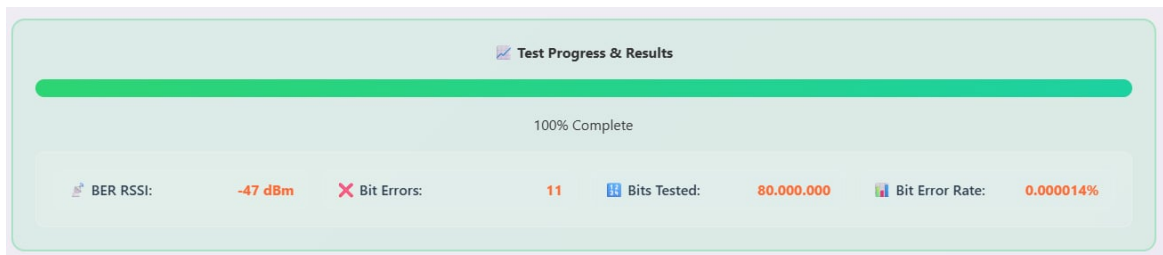
4.5.1 Bố trí hệ thống



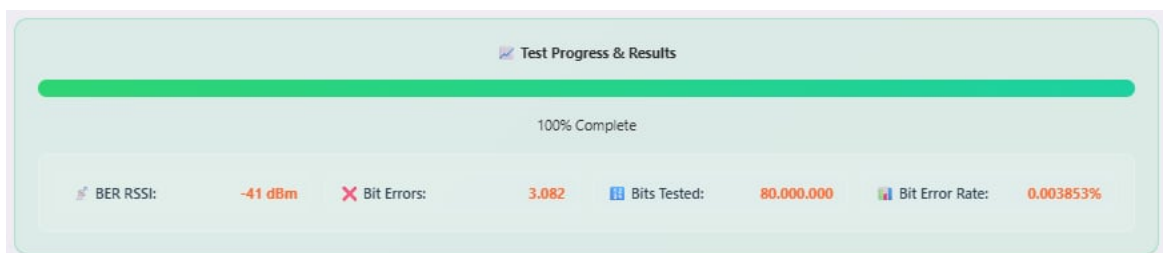
Hình 4.14. Sơ đồ thiết lập để đánh giá BER

Hình A.4 mô tả cách thiết lập để đo thông số BER. Hai kit được đặt cố định và cách nhau 1m, một kit truyền liên tục và một kit nhận, số bit test là 80.000.000.

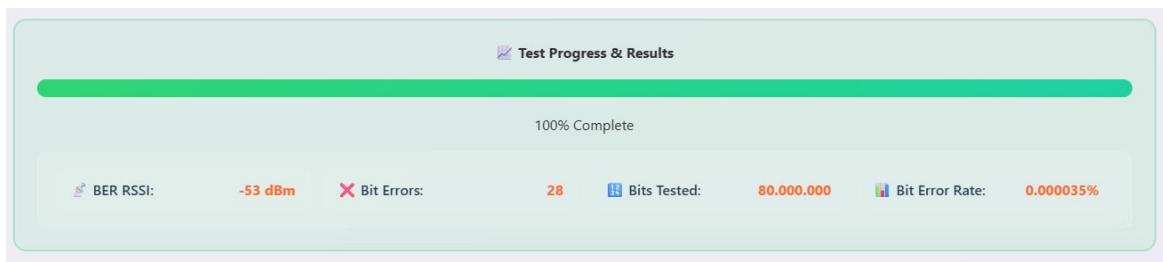
4.5.2 Kết quả BER thu được



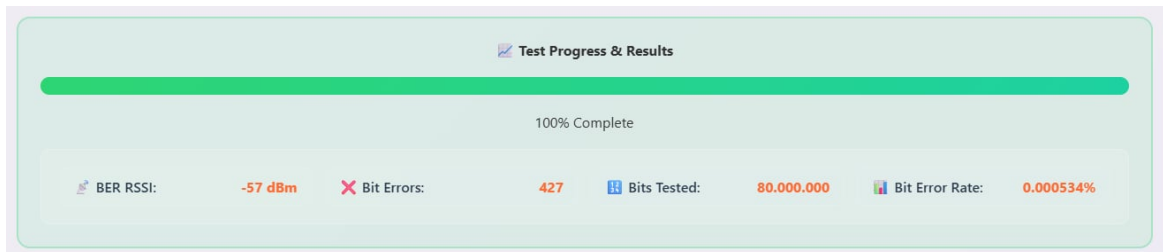
Hình 4.15. Test BER trên 2 kit EFR32MG24



Hình 4.16. Test BER trên 2 kit tự thiết kế



Hình 4.17. Test BER với kit gửi là kit tự thiết kế, kit nhận là EFR32MG24



Hình 4.18. Test BER với kit gửi là EFR32MG24, kit nhận là kit tự thiết kế

Từ các kết quả trên ta có thể thấy tỉ lệ lỗi bit khi test trên 2 kit EFR32MG24 của Silicon Lab rất thấp (0.00014 % theo hình 4.15.). Khi test trên 2 kit tự thiết kế, kết quả BER cao hơn nhưng vẫn ở mức đạt hiệu suất cao (0.003853 % theo hình 4.16.). Khi đặt kit tự thiết kế là kit truyền, EFR32MG24 là kit nhận và ngược lại thì kết quả thu được lần lượt ở hình 4.17. và 4.18..

Kết luận chương

Chương 4 đã trình bày về 2 nhóm kịch bản chính mà nhóm dùng để kiểm thử hệ thống: kịch bản kiểm thử phần cứng (BER test) và kịch bản mô tả ứng dụng người dùng (mạng Zigbee với ứng dụng bật tắt đèn từ xa) và các kết quả đạt được.

CHƯƠNG 5. KẾT LUẬN VÀ HƯỚNG PHÁT TRIỂN

5.1 Kết luận

Qua quá trình thực hiện, đồ án đã thành công trong việc thiết kế và xây dựng một hệ thống testbed hoàn chỉnh và đa chức năng cho chuẩn mạng không dây IEEE 802.15.4 và chồng giao thức Zigbee. Hệ thống không chỉ đáp ứng đầy đủ các mục tiêu ban đầu đề ra mà còn mở rộng thêm nhiều tính năng hữu ích, chứng tỏ tính khả thi và tiềm năng ứng dụng cao trong thực tế.

Các kết quả chính đã đạt được:

- Xây dựng thành công hệ thống điều khiển và giám sát tập trung: Đồ án đã phát triển một giao diện web trực quan, hoạt động như một trung tâm điều khiển. Từ giao diện này, người dùng có thể quản lý đồng thời nhiều node mạng vật lý, thực hiện các thao tác từ xa như khởi tạo, thêm/xóa node, và giám sát trạng thái kết nối theo thời gian thực.
- Hoàn thiện chức năng phân tích gói tin: Hệ thống có khả năng thu thập, giải mã và phân tích chi tiết từng gói tin theo cấu trúc của chồng giao thức. Sử dụng các lớp đối tượng như ZigbeePacket, IEEE802154, và ZigbeeNetwork, đồ án đã tái hiện thành công chức năng của một trình phân tích mạng. Người dùng có thể xem xét từng trường dữ liệu trong các lớp PHY/MAC (IEEE 802.15.4), Network (Zigbee NWK), và Security, cung cấp một công cụ mạnh mẽ cho việc gỡ lỗi, nghiên cứu và học tập.
- Tích hợp bộ công cụ kiểm thử hiệu năng mạng: Đồ án đã triển khai thành công hai bài kiểm thử quan trọng:
 - Kiểm thử Tỷ lệ Lỗi Bit (Bit Error Rate - BER): Cho phép đánh giá chất lượng kênh truyền vật lý giữa hai node bất kỳ bằng cách truyền một luồng dữ liệu lớn và đếm số bit lỗi. Giao diện web cung cấp khả năng cấu hình, khởi động và theo dõi tiến trình kiểm thử một cách trực quan.
 - Quét năng lượng kênh (Energy Scan): Cung cấp công cụ để đo lường mức năng lượng trên các kênh tần số của chuẩn IEEE 802.15.4, giúp người dùng lựa chọn kênh hoạt động tốt nhất, tránh nhiễu.
- Kiến trúc hệ thống linh hoạt và bền vững: Việc lựa chọn kiến trúc Client-Server với backend là Flask và frontend là giao diện web đã chứng tỏ hiệu quả. Hệ thống hoạt động ổn định, đáp ứng nhanh và dễ dàng mở rộng. Đặc biệt, cơ chế quản lý tài nguyên và tự động dọn dẹp khi dừng server hoặc đóng port đảm bảo hệ thống không gây ra lỗi treo cổng COM, một vấn đề thường gặp trong các hệ thống giao tiếp phần cứng.

- Xây dựng cơ sở dữ liệu cho phân tích dài hạn: Tất cả các gói tin và kết quả quét năng lượng đều được lưu trữ vào cơ sở dữ liệu SQLite. Điều này không chỉ giúp truy xuất dữ liệu tức thời mà còn tạo ra một kho dữ liệu quý giá cho việc phân tích offline, thống kê xu hướng và các nghiên cứu chuyên sâu sau này.

5.2 Hướng phát triển

Dựa trên nền tảng mà đồ án đã xây dựng, có rất nhiều tiềm năng để mở rộng và phát triển hệ thống thành một công cụ nghiên cứu, phát triển toàn diện và chuyên nghiệp hơn. Các hướng phát triển trong tương lai có thể được trình bày thành các đoạn văn chi tiết như sau:

- Một trong những hướng phát triển tự nhiên và giá trị nhất là nâng cao khả năng phân tích và trực quan hóa dữ liệu. Hiện tại, hệ thống đã giải mã thành công các lớp giao thức đến lớp mạng và bảo mật, tuy nhiên, phần dữ liệu ứng dụng vẫn đang được hiển thị dưới dạng chuỗi byte thô. Một bước tiến quan trọng là xây dựng một bộ giải mã cho Thư viện cụm chức năng của Zigbee (Zigbee Cluster Library - ZCL). Bằng cách phân tích các trường Cluster ID, Attribute ID, và Command ID, hệ thống có thể diễn giải được ý nghĩa thực sự của payload, ví dụ như "lệnh bật đèn" hoặc "giá trị nhiệt độ là 25°C". Việc này sẽ nâng tầm hệ thống từ một trình phân tích mạng thành một trình gỡ lỗi ở cấp độ ứng dụng, cung cấp cái nhìn sâu sắc và trực quan hơn cho người dùng.
- Bên cạnh việc phân tích sâu hơn, việc trực quan hóa cấu trúc mạng cũng là một hướng đi đầy hứa hẹn. Thay vì chỉ hiển thị danh sách các node, hệ thống có thể được nâng cấp để vẽ một bản đồ mạng động trên giao diện web. Bằng cách tận dụng thông tin địa chỉ nguồn/đích và các chỉ số chất lượng liên kết từ các gói tin đã thu thập, hệ thống có thể sử dụng các thư viện JavaScript như D3.js hoặc Cytoscape.js để hiển thị các node dưới dạng các đỉnh và các liên kết giữa chúng dưới dạng các cạnh. Độ dày hoặc màu sắc của các cạnh có thể biểu diễn cho chất lượng kết nối, giúp người dùng nhanh chóng xác định các điểm yếu trong mạng. Hơn nữa, việc tích hợp các biểu đồ động để theo dõi các chỉ số RSSI, LQI, và thông lượng mạng theo thời gian thực sẽ giúp người dùng dễ dàng nhận ra các biến động hoặc sự cố, chẳng hạn như nhiễu định kỳ hoặc kết nối chập chờn.
- Để nâng cao hiệu quả và tính chuyên nghiệp của quá trình kiểm thử, việc tự động hóa là một hướng phát triển không thể thiếu. Hệ thống có thể được bổ sung một bộ tạo kịch bản, cho phép người dùng định nghĩa các chuỗi thao tác kiểm thử một cách tự động. Ví dụ, người dùng có thể tạo một kịch bản để một node tạo mạng, các node khác lần lượt gia nhập, sau đó một node bắt đầu gửi dữ liệu để kiểm thử BER, và cuối cùng hệ thống tự động xuất báo cáo. Việc tự động hóa này không chỉ tiết kiệm thời gian mà còn đảm bảo tính nhất quán và lặp lại cho các bài kiểm

thử phức tạp. Song song đó, việc chuẩn hóa các route Flask thành một bộ API công khai (Public RESTful API) sẽ cho phép các hệ thống bên ngoài, như các quy trình CI/CD (Jenkins, GitLab CI), có thể điều khiển testbed, thực hiện kiểm thử phần cứng trong vòng mỗi khi có một phiên bản firmware mới, tự động hóa hoàn toàn quy trình kiểm thử hồi quy.

- Nhằm tăng tính linh hoạt và phạm vi ứng dụng của testbed, việc mở rộng hỗ trợ cho các giao thức và phần cứng khác là vô cùng cần thiết. Nền tảng IEEE 802.15.4 còn được sử dụng cho các giao thức IoT quan trọng khác như Thread và 6LoWPAN. Do đó, phương thức Decode() có thể được mở rộng để nhận dạng và phân tích các cấu trúc gói tin của những giao thức này, biến testbed thành một công cụ đa năng cho nhiều hệ sinh thái IoT. Đồng thời, để giảm sự phụ thuộc vào một loại phần cứng cụ thể, hệ thống nên xây dựng một lớp trừu tượng hóa phần cứng. Bằng cách này, logic điều khiển chính của testbed sẽ không thay đổi khi tích hợp các module, vì điều khiển từ các nhà sản xuất khác nhau, chỉ cần viết một lớp triển khai riêng cho phần cứng mới.
- Cuối cùng, hệ thống có thể được phát triển để phục vụ các hướng nghiên cứu nâng cao về an ninh mạng và các vấn đề cùng tồn tại của mạng không dây. Testbed có thể được bổ sung các module cho phép tạo và gửi các gói tin không hợp lệ để mô phỏng các loại tấn công phổ biến trên mạng Zigbee như tấn công phát lại hoặc tấn công từ chối dịch vụ (DoS). Ở chiều ngược lại, backend có thể phát triển các thuật toán để phân tích luồng gói tin và phát hiện các dấu hiệu bất thường, cảnh báo cho người dùng về một cuộc tấn công tiềm tàng. Ngoài ra, một hướng nghiên cứu rất thực tiễn là phân tích khả năng chung sống (co-existence) với các mạng Wi-Fi và Bluetooth. Bằng cách kết hợp dữ liệu từ chức năng quét năng lượng kênh với kết quả kiểm thử BER và thống kê mất gói, hệ thống có thể xây dựng một dashboard chuyên dụng để trực quan hóa mối tương quan giữa mức độ nhiễu và sự suy giảm hiệu năng của mạng Zigbee, một vấn đề rất phổ biến trong thực tế.

TÀI LIỆU THAM KHẢO

- [1] M. T. Inc. (2025) 14.3.1 introduction – ieee 802.15.4-2006 frame format. Accessed on 2025-06-20. [Online]. Available: <https://onlinedocs.microchip.com/oxy/GUID-4B3D771E-5647-4191-AD45-C897B0D23071-en-US-3/GUID-BB02A661-7BDD-4B28-9F66-E9E8292B9423.html>
- [2] Stats and Insights. (2025) Iot wireless tech: Wi-fi, bluetooth, zigbee – guide & comparison. Accessed on 2025-06-20. [Online]. Available: <https://statsandinsights.com/2025/01/13/iot-wireless-tech-wifi-bluetooth-zigbee-guide-comparison/>
- [3] S. Labs. (2025) Efr32 main - rail api documentation. Accessed on 2025-06-20. [Online]. Available: <https://docs.silabs.com/rail/latest/rail-api/efr32-main>
- [4] Coursera, “What is the internet of things (iot)? with examples,” Coursera, March 2025. [Online]. Available: <https://www.coursera.org/articles/internet-of-things>
- [5] C. Sharma and N. K. Gondhi, “Communication protocol stack for constrained iot systems,” in *2018 3rd International Conference On Internet of Things: Smart Innovation and Usages (IoT-SIU)*, 2018, pp. 1–6.
- [6] “Ieee standard for low-rate wireless networks,” *IEEE Std 802.15.4-2020 (Revision of IEEE Std 802.15.4-2015)*, pp. 1–800, 2020.
- [7] “Ieee draft standard for low-rate wireless networks amendment: Higher data rate extension to ieee 802.15.4 smart utility network (sun) frequency shift keying (fsk) physical layer (phy),” *IEEE P802.15.4aa/D9, September 2021*, pp. 1–19, 2021.
- [8] GeeksforGeeks. (2025) Direct sequence spread spectrum in wireless networks. Accessed on 2025-06-20. [Online]. Available: <https://www.geeksforgeeks.org/ethical-hacking/direct-sequence-spread-spectrum-in-wireless-networks/>
- [9] user: Marcus Müller *et al.* (2017) Envelope behavior difference between qpsk, oqpsk and $\pi/4$ -qpsk. Accessed on 2025-06-20. [Online]. Available: <https://dsp.stackexchange.com/questions/41130/envelope-behavior-difference-between-qpsk-oqpsk-and-pi-4-qpsk>
- [10] “Ieee standard for information technology– local and metropolitan area networks– specific requirements– part 15.4: Wireless medium access control (mac) and physical layer (phy) specifications for low rate wireless personal area networks (wpans),” *IEEE Std 802.15.4-2006 (Revision of IEEE Std 802.15.4-2003)*, pp. 1–320, 2006.

- [11] S. S. E. Community. (2024) How is the mic (message integrity code) generated in wpa2? Accessed on 2025-06-20. [Online]. Available: <https://security.stackexchange.com/questions/244316/how-is-the-mic-message-integrity-code-generated-in-wpa2>
- [12] A. Charlton. (2024) What is zigbee, and why it's a must-have for your smart home. TechRadar. Accessed on 2025-06-20. [Online]. Available: <https://www.techradar.com/news/what-is-zigbee-and-why-its-a-must-have-for-your-smart-home>
- [13] R. W. World. (2025) Zigbee aodv protocol basics. Accessed on 2025-06-20. [Online]. Available: <https://www.rfwireless-world.com/tutorials/zigbee-aodv-protocol-basics>
- [14] T. Kim, S. H. Kim, J. Yang, S.-e. Yoo, and D. Kim, "Neighbor table based shortcut tree routing in zigbee wireless networks," *IEEE Transactions on Parallel and Distributed Systems*, vol. 25, no. 3, pp. 706–716, 2014.
- [15] R. W. World. (2025) Zigbee mesh network tutorial: Basics and features. Accessed on 2025-06-20. [Online]. Available: <https://www.rfwireless-world.com/tutorials/zigbee-mesh-network-tutorial>
- [16] S. Labs. (2025) Flex sdk - radio abstraction interface layer (rail). Accessed on 2025-06-20. [Online]. Available: <https://www.silabs.com/developer-tools/flex-sdk-radio-abstraction-interface-layer>
- [17] ——. (2025) Efr32mg24 series 2 multiprotocol wireless soc. Accessed on 2025-06-20. [Online]. Available: <https://www.silabs.com/wireless/zigbee/efr32mg24-series-2-socs>
- [18] A. Theory. (2025) Pifa (planar inverted-f antenna). Accessed on 2025-06-20. [Online]. Available: <https://www.antenna-theory.com/antennas/patches/pifa.php>
- [19] Electrical4U. (2025) Impedance matching: Formula, circuit & applications. Accessed on 2025-06-20. [Online]. Available: <https://www.electrical4u.com/impedance-matching/>
- [20] P. Casey, K. Tepe, and N. Kar, "Design and implementation of a testbed for ieee 802.15.4 (zigbee) performance measurements," *EURASIP J. Wireless Comm. and Networking*, vol. 2010, 04 2010.
- [21] M. D. Abrignani, C. Buratti, D. Dardari, N. E. Rachkidy, A. Guitton, F. Martelli, A. Stajkic, and R. Verdone, "The EuWIn Testbed for 802.15.4/Zigbee Networks: From the Simulation to the Real World," in *ISWCS (International Symposium on Wireless Communication Systems)*, Ilmenau, Germany, 2013. [Online]. Available: <https://hal.science/hal-01789144>

- [22] F. Graf, T. Watteyne, F. Maksimovic, and M. Villnow, “Bit- and Symbol-Error Patterns in IEEE 802.15.4 TSCH Mode,” in *29th IEEE Symposium on Computers and Communications (ISCC)*, Paris, France, Jun. 2024. [Online]. Available: <https://hal.science/hal-04618351>
- [23] P. S. Foundation. (2025) sqlite3 — db-api 2.0 interface for sqlite databases. Accessed on 2025-06-20. [Online]. Available: <https://docs.python.org/3/library/sqlite3.html>
- [24] GeeksforGeeks. (2025) Flask tutorial. Accessed on 2025-06-20. [Online]. Available: <https://www.geeksforgeeks.org/python/flask-tutorial/>
- [25] S. Labs. (2025) Transmission test | efr32 rf evaluation guide | rail v2.19.0. Accessed on 2025-06-20. [Online]. Available: <https://docs.silabs.com/rail/2.19.0/efr32-rf-eval-guide/03-transmission-test>

PHỤ LỤC

A HƯỚNG DẪN NGƯỜI DÙNG SỬ DỤNG TESTBED

A.1 Hướng dẫn người dùng tích hợp module log và module controller vào firmware chính

A.1.1 Tích hợp module log vào firmware chính

Để tích hợp module log vào firmware của mình, người dùng làm theo hướng dẫn sau:

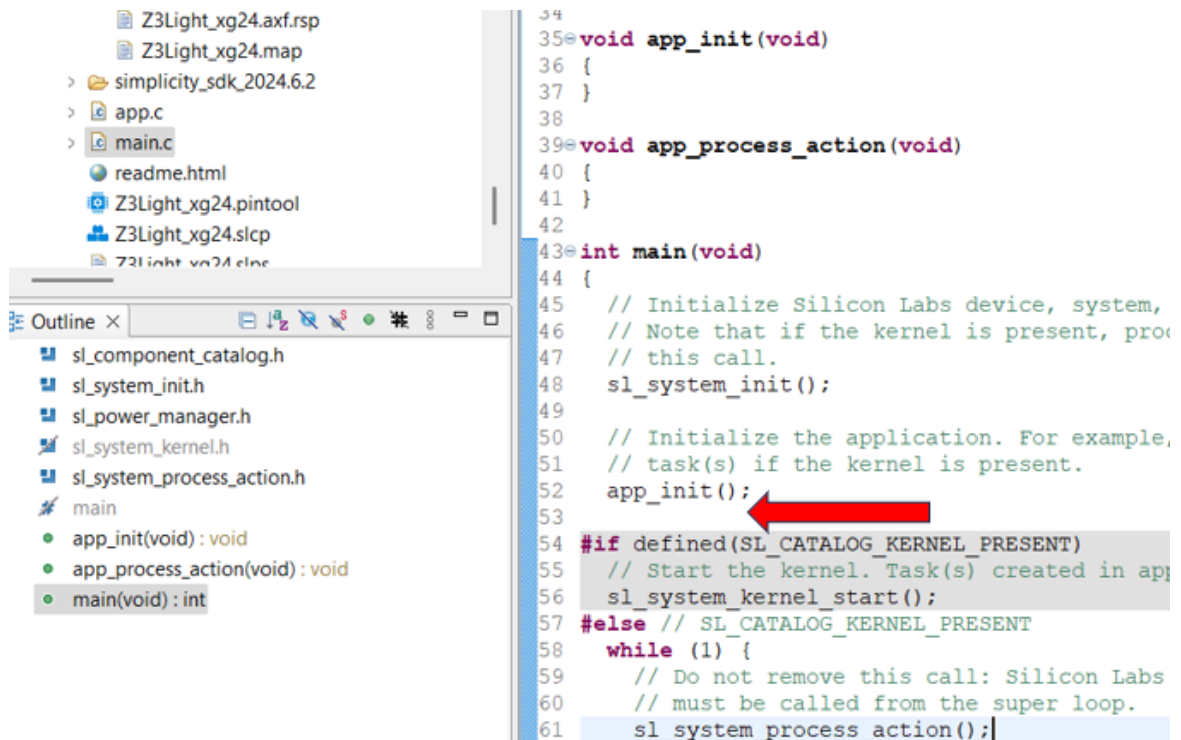
- Truy cập vào folder dự án trong Simplicity Studio, folder "autogen", tại đây tìm và chọn file "sl_rail_until_ieee802154_stack_event.c"
- Thay thế toàn bộ nội dung hiện tại bằng nội dung trong file tương ứng thuộc testbed. Hệ thống log gói tin tự động sẽ hoạt động độc lập và không gây ảnh hưởng đến firmware chính của người dùng

Hệ thống log gói tin tự động sẽ hoạt động độc lập và không gây ảnh hưởng đến firmware chính của người dùng

A.1.2 Tích hợp module controller vào firmware chính

Để tích hợp module controller vào hệ thống(để sử dụng chức năng quét mức năng lượng các kênh,...) người dùng làm theo hướng dẫn sau:

- Thêm thư viện "Add.h" vào thư mục có chứa main.c của dự án và include thư viện này trong file "main.c".
- Trong file "main.c" của dự án, hãy thêm hàm khởi "init_custom_cli_commands()"(là hàm khởi tạo module controller, được định nghĩa trong thư viện "Add.h") ngay sau hàm "app_init".
- Sau đó thêm hàm "Rail_ConfigEvents" để bật các cờ báo sự kiện RAIL.



Hình A.1 Vị trí thêm các hàm module controller

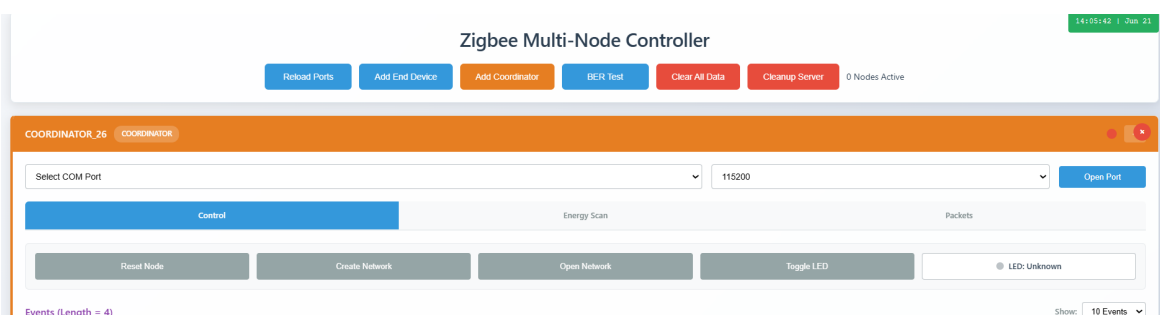
A.2 Hướng dẫn người dùng sử dụng giao diện web

A.2.1 Theo dõi thông tin log và gửi lệnh điều khiển

Sau khi thiết lập các thiết bị trong mạng đều kết nối UART đến máy tính trung tâm và tích hợp module log, module controller vào firmware chính của mình, người dùng có thể bắt đầu theo dõi thông tin các gói tin và thực hiện các chức năng gửi lệnh điều khiển.

Người dùng có thể bật Debug mode bằng 1,2 hoặc 3 để theo dõi và tương tác với hệ thống qua giao diện dòng lệnh.

Hoặc người dùng có thể vào đường link "http://localhost:5000" sau khi khởi chạy hệ thống để theo dõi, gửi lệnh điều khiển trực quan hơn, giao diện web như hình A.2



Hình A.2 Giao diện chính hệ thống

Ở trên cùng có các nút chức năng để người dùng có thể thêm thiết bị mới(Coordinator, End device) vào mạng, chức năng BER test,... Sau khi thêm thiết bị mới vào mạng,

Coordinator hay End device cũng đều được xây dựng các chức năng tương đương: Control, Energy Scan, Packet.

- Mục "Control" giúp người dùng có thể gửi lệnh điều khiển xuống kit tương ứng. Với Coordinator sẽ có các lệnh reset, tạo mạng, mở mạng, nháy LED. End device có các chức năng: reser, rời mạng, vào mạng, nháy LED.
- Mục "Energy scan" sẽ hiển thị mức năng lượng của các kênh từ 11 đến 26 sau khi nhấn nút "Energy Scan"
- Tại mục "Packets", người dùng có thể xem các thông tin chi tiết về các gói tin, khi ấn vào người dùng có thể xem phần giải mã cụ thể của gói tin đó

Data Packets (Length > 0)

Latest packets shown first. Click on any packet to view detailed analysis

ID64	TIME (MS) / REAL TIME	RSSI	LQI	CHANNEL	LENGTH	EVENT	PACKET TYPE	PAYLOAD
842E14FFFEF648FE	-1132739926 14:00:00.379	N/A	0	CH/N/A	46	TX_PENDING_MAC	Toggle Data	2F 41 88 2C CD AB FF FF 00 00 12 FC FF 00 00 01 C3 FE 4B FE FF 14 2E 84 28 DB 30 04 00 FE 4B F 6 FE FF 14 2E 84 00 24 4D 83 FD 45 8B
842E14FFFEF648FE	-1148503594 14:00:00.376	N/A	0	CH/N/A	46	TX_PENDING_MAC	Toggle Data	2F 41 88 2C CD AB FF FF 00 00 12 FC FF 00 00 01 C3 FE 4B FE FF 14 2E 84 28 DB 30 04 00 FE 4B F 6 FE FF 14 2E 84 00 29 A0 0B F3 D5 43

Hình A.3 Giao diện hiển thị thông số gói tin

A.2.2 Sử dụng chức năng BER test

Sau khi click vào mục "BER test" trên web, giao diện ở hình A.4 sẽ xuất hiện.

Global Test Parameters

Number of Bytes: 10000000

Frequency (Hz): 2405000000

Timeout (seconds): 1000

Node 1 (TX)

TX Mode

Sync Words Configuration: 500421792 0 32

Config Sync Words

Port Configuration: COM11 115200 Close Port

Node Controls: Config Sync Reset Mode

Node Status: Connected

Node 2 (RX)

RX Mode

Sync Words Configuration: 500421792 0 32

Config Sync Words

Port Configuration: COM19 115200 Close Port

Node Controls: Config Sync Reset Node

Node Status: Connected

Start BER Test

Make sure both nodes are connected and configured properly before starting the test.

Stop Test Clear Results

Test Progress & Results

37.75% Complete

BER RSSI: -41 dBm Bit Errors: 765 Bits Tested: 30,200,936 Bit Error Rate: 0.002533%

Hình A.4 Giao diện BER test

Tại đây, sau khi đã nạp cho kịch bản test BER, người dùng có thể cấu hình số bit muốn test, tần số truyền, thời gian time out và đặt biệt có thể cấu hình để có thể truyền nhận giữa 2 loại kit khác nhau (cùng sản xuất bởi Silicon Lab). Các bước thực hiện như sau:

- Number of Bytes: Người dùng chọn số bytes muốn thử(1 byte tương đương 8 bits)
- Frequency: Người dùng chọn tần số thử, từ 2405MHz đến 2500MHz
- Timeout: Người dùng chỉnh thời gian chờ tối đa cho quá trình đo, nếu hết thời gian này, dù chưa chạy xong nhưng hệ thống sẽ dừng chế độ ber test
- Mở kết nối với 2 thiết bị, từ đồng bộ sẽ được tự động cập nhật
- Người dùng cần chỉnh từ đồng bộ cho 2 thiết bị giống nhau thì ber test mới hoạt động được
- Người dùng bấm vào Start Ber Test để bắt đầu quá trình
- Bảng kết quả hiển thị tiến trình, trung bình của RSSI, số lỗi bit, số lượng bit đã truyền và tỉ lệ lỗi bit